

APOSTILA DO ALUNO

UC5 – Desenvolver banco de dados

Caso de estudo: Mercadinho Bom Preço

Curso Técnico em Desenvolvimento de Sistemas – Noturno

Instrutor: Higo Viganor Camporez

24 aulas de 3 horas · 72 horas

PostgreSQL (Supabase e instalação local)

Material didático elaborado para estudo. Todos os dados, nomes, CPFs e CNPJs usados nos exemplos são fictícios.

Sumário

Apresentação	3
Aula 1 – O caso, os dados e o papel do SGBD	8
Aula 2 – Ambiente em nuvem: Supabase.....	12
Aula 3 – Instalação e configuração local do PostgreSQL.....	15
Aula 4 – Modelagem conceitual I: entidades e relacionamentos	19
Aula 5 – Modelagem conceitual II: chaves, integridade e dicionário de dados	22
Aula 6 – Normalização.....	25
Aula 7 – Do modelo lógico ao modelo físico	29
Aula 8 – Checkpoint 1: apresentação e arguição do modelo	32
Aula 9 – DDL I: criação das estruturas	36
Aula 10 – DDL II: alteração de estruturas e reprodução no ambiente local	41
Aula 11 – DML: inserção, atualização e exclusão de dados	44
Aula 12 – Consultas I: seleção, filtros, ordenação e datas	51
Aula 13 – Consultas II: agregação e agrupamento	55
Aula 14 – Junções	58
Aula 15 – União, interseção e diferença de dados.....	62
Aula 16 – Subconsultas.....	65
Aula 17 – Views e índices	69
Aula 18 – Checkpoint 2: mudança de requisito e manutenção do banco.....	74
Aula 19 – Importação de dados.....	78
Aula 20 – Exportação de dados e migração	84
Aula 21 – Segurança do banco de dados.....	89
Aula 22 – Backup: planejamento e execução.....	98
Aula 23 – Restore: restauração e teste de recuperação	104
Aula 24 – Integração e entrega do pacote técnico.....	109
Apêndice A – Guia rápido de SQL e psql	115
Apêndice B – Modelo de referência do Mercadinho.....	118
Apêndice C – De Oracle para PostgreSQL: equivalências	122
Apêndice D – Modelos de documentos	124
Apêndice E – Glossário.....	126
Apêndice F – Referências.....	127

Apresentação

Bem-vindo à UC5

Esta apostila acompanha a Unidade Curricular **UC5 – Desenvolver banco de dados**, do curso Técnico em Desenvolvimento de Sistemas. Ao longo de **24 aulas de 3 horas (72 horas)** você vai trabalhar como integrante da equipe de dados de um comércio varejista fictício, o **Mercadinho Bom Preço**, e entregar, no final, um banco de dados completo: modelado, implementado, populado, protegido e com rotinas de backup e restore testadas.

Aqui você não vai apenas ler sobre bancos de dados. Em cada aula você vai **fazer, pensar sobre o que fez e melhorar o que fez**. É o ciclo ação – reflexão – nova ação, que organiza todos os capítulos desta apostila.

O que o Plano de Curso diz sobre esta UC

As informações abaixo foram extraídas do Plano de Curso e não foram alteradas.

Unidade Curricular	UC5 – Desenvolver banco de dados
Carga horária	72 horas
Curso	Técnico em Desenvolvimento de Sistemas (Habilitação Profissional Técnica de Nível Médio)

Indicadores

Os indicadores descrevem o que você precisa **saber fazer** ao final da UC. Neste material eles aparecem como I1 a I5.

	Indicador
I1	Constrói diagramas de dados aferindo a modelagem e estrutura do banco de dados, de acordo com as especificações técnicas.
I2	Realiza a instalação e configuração de sistema gerenciador de banco de dados (SGBD), de acordo com especificações técnicas.
I3	Elabora scripts SQL de construção, inserção e manipulação dos dados, conforme especificação técnica da linguagem SQL.
I4	Realiza importação e exportação de dados, conforme regras do sistema gerenciador de banco de dados (SGBD).
I5	Planeja rotinas de backup e restore da base de dados, conforme especificações técnicas do sistema gerenciador de banco de dados (SGBD).

Habilidades e atitudes/valores

Habilidades	Atitudes/Valores
H1 Criar a documentação do projeto.	V1 Cordialidade no trato com as pessoas.
H2 Modelar estrutura de banco de dados.	V2 Flexibilidade nas situações de trabalho.
H3 Organizar arquivos.	V3 Proatividade na resolução de problemas.
H4 Utilizar termos técnicos nas rotinas de trabalho.	V4 Sigilo no tratamento de dados e informações.

Conhecimentos

O Plano organiza os conhecimentos em três blocos. Cada capítulo indica quais deles são mobilizados.

- **C1 – Modelagem conceitual do banco de dados:** levantamento de dados e especificação de requisitos; dicionário de dados; integridade referencial; arquitetura de arquivos de dados; teoria dos conjuntos; tipos de dados; normalização; modelo de entidade e relacionamento (entidade, visão, atributos, índices, chaves candidata, primária e estrangeira, relacionamentos).
- **C2 – Modelagem física de banco de dados relacional:** linguagem SQL (histórico, definições e aplicabilidade); ferramentas de modelagem e manutenção; criação, manipulação, ordenação, listagens e operações em consultas; subconsultas; datas; união, intersecção e junção de dados; entidade, views, atributos, índices, chaves e integridade referencial; importação e exportação de dados.
- **C3 – Segurança do banco de dados:** normas; redundância, disponibilidade, integridade e consistência; criptografia; autenticação; falhas; ameaças; controle de acesso; backup e restore.

A escolha do ambiente de prática

Decisão didática desta UC

O Plano de Curso cita o SQL Plus e o SQL Developer como ferramentas de administração de objetos. Nesta UC, **por decisão didática do projeto pedagógico**, a prática é feita em **PostgreSQL**, usado de duas formas: em **nuvem**, por meio do **Supabase**, e **instalado no seu computador**. As ferramentas equivalentes são o **psql**, o **pgAdmin** ou o **DBeaver** e o **SQL Editor** do **Supabase**. Os indicadores e conhecimentos do Plano não mudam: o que você aprende vale para qualquer SGBD relacional.

O desafio da UC

Um comércio local, o **Mercadinho Bom Preço**, controla estoque, vendas, clientes e fornecedores em planilhas. Há produtos duplicados, preços diferentes para o mesmo item, nenhum histórico de vendas e dados de clientes expostos a quem recebe o arquivo.

Seu desafio, em dupla: projetar e implementar o banco de dados do Mercadinho e entregar um **pacote técnico** com:

1. diagramas e dicionário de dados;
2. scripts SQL versionados (construção, inserção, manipulação e consulta);
3. banco implementado no PostgreSQL (Supabase e instalação local);
4. procedimentos de importação e exportação de dados;
5. política de acesso;
6. plano de backup e restore, com teste executado;
7. apresentação técnica, com justificativa das decisões de modelagem e documentação dos procedimentos.

Todos os dados usados nesta UC são **fictícios**. Nomes, CPFs, CNPJs, telefones e e-mails foram inventados para o estudo.

Como esta apostila está organizada

A UC tem três etapas, que correspondem às três partes da apostila:

Parte	Aulas	O que você faz
Parte I – Compreender, preparar o ambiente e modelar	1 a 8	Analisa o caso, prepara os ambientes e constrói os diagramas e o dicionário de dados.

Parte	Aulas	O que você faz
Parte II – Construir e manipular	9 a 18	Cria o banco com SQL, insere e consulta dados, cria views e índices e atende a uma mudança de requisito.
Parte III – Movimentar, proteger e entregar	19 a 24	Importa e exporta dados, define acessos, planeja e testa backup e restore e apresenta o pacote técnico.

Cada capítulo corresponde a uma aula e segue a mesma estrutura:

Seção	Para que serve
Nesta aula	Mostra indicadores, conhecimentos e o que você vai entregar.
No caso do Mercadinho	Situa a aula no problema profissional.
Conceito, exemplo e demonstração	Explica o que você precisa saber, com exemplos.
Ação	Prática orientada: você faz.
Reflexão	Perguntas para analisar, comparar e justificar o que fez.
Nova ação	Você melhora, corrige ou amplia sua solução.
Desafio	Atividade extra para ir além.
Síntese e entrega	Resumo da aula e a evidência que você deve guardar.

Os quadros coloridos ajudam a ler:

Dica

Sugestões que facilitam o trabalho.

Atenção

Cuidados e erros comuns. Leia antes de executar.

Complementação didática

Conteúdo que vai além do texto do Plano de Curso e que contribui diretamente para um indicador.

Como você será avaliado

A avaliação é feita por **evidências** do que você produz e faz, relacionadas aos indicadores. Ao longo da UC o professor registra **menções por indicador**: durante o processo, **A** (alcançado), **PA** (parcialmente alcançado) ou **NA** (não alcançado); ao final, **A** ou **NA**. Para ser aprovado, você precisa alcançar todos os indicadores, e há recuperação imediata sempre que uma dificuldade é identificada. A frequência mínima exigida é de 75%.

Indicador	Principais evidências	Onde são observadas
I1	DER, modelo físico, dicionário de dados, aferição do banco contra o diagrama	Aulas 4 a 8, 9, 18 e 24
I2	Checklist de ambiente, instalação local documentada, acessos	Aulas 2, 3, 10, 20, 21, 23 e 24
I3	Scripts SQL de construção, inserção, manipulação e consulta; prática individual	Aulas 9 a 18 e 24
I4	Procedimento de importação e exportação, conferência de dados	Aulas 19, 20 e 24

Indicador	Principais evidências	Onde são observadas
I5	Plano de backup e restore e evidência do teste	Aulas 22, 23 e 24

Os **checkpoints** das aulas 8 e 18 são momentos formativos: servem para você saber como está e corrigir o rumo antes da avaliação final, na aula 24.

Combinados de trabalho

- **Dados fictícios sempre.** Nunca use dados reais de pessoas nos exercícios.
- **Credenciais são secretas.** Senhas do banco e do Supabase não vão para mensagens, prints compartilhados, repositórios ou documentos. Guarde em local seguro. Isso é parte do sigilo profissional (V4).
- **Cada um cria a própria conta no Supabase,** seguindo a orientação do professor sobre termos de uso.
- **Trabalho em dupla com responsabilidade individual.** Os dois devem entender e saber explicar tudo o que foi entregue.
- **Cordialidade e flexibilidade.** Revisar o trabalho do colega exige respeito; mudar de requisito faz parte da profissão.
- **Uso de IA.** Quando a atividade permitir, use como apoio e **valide tudo** executando e conferindo o resultado. Nunca envie senhas ou dados reais.

Organize o seu projeto desde já

Crie uma pasta para o projeto e mantenha esta estrutura durante toda a UC (habilidade H3 – organizar arquivos):

```
bom-preco-db/
├── 01_requisitos/      lista de requisitos, regras de negócio
├── 02_modelagem/      DER, modelo físico, dicionário de dados
├── 03_scripts/        scripts SQL numerados (01_..., 02_...)
├── 04_dados/          arquivos CSV de entrada e saída
├── 05_backup/         arquivos de backup e testes de restore
└── 06_documentacao/   checklists, política de acesso, plano de backup
```

Nomeie os arquivos com número de ordem e nome claro, por exemplo `01_criar_tabelas.sql`. Se o laboratório tiver controle de versão (como o Git), use-o para versionar a pasta.

Parte I – Compreender, preparar o ambiente e modelar

Aulas 1 a 8

Aula 1 – O caso, os dados e o papel do SGBD

Nesta aula

- **Etapa:** 1 – Compreender, preparar o ambiente e modelar
- **Indicador:** I1 (início)
- **Conhecimentos:** C1 (levantamento de dados e requisitos, arquitetura de arquivos de dados, tipos de dados); C2 (definições e aplicabilidade); introdução a C3 (redundância, integridade, consistência)
- **Habilidades:** H1, H4 · **Atitudes/valores:** V1, V4
- **Você vai entregar:** a lista de requisitos de dados (versão 2)

No caso do Mercadinho

Segunda-feira, 8h. O dono do **Mercadinho Bom Preço** recebe você e sua dupla e abre três planilhas: vendas, estoque e cadastro de clientes. "Funciona há anos", ele diz, "mas ninguém confia nos números". Seu trabalho, como equipe de dados, é entender o que existe antes de propor qualquer solução.

Conceito: dado, informação e banco de dados

Dado é um registro bruto, sem contexto: 27,90, Arroz, 03/08/2026. **Informação** é o dado interpretado em um contexto: "o pacote de arroz de 5 kg custava R\$ 27,90 em 3 de agosto de 2026".

Um **banco de dados** é uma coleção organizada de dados relacionados, armazenada para ser consultada e atualizada de forma confiável. O **Sistema Gerenciador de Banco de Dados (SGBD)** é o software que cuida disso: guarda os dados em arquivos, controla quem pode acessá-los, garante regras de integridade, permite várias pessoas trabalhando ao mesmo tempo e recupera os dados após falhas. Neste curso, o SGBD é o **PostgreSQL**.

Por que não continuar com planilhas?

Planilhas são ótimas para cálculos e pequenas listas. O problema aparece quando elas viram o "sistema" do negócio. Compare:

Aspecto	Planilha / arquivos soltos	SGBD
Redundância	O mesmo dado é digitado várias vezes (o nome do fornecedor em cada linha de venda).	Cada dado é armazenado uma vez e referenciado onde preciso.
Consistência	Uma cópia é corrigida e outra não: o preço fica diferente em dois lugares.	Uma só fonte de verdade.
Integridade	Qualquer um digita qualquer coisa ("três" na coluna de estoque).	Regras do banco impedem dados inválidos.
Sigilo e controle de acesso	Quem recebe o arquivo vê tudo.	Cada pessoa vê e altera só o que lhe cabe.
Disponibilidade e recuperação	Se o arquivo se corrompe ou é apagado, perde-se o histórico.	Backup e restore planejados.
Concorrência	Duas pessoas editando a cópia do outro.	Várias conexões simultâneas, com controle.

Arquitetura de arquivos de dados: da pasta de arquivos ao SGBD

Antes dos SGBDs, cada sistema gravava seus próprios arquivos, em formato próprio. Isso criava **ilhas de dados**: o cadastro de clientes da loja era diferente do cadastro do financeiro. Em um SGBD, os dados ficam em arquivos gerenciados pelo próprio sistema (você não abre esses arquivos diretamente), e todos os programas e pessoas os acessam por meio do SGBD, em geral usando a linguagem **SQL**. Na aula 3 você vai ver onde o PostgreSQL guarda esses arquivos.

Tipos de dados (primeiro contato)

Todo dado tem um **tipo**, que define que valores aceita e que operações permite: números inteiros, números com casas decimais, textos, datas, verdadeiro/falso. Escolher o tipo certo evita erros: um preço guardado como texto não pode ser somado. Voltaremos a isso nas aulas 5 e 7.

SQL: histórico, definições e aplicabilidade

O **modelo relacional**, que organiza dados em tabelas (relações) ligadas entre si, foi proposto por Edgar F. Codd em 1970. Pouco depois surgiu uma linguagem para trabalhar com ele, que deu origem à **SQL** (Structured Query Language). A SQL foi padronizada pela ANSI em 1986 e pela ISO em 1987 e é, até hoje, a linguagem comum dos bancos relacionais. O PostgreSQL descende do projeto POSTGRES, da Universidade da Califórnia em Berkeley, e adota o nome PostgreSQL desde 1996.

A SQL se aplica em praticamente todo sistema que guarda dados: lojas virtuais, aplicativos bancários, sistemas hospitalares, aplicativos móveis. Quem desenvolve sistemas precisa saber conversar com um banco.

Levantamento de dados e requisitos

Antes de modelar, é preciso descobrir **o que o negócio precisa guardar**. Dois conceitos ajudam:

- **Requisito de dados**: o que o sistema precisa armazenar. Exemplo: "Armazenar nome, telefone e cidade de cada cliente."
- **Regra de negócio**: uma condição do negócio que os dados precisam respeitar. Exemplo: "O preço de venda de um produto deve ser maior que zero."

Uma boa lista de requisitos é **clara, numerada, verificável** e escrita com a linguagem do negócio. Quando algo não estiver claro nas planilhas, anote como **pergunta ao cliente**: não invente a regra.

As planilhas do Mercadinho

Planilha 1 – Vendas (trecho)

Data	Cliente	Telefone	Produto	Qtd	Preço	Fornecedor
03/08/2026	Ana Souza	27 99900-0001	Arroz 5kg	1	26,90	Grão Dourado
03/08/2026	Ana B. Souza	279990000001	ARROZ 5 KG	2	27,90	Grão Dourado
05/08/2026	Bruno Carvalho		Feijão carioca 1kg	3	8,49	Grão Dourado
07/08/2026	Carla Mendes	27 99900-0003	Suco uva 1L	2	12,90	Bebidas Litoral
07/08/2026	Carla Mendes	27 99900-0033	Suco de uva integral 1 L	1	13,50	Bebidas Litoral S.A.
10/08/2026	Diego Almeida	27 99900-0004	Sabonete	6	2,49	Limpeza Brilho
12/08/2026	Eduarda Lima		Tomate kg	2	8,99	Vale Verde

Data	Cliente	Telefone	Produto	Qtd	Preço	Fornecedor
12/08/2026			Pão francês	1	16,90	Pão & Cia

Planilha 2 – Estoque (trecho)

Produto	Estoque	Mínimo	Obs.
Açúcar 1kg	8	15	repor
Acucar refinado 1 kg	20		sobra do mês passado
Macarrão 500g	três	10	
Sabonete 85g	6	20	repor
Tomate	4	8	repor urgente

Planilha 3 – Cadastro de clientes (trecho)

Nome	CPF	Cidade	Obs.
Ana Beatriz Souza	10000000001	Vitória	paga no Pix; senha do app: ana2026
Bruno Carvalho	10000000002	Vila Velha	
Carla Mendes	10000000003	Serra	recebe promoções por e-mail

Atenção

Estas planilhas são compartilhadas com todos os funcionários por um link aberto. Qualquer pessoa que receba o link pode ver e editar tudo.

Ação

Em dupla, analise as três planilhas e preencha um quadro como o do modelo abaixo (faça no seu editor de texto ou planilha, na pasta [01_requisitos/](#)).

Dado / informação	Onde aparece	Problema encontrado	Tipo de problema
Preço do arroz	Planilha 1, linhas 1 e 2	26,90 e 27,90 para o mesmo produto	Inconsistência

Depois, liste:

1. as **entidades** (as "coisas" sobre as quais o negócio guarda dados): por exemplo, clientes;
2. os **dados** de cada entidade;
3. as **regras de negócio** que você consegue deduzir (ou perguntas que faria ao dono).

Use estes tipos de problema para classificar: redundância, inconsistência, falta de integridade, exposição de dados (sigilo) e risco de disponibilidade.

Reflexão

Discuta com a turma, justificando com exemplos das planilhas:

1. Quais problemas poderiam ser evitados por um SGBD? Quais dependem também de **regras e de pessoas**?

2. Por que "Ana Souza" e "Ana B. Souza" é um problema para saber quantas compras ela fez?
3. Quem **não deveria** ver a coluna "Obs." do cadastro de clientes? Que dado ali nunca deveria ser guardado?
4. O que acontece com o negócio se o arquivo for apagado hoje? O que vocês gostariam de ter?

Nova ação

Reescreva sua lista como **requisitos de dados (RD)** e **regras de negócio (RN)**, numerados, incorporando o que a turma discutiu. Use este formato (exemplos de outro negócio, uma papelaria):

RD01 – O sistema deve armazenar, para cada produto, nome, preço e quantidade em estoque.
RD02 – O sistema deve registrar cada venda com data, cliente e itens vendidos.
RN01 – Não é permitido vender um produto com quantidade menor ou igual a zero.
RN02 – Cada produto pertence a uma única categoria.
Pergunta ao cliente: um produto pode ter mais de um fornecedor?

Entregue a **versão 2** da lista, com pelo menos 10 requisitos de dados, 6 regras de negócio e as perguntas em aberto.

Desafio

Escolha um sistema que você usa no dia a dia (um aplicativo de entregas, de transporte, de streaming). Liste cinco dados que ele precisa guardar, duas regras de negócio e um risco de sigilo se esses dados vazarem.

Síntese e entrega

- Um **SGBD** reduz redundância, protege a integridade, controla acessos e permite recuperar dados.
- **Requisitos de dados** dizem o que guardar; **regras de negócio** dizem o que precisa ser respeitado.
- Dúvidas viram **perguntas ao cliente**, nunca suposições escondidas.

Evidência desta aula

Lista de requisitos de dados e regras de negócio, versão 2, em [01_requisitos/](#). Esta aula inclui também uma **avaliação diagnóstica** (questionário e observação), que serve para o professor conhecer a turma. Ela não gera menção.

Aula 2 – Ambiente em nuvem: Supabase

Nesta aula

- **Etapa:** 1 – Compreender, preparar o ambiente e modelar
- **Indicador:** I2 (principal)
- **Conhecimentos:** C2 (histórico e ferramentas de modelagem e manutenção); C3 (autenticação); C1 (arquitetura, no nível conceitual)
- **Habilidades:** H3, H4 · **Atitudes/valores:** V3, V4
- **Você vai entregar:** o checklist de configuração do ambiente

No caso do Mercadinho

O dono já entendeu que as planilhas não bastam, mas não quer comprar um servidor agora. A equipe de dados vai construir o banco em um **serviço em nuvem**, sem precisar de uma máquina dedicada no mercado. Antes de modelar qualquer coisa, é preciso ter onde guardar o banco.

Conceito: modelo cliente-servidor e serviço gerenciado

Um SGBD funciona no modelo **cliente-servidor**: o **servidor** é o programa que guarda e protege os dados; os **clientes** (programas como o `psql`, o pgAdmin, o DBeaver ou uma aplicação) se conectam a ele pela rede e enviam comandos SQL. Para se conectar, o cliente precisa de cinco informações: **servidor (host)**, **porta**, **nome do banco**, **usuário** e **senha**.

Um **serviço gerenciado** (às vezes chamado de banco como serviço) é um PostgreSQL que alguém opera para você. O **Supabase** oferece um banco PostgreSQL completo em nuvem, com uma interface web para executar SQL (o **SQL Editor**) e para ver tabelas (o **Table Editor**).

Quem faz o quê?

No serviço gerenciado, o provedor cuida de instalar o servidor, mantê-lo ligado e atualizá-lo. **Continuam sendo responsabilidade do técnico:** a senha e os acessos, o que existe dentro do banco, a organização dos scripts e, principalmente, **a certeza de que existe cópia de segurança dos dados** (aula 22).

Anatomia de uma conexão

Uma conexão com PostgreSQL costuma ser descrita em uma **cadeia de conexão** (connection string):

```
postgresql://USUARIO:SENHA@HOST:PORTA/BANCO
```

No Supabase existem diferentes formas de conexão, que você encontra no botão **Connect** do projeto. Segundo a documentação do Supabase, em resumo:

Forma de conexão	Porta	Quando usar
Direta (<code>db.[ref].supabase.co</code>)	5432	Servidores e ferramentas com suporte a IPv6; usada em migrações e backups. Em IPv4 só com recurso adicional pago.
Pooler em modo sessão (<code>aws-...pooler.supabase.com</code>)	5432	Redes só com IPv4 , ferramentas de terceiros e clientes de banco. O usuário tem o formato <code>postgres.[ref]</code> .
Pooler em modo transação	6543	Aplicações de vida curta (funções serverless). Não suporta comandos preparados.

Dica

Se o laboratório ou a sua rede só tem IPv4, use a conexão pelo **pooler em modo sessão**. Teste a conexão com o professor antes do dia da prática. Os nomes de menus e opções do Supabase mudam com o tempo: siga as telas atuais e a documentação do serviço.

Ação

Siga as etapas. O professor orienta sobre termos de uso e criação de conta.

1. **Crie sua conta** no Supabase e, se solicitado, uma organização.
2. **Crie um projeto** novo com:
 - nome **bom-preco-** seguido do seu nome (por exemplo, **bom-preco-maria**);
 - **senha do banco forte** (gere e guarde em local seguro, como um gerenciador de senhas; nunca em arquivos compartilhados);
 - região mais próxima disponível;
 - plano gratuito.
3. Aguarde o projeto ficar pronto e abra o **SQL Editor**.
4. Execute o script de teste abaixo e confira se obteve resultado:

```
SELECT version();
SELECT current_database() AS banco, current_user AS usuario, now() AS agora;
SHOW server_encoding;
SHOW timezone;
```

1. Verifique se você consegue criar e apagar uma tabela de teste (isso confirma permissão para comandos de definição):

```
CREATE TABLE teste_conexao (id integer, mensagem text);
INSERT INTO teste_conexao VALUES (1, 'Olá, banco de dados!');
SELECT * FROM teste_conexao;
DROP TABLE teste_conexao;
```

1. Clique em **Connect** e localize a cadeia de conexão do **pooler em modo sessão**. Anote, em um arquivo na pasta **06_documentacao/**, o **host**, a **porta** e o **usuário**. **Não anote a senha**.
2. Se o seu computador tiver o **psql**, teste a conexão pelo terminal (substitua os trechos entre colchetes; o **psql** pedirá a senha):

No terminal

```
psql "postgres://postgres.[REF]@[HOST-DO-POOLER]:5432/postgres"
```

Ao entrar, digite **\conninfo** para ver os dados da conexão e **\q** para sair.

Atenção

Mantenha dados reais e sensíveis fora do banco de estudo. Projetos do plano gratuito podem ter limites de uso e podem ser pausados por inatividade. Consulte a política vigente do serviço e guarde seus scripts no seu computador, não só no navegador.

Reflexão

Em dupla, comparem e registrem por escrito:

1. O que o Supabase fez por vocês que, em um servidor próprio, precisaria ser feito pelo técnico?
2. O que **continua sendo responsabilidade de vocês**? Por que a senha do banco é tão importante?
3. Que riscos existem em um banco acessível pela internet? Como reduzir (senha forte, conexão segura, menor exposição)?
4. Qual a vantagem de ter também um PostgreSQL instalado no computador (aula 3)?

Nova ação

Elabore o **checklist de configuração do ambiente** (em `06_documentacao/`) e aplique os cuidados com credenciais. Use esta base:

Item	Feito	Evidência
Conta criada e projeto <code>bom-preco-...</code> ativo		print da tela do projeto (sem a senha)
Senha do banco guardada em local seguro		descrição de onde está guardada (não a senha)
Script de teste executado no SQL Editor		print do resultado
Tabela de teste criada e apagada		print
Dados de conexão (host, porta, usuário) anotados		arquivo <code>conexao.txt</code> sem a senha
Conexão testada por um cliente local, se disponível		resultado de <code>\conninfo</code>

Desafio

Pesquise na documentação do Supabase e descreva, em cinco linhas, a diferença entre a conexão direta e o pooler em modo sessão. Indique qual funcionaria melhor na rede do seu laboratório e por quê.

Síntese e entrega

- O SGBD funciona em **cliente-servidor**; uma conexão exige host, porta, banco, usuário e senha.
- O serviço gerenciado poupa trabalho de infraestrutura, mas **segurança e backup continuam sendo seus**.
- Senhas são credenciais: **nunca** vão para locais compartilhados.

Evidência desta aula

Checklist de configuração do ambiente preenchido, com as evidências (prints sem senha) em `06_documentacao/`. O professor observa a configuração e dá feedback; este é o primeiro registro do **indicador I2**.

Aula 3 – Instalação e configuração local do PostgreSQL

Nesta aula

- **Etapas:** 1 – Compreender, preparar o ambiente e modelar
- **Indicador:** I2 (principal)
- **Conhecimentos:** C1 (arquitetura de arquivos de dados); C2 (ferramentas de modelagem e manutenção); C3 (autenticação, controle de acesso)
- **Habilidades:** H1, H3, H4 · **Atitudes/valores:** V2, V3
- **Você vai entregar:** o procedimento de instalação e configuração, com evidências

No caso do Mercadinho

O dono pergunta: "e se a internet do mercado cair? E se eu quiser testar uma mudança sem mexer no banco que está valendo?" A equipe decide ter **também um PostgreSQL instalado no computador**. Ele será usado para ensaiar mudanças, receber a migração dos dados (aula 20) e, mais adiante, testar a recuperação de um backup (aula 23).

Conceito: instalar e configurar um SGBD

Instalar é colocar o software no computador. **Configurar** é deixá-lo pronto para o uso: definir quem pode se conectar, em que porta, onde guardar os dados e qual será o banco do projeto.

O que o instalador do PostgreSQL cria

Uma instalação típica contém:

- o **servidor** (processo que escuta conexões, por padrão na porta **5432**);
- um **diretório de dados** (**PGDATA**), onde ficam os arquivos do banco;
- o cliente de linha de comando **psql** e outras ferramentas (**pg_dump**, **pg_restore**);
- opcionalmente, o **pgAdmin**, uma ferramenta gráfica de administração;
- um usuário administrador, o **postgres** (superusuário), com a senha que você define na instalação.

Arquitetura de arquivos de dados do PostgreSQL

Dentro do diretório de dados existem, entre outros:

Item	Função
base/	Arquivos de dados dos bancos. Cada banco tem uma pasta, identificada por um número (OID). As tabelas são gravadas em páginas de 8 KB.
pg_wal/	Log de transações (WAL): registro das mudanças antes de irem para os arquivos de dados. É o que permite recuperar o banco após uma falha.
postgresql.conf	Configurações do servidor (porta, memória, endereços em que escuta).
pg_hba.conf	Regras de autenticação : quem pode conectar, de onde, a quais bancos e com que método.
global/	Dados compartilhados por todos os bancos, como a lista de papéis (usuários e grupos).

Atenção

Nunca copie, apague ou edite à mão os arquivos do diretório de dados com o servidor em funcionamento. Para copiar o banco, use as ferramentas de backup (aula 22).

Papéis, autenticação e controle de acesso

No PostgreSQL, usuários e grupos são **papéis** (*roles*). Um papel com a opção **LOGIN** pode se conectar. Regra de ouro: **o superusuário `postgres` serve para administrar o servidor; o dia a dia do projeto é feito com um papel próprio**, com menos poderes. Isso é o princípio do **menor privilégio**, que você aprofunda na aula 21.

Ação

Instale o PostgreSQL e um cliente gráfico no seu computador. Se o laboratório não permitir instalação (falta de permissão de administrador), o professor indicará uma alternativa.

Windows

1. Baixe o instalador do PostgreSQL na página oficial de downloads (postgresql.org/download), versão 16 ou superior.
2. Na instalação, mantenha selecionados **PostgreSQL Server**, **pgAdmin 4** e **Command Line Tools**.
3. Defina a **senha do superusuário `postgres`** (guarde em local seguro), mantenha a **porta 5432** e conclua a instalação.

Linux (Debian/Ubuntu)**No terminal**

```
sudo apt update
sudo apt install postgresql postgresql-contrib
sudo systemctl status postgresql
sudo -u postgres psql
```

Em outros sistemas, siga a orientação do professor.

Criar o papel e o banco do projeto. Conectado como `postgres` (no Windows, abra o *SQL Shell (psql)* ou use o pgAdmin; no Linux, o último comando acima já abre o `psql`), execute:

Executar como administrador do servidor

```
CREATE ROLE comercio_app LOGIN PASSWORD 'troque_esta_senha';
CREATE DATABASE comercio OWNER comercio_app;
```

Atenção

Troque `troque_esta_senha` por uma senha própria e forte. Esta é uma senha do **seu ambiente local de estudo**, não uma senha de produção.

Conectar com o papel do projeto e explorar. Em um terminal, conecte-se ao banco novo:

No terminal

```
psql -h localhost -U comercio_app -d comercio
```

Experimente os comandos do `psql` (começam com `\`):

Comando	O que faz
<code>\conninfo</code>	Mostra dados da conexão atual.

Comando	O que faz
\l	Lista os bancos.
\c banco	Conecta a outro banco.
\dt	Lista as tabelas.
\d tabela	Descreve uma tabela.
\du	Lista papéis (usuários e grupos).
\i arquivo.sql	Executa um arquivo de script.
\q	Sai do <code>psql</code> .

Execute algumas consultas de reconhecimento do servidor:

```
SHOW port;
SHOW server_encoding;
SELECT version();
SELECT datname, oid FROM pg_database WHERE datname = 'comercio';
```

O número (`oid`) é o nome da pasta do seu banco dentro de `base/`. Se você usar um papel de administrador, também consegue ver onde ficam os arquivos do servidor:

```
SHOW data_directory;
SHOW config_file;
SHOW hba_file;
```

Configurar o cliente gráfico. No pgAdmin (ou DBeaver), registre uma conexão com host `localhost`, porta `5432`, banco `comercio`, usuário `comercio_app`. Execute no editor de consultas dele a consulta `SELECT current_user;`.

Reflexão

Compare o PostgreSQL local com o Supabase e registre as conclusões:

Aspecto	Supabase (nuvem)	PostgreSQL local
Quem instala e atualiza o servidor?		
Onde ficam os arquivos de dados?		
Como é a autenticação?		
Funciona sem internet?		
Quem faz o backup?		

Depois, discuta: por que não usar o superusuário `postgres` para tudo? Por que, por padrão, um servidor instalado no computador só aceita conexões do próprio computador?

Nova ação

Algo deve ter dado errado em alguma máquina da dupla (senha digitada errada, porta ocupada, serviço parado, `psql` fora do caminho de comandos). **Corrija as falhas e registre o procedimento** em `06_documentacao/instalacao_postgresql.md` com este modelo:

```
Registro de instalação - PostgreSQL local
Sistema operacional: ...
Versão do PostgreSQL: ...
Porta: ...
Banco do projeto: comercio      Papel do projeto: comercio_app
Passos executados: 1) ... 2) ... 3) ...
Problemas encontrados e como foram resolvidos: ...
Evidências (prints): conexão pelo psql; conexão pelo pgAdmin/DBaver
```

Desafio

Descubra, na documentação do PostgreSQL, para que serve o parâmetro `listen_addresses` e qual o valor padrão. Explique, em duas frases, o risco de configurá-lo para aceitar conexões de qualquer endereço sem outras proteções.

Síntese e entrega

- Instalar é colocar o software; **configurar** é deixá-lo pronto, seguro e com o banco do projeto.
- O diretório de dados guarda os arquivos do banco, o log de transações (WAL) e os arquivos de configuração e autenticação.
- O dia a dia do projeto usa um papel próprio, não o superusuário.

Evidência desta aula

Procedimento de instalação documentado e conexão validada por `psql` e por uma ferramenta gráfica. O professor registra o **primeiro resultado parcial do indicador I2** (A, PA ou NA) e dá feedback.

Aula 4 – Modelagem conceitual I: entidades e relacionamentos

Nesta aula

- **Etapas:** 1 – Compreender, preparar o ambiente e modelar
- **Indicador:** I1 (principal)
- **Conhecimentos:** C1 (modelo entidade-relacionamento; teoria dos conjuntos aplicada à cardinalidade)
- **Habilidades:** H1, H2 · **Atitudes/valores:** V1, V2
- **Você vai entregar:** o DER do caso, versões 1 e 2

No caso do Mercadinho

Com a lista de requisitos nas mãos, a equipe de dados precisa de um **desenho** que todos entendam: o dono, os vendedores e quem vai programar. Um bom diagrama conversa melhor do que dez páginas de texto.

Conceito: modelagem de dados

Modelar é representar, de forma simplificada, os dados do negócio. Existem três níveis de modelo:

Nível	Pergunta que responde	Depende do SGBD?
Conceitual	Que dados o negócio tem e como se relacionam?	Não
Lógico	Como organizar em tabelas, chaves e relações?	Não (relacional)
Físico	Como criar no SGBD escolhido, com tipos e restrições?	Sim

Nesta aula você trabalha o **modelo conceitual**, que se representa com o **Diagrama Entidade-Relacionamento (DER)**.

Elementos do modelo entidade-relacionamento

- **Entidade:** algo do negócio sobre o qual guardamos dados (cliente, produto, pedido). Cada ocorrência é uma **instância** (a cliente "Ana", o produto "Arroz 5 kg").
- **Atributo:** característica de uma entidade (nome, preço, data). Há atributos **obrigatórios** e **opcionais**; um atributo **identificador** distingue uma instância das outras.
- **Relacionamento:** associação entre entidades ("cliente **faz** pedido", "fornecedor **fornece** produto").
- **Cardinalidade:** quantas instâncias de uma entidade podem se associar a cada instância da outra.

Cardinalidade e teoria dos conjuntos

Uma entidade pode ser vista como um **conjunto** de instâncias, e um relacionamento, como uma **ligação entre elementos de dois conjuntos**. A cardinalidade descreve quantos elementos de um conjunto se ligam a cada elemento do outro:

Cardinalidade	Leitura	Exemplo
1:1	Cada A se liga a no máximo um B, e vice-versa.	Pessoa e passaporte.

Cardinalidade	Leitura	Exemplo
1:N	Cada A se liga a vários B, mas cada B a um só A.	Um cliente faz vários pedidos; cada pedido é de um só cliente.
N:N	Cada A se liga a vários B, e cada B a vários A.	Um pedido tem vários produtos; um produto está em vários pedidos.

Para descobrir a cardinalidade, **pergunte nos dois sentidos**: "um cliente pode fazer quantos pedidos?" e "um pedido pode ser de quantos clientes?"

Relacionamento N:N

Um relacionamento N:N não pode ser guardado diretamente em um banco relacional. Ele é resolvido criando uma **entidade associativa**, que fica entre as duas e guarda os dados da ligação (por exemplo, a quantidade de cada produto em cada pedido).

Demonstração: biblioteca escolar

Para treinar sem antecipar o seu caso, veja o método em outro negócio: a biblioteca de uma escola. Requisitos: "A biblioteca registra os alunos, os livros e os empréstimos. Cada empréstimo é feito por um aluno para um livro, em uma data, e é devolvido em outra data."

Passo 1 – Entidades (os substantivos): aluno, livro, empréstimo. **Passo 2 – Atributos**: aluno (matrícula, nome, turma); livro (título, autor); empréstimo (data de empréstimo, data de devolução). **Passo 3 – Relacionamentos** (os verbos): aluno **faz** empréstimo; livro **é emprestado** em empréstimo. **Passo 4 – Cardinalidades**: um aluno faz vários empréstimos, e cada empréstimo é de um aluno (1:N). Um livro é emprestado várias vezes, e cada empréstimo é de um livro (1:N).

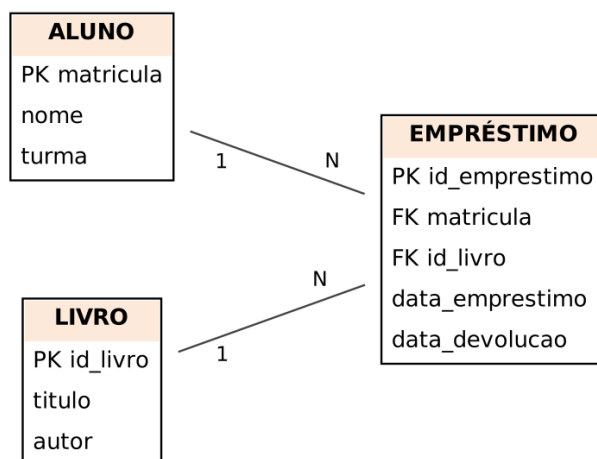


Figura 1 – DER do exemplo da biblioteca escolar (o 1 e o N indicam a cardinalidade de cada lado).

Repare que **empréstimo** nasceu como uma entidade própria porque guarda dados da ligação (datas), e que o identificador de cada entidade foi indicado (PK) e as ligações aparecem como chaves estrangeiras (FK). Você vai estudar chaves com mais detalhe na aula 5.

Ferramentas de diagramação

Use a ferramenta indicada pelo professor. Exemplos: diagrams.net (draw.io), dbdiagram.io, a ferramenta de ERD do pgAdmin ou o diagrama do DBeaver. O importante é que o diagrama seja claro, com **nomes padronizados** e **cardinalidades visíveis**.

Ação

Com a lista de requisitos da aula 1 (versão 2) em mãos, **desenhe o DER versão 1** do Mercadinho Bom Preço. Siga o roteiro:

1. Sublinhe na lista de requisitos os **substantivos** (candidatos a entidade) e descarte os que são apenas atributos de outro.
2. Para cada entidade, liste os **atributos** e marque o identificador.
3. Identifique os **relacionamentos** (os verbos) e dê um nome a cada um.
4. Defina a **cardinalidade** perguntando nos dois sentidos.
5. Resolva qualquer relacionamento **N:N** com uma entidade associativa.
6. Desenhe no software escolhido e salve o arquivo em `02_modelagem/` como `der_v1`.

Reflexão

Troque o DER com outra dupla e faça uma **revisão por pares**, registrando por escrito, com respeito e justificativa técnica:

- Há alguma entidade ausente, desnecessária ou duplicada?
- Cada cardinalidade está correta? Que pergunta você faria para confirmar?
- Algum atributo está na entidade errada (por exemplo, o telefone do fornecedor dentro do produto)?
- Os nomes são claros e padronizados?

Dê ao menos um ponto positivo e uma sugestão de melhoria.

Nova ação

Elabore a **segunda versão do DER** a partir do feedback recebido. Em `02_modelagem/`, anote em um arquivo `alteracoes_der.md` o que mudou da v1 para a v2 e por quê (uma linha por alteração).

Desafio

O dono do Mercadinho quer começar a registrar **entregas a domicílio**. Pense em como isso mudaria o seu DER: nova entidade? Novos atributos? Que perguntas você faria ao cliente antes de modelar? Anote suas ideias, sem alterar o diagrama ainda.

Síntese e entrega

- O DER representa **entidades**, **atributos** e **relacionamentos** com suas **cardinalidades**.
- Cardinalidade se descobre perguntando **nos dois sentidos**.
- Relacionamento **N:N** vira uma entidade associativa.

Evidência desta aula

`der_v1` e `der_v2` em `02_modelagem/`, com o registro de alterações. O professor avalia com um checklist do indicador I1 e dá feedback.

Aula 5 – Modelagem conceitual II: chaves, integridade e dicionário de dados

Nesta aula

- **Etapas:** 1 – Compreender, preparar o ambiente e modelar
- **Indicador:** I1 (principal)
- **Conhecimentos:** C1 (chaves candidata, primária e estrangeira; integridade referencial; dicionário de dados; índices; visão; tipos de dados)
- **Habilidades:** H1, H2, H4 · **Atitudes/valores:** V1
- **Você vai entregar:** o DER com chaves e o dicionário de dados

No caso do Mercadinho

Na segunda reunião, o dono conta uma história: "semana passada apaguei um cliente da planilha e perdi o histórico de compras dele". Para evitar isso, o banco precisa **saber** que um pedido não pode existir sem cliente, e que um cliente com pedidos não pode simplesmente desaparecer. É para isso que servem as **chaves** e a **integridade referencial**.

Conceito: chaves

Uma **chave** é um conjunto de atributos que identifica ou liga linhas.

Tipo de chave	O que é	Exemplo no Mercadinho
Chave candidata	Atributo (ou conjunto) que poderia identificar uma linha de forma única.	Em <i>cliente</i> : cpf , ou um número de cadastro.
Chave primária (PK)	A candidata escolhida para identificar a linha. Não pode ser vazia nem repetida.	id_cliente
Chave alternativa	Candidata não escolhida. Continua única.	cpf
Chave estrangeira (FK)	Atributo que referencia a chave primária de outra tabela.	id_cliente em <i>pedido</i>
Chave composta	Chave formada por mais de um atributo.	(id_pedido, id_produto) em <i>item do pedido</i>

Chave natural ou substituta? Uma chave **natural** é um dado do negócio (CPF, CNPJ, código de barras). Uma chave **substituta** (*surrogate key*) é um número gerado pelo banco, sem significado de negócio (**id_cliente**). A substituta é estável: o CPF pode ser digitado errado e corrigido, mas o número gerado nunca muda. Por isso, é comum usar uma chave substituta como PK e manter a natural como **chave alternativa** única.

Conceito: integridade

Integridade é a garantia de que os dados são válidos e coerentes. Existem quatro formas:

Integridade	Regra	Como é garantida
De entidade	Toda linha tem uma PK única e não vazia.	Chave primária.

Integridade	Regra	Como é garantida
Referencial	Toda FK aponta para uma linha que existe (ou é vazia, se permitido).	Chave estrangeira.
De domínio	O valor está no conjunto permitido (tipo, tamanho, faixa).	Tipo de dado, NOT NULL , CHECK .
De regras de negócio	O dado respeita as regras do negócio.	CHECK , UNIQUE e outras restrições.

Integridade referencial e o que fazer ao excluir

Na integridade referencial, o que acontece se alguém tentar **excluir** um cliente que tem pedidos? O modelo precisa definir:

- **Bloquear** a exclusão (padrão e mais seguro): o banco recusa enquanto houver pedidos.
- **Em cascata**: excluir também os pedidos do cliente (perigoso; só faz sentido para dados "filhos" sem valor próprio, como os itens de um pedido).
- **Anular**: colocar vazio na FK (só quando o atributo for opcional).

Visão e índice (primeiro contato)

- **Visão (view)**: uma "tabela virtual", definida por uma consulta, que simplifica relatórios e controla o que cada pessoa pode ver. Você cria views na aula 17.
- **Índice**: estrutura auxiliar que acelera buscas em uma coluna, como o índice remissivo de um livro. A chave primária e as chaves únicas já geram índices.

Tipos de dados e o dicionário de dados

O **dicionário de dados** é o documento que descreve cada tabela e cada campo: significado, tipo, tamanho, obrigatoriedade, chaves e regras. É a "bula" do banco e a base da documentação (habilidade H1).

Demonstração: dicionário da biblioteca

Tabela ALUNO – Estudantes que podem pegar livros emprestados.

Campo	Descrição	Tipo	Obrig.	Chave	Regra
matricula	Número de matrícula do aluno	inteiro	Sim	PK	Único
nome	Nome completo	texto (120)	Sim		
turma	Turma atual	texto (10)	Não		

Tabela EMPRESTIMO – Cada retirada de um livro por um aluno.

Campo	Descrição	Tipo	Obrig.	Chave	Regra
id_emprestimo	Identificador do empréstimo	inteiro	Sim	PK	Gerado pelo banco
matricula	Aluno que retirou	inteiro	Sim	FK → aluno	Deve existir em aluno
id_livro	Livro retirado	inteiro	Sim	FK → livro	Deve existir em livro
data_emprestimo	Data da retirada	data	Sim		Não pode ser futura

Campo	Descrição	Tipo	Obrig.	Chave	Regra
data_devolucao	Data da devolução	data	Não		Não pode ser anterior à retirada

Ação

1. **Defina as chaves** de todas as entidades do seu DER v2: PK de cada uma (e justifique se é natural ou substituta), chaves alternativas e chaves estrangeiras.
2. Para cada FK, decida o que deve acontecer ao excluir a linha referenciada: **bloquear**, **cascata** ou **anular**. Anote o motivo.
3. Monte o **dicionário de dados** no modelo da demonstração, para todas as tabelas, em `02_modelagem/dicionario_de_dados.md` (ou planilha).
4. Anote as regras de domínio e de negócio de cada campo (por exemplo: preço maior que zero; estoque não negativo; status do pedido limitado a alguns valores).

Reflexão

Analise, em grupo, estes casos. Para cada um, diga **qual integridade** foi violada e **como o seu modelo impede** o problema:

1. Excluir um cliente que já tem pedidos.
2. Registrar um pedido para um cliente que não existe.
3. Cadastrar dois clientes com o mesmo CPF.
4. Cadastrar um produto com preço negativo.
5. Registrar o mesmo produto duas vezes no mesmo pedido.
6. Excluir um pedido: o que deve acontecer com seus itens?

Nova ação

Complete o **DER com as chaves** (PK, FK e chaves alternativas) e atualize o **dicionário de dados** com as regras de integridade e as ações de exclusão. Se descobriu na reflexão que algo estava faltando, registre o ajuste.

Desafio

O dono pergunta: "posso usar o CPF como chave primária do cliente?" Escreva uma resposta curta, com **dois argumentos a favor e dois contra**, e a sua recomendação.

Síntese e entrega

- **PK** identifica; **FK** liga; **chave alternativa** continua única.
- A integridade tem quatro formas: de entidade, referencial, de domínio e de regras de negócio.
- O **dicionário de dados** documenta cada campo e é a base do modelo físico.

Evidência desta aula

DER com chaves e dicionário de dados em [02_modelagem/](#). Avaliação por checklist do indicador I1, com feedback.

Aula 6 – Normalização

Nesta aula

- **Etapas:** 1 – Compreender, preparar o ambiente e modelar
- **Indicador:** I1 (principal)
- **Conhecimentos:** C1 (normalização; teoria dos conjuntos; integridade)
- **Habilidades:** H2, H4 · **Atitudes/valores:** V3
- **Você vai entregar:** o modelo normalizado, com justificativa

No caso do Mercadinho

A funcionária do caixa consolidou os pedidos de agosto em uma única planilha para "facilitar". Ficou fácil de ler, mas difícil de manter: o telefone da Ana aparece em cada linha, o nome do fornecedor está escrito de três jeitos e, quando um fornecedor muda de telefone, é preciso corrigir dezenas de células. A **normalização** é o método para desfazer esse tipo de bagunça de forma organizada.

Conceito: por que normalizar

Normalizar é organizar os dados em tabelas de modo que cada fato seja guardado **uma única vez**. O objetivo é evitar as **anomalias**:

Anomalia	O que acontece
De inserção	Não é possível cadastrar um dado sem inventar outro (cadastrar um produto novo exige ter um pedido).
De atualização	Mudar um dado exige alterar muitas linhas; se esquecer uma, o banco fica inconsistente .
De exclusão	Apagar uma linha faz desaparecer, junto, outro dado que deveria ser mantido (apagar o único pedido de um cliente apaga o cadastro dele).

Relação como conjunto

Uma tabela, no modelo relacional, é uma **relação**: um **conjunto de linhas** (tuplas). Como todo conjunto, ela **não tem linhas repetidas** e a ordem das linhas não importa. É por isso que toda tabela precisa de uma chave que distinga uma linha da outra.

Dependência funcional

Um atributo B **depende funcionalmente** de A quando, conhecendo A, só existe um valor possível para B. Exemplo: conhecendo o `id_cliente`, só existe um nome. Escreve-se `id_cliente → nome`. As formas normais tratam justamente de dependências.

As três primeiras formas normais

Forma normal	Regra	Problema que resolve
1FN	Cada campo guarda um só valor (atômico), sem listas e sem grupos repetidos. A tabela tem uma chave.	Células como "Arroz; Feijão".
2FN	Está na 1FN e nenhum atributo depende de só uma parte da chave composta.	Dados do pedido repetidos para cada produto.

Forma normal	Regra	Problema que resolve
3FN	Está na 2FN e nenhum atributo não-chave depende de outro atributo não-chave (dependência transitiva).	Dados do cliente guardados dentro do pedido.

Demonstração: ordem de serviço de uma oficina

Uma oficina mecânica guarda cada ordem de serviço (OS) assim:

OS	Data	Placa	Modelo	Proprietário	Telefone	Peças usadas	Valor
1	02/08	ABC1D23	Hatch 1.0	Rui Dias	27 98888-0001	Filtro de óleo; Pastilha de freio	35,00; 120,00
2	04/08	ABC1D23	Hatch 1.0	Rui Dias	27 98888-0001	Filtro de óleo	35,00
3	04/08	XYZ9K88	Sedan 1.6	Léa Prado	27 98888-0002	Correia dentada	210,00

Passo 1 – 1FN: separar as listas. Cada linha passa a ter uma só peça, e a chave é **(OS, Peça)**.

OS	Data	Placa	Modelo	Proprietário	Telefone	Peça	Valor
1	02/08	ABC1D23	Hatch 1.0	Rui Dias	27 98888-0001	Filtro de óleo	35,00
1	02/08	ABC1D23	Hatch 1.0	Rui Dias	27 98888-0001	Pastilha de freio	120,00
2	04/08	ABC1D23	Hatch 1.0	Rui Dias	27 98888-0001	Filtro de óleo	35,00
3	04/08	XYZ9K88	Sedan 1.6	Léa Prado	27 98888-0002	Correia dentada	210,00

Passo 2 – 2FN: a chave é composta **(OS, Peça)**. Data, Placa, Modelo, Proprietário e Telefone dependem **só da OS**; Valor depende **só da peça**. Separe:

- ORDEM_SERVICO (**OS**, Data, Placa, Modelo, Proprietário, Telefone)
- PECA (**Peça**, Valor)
- ITEM_OS (**OS**, **Peça**)

Passo 3 – 3FN: em ORDEM_SERVICO, Modelo, Proprietário e Telefone dependem da **Placa**, e não da OS (dependência transitiva: OS → Placa → Modelo). O Telefone, por sua vez, depende do Proprietário. Separe:

- ORDEM_SERVICO (**OS**, Data, *Placa*)
- VEICULO (**Placa**, Modelo, *ID_Proprietário*)
- PROPRIETARIO (**ID_Proprietário**, Nome, Telefone)
- PECA (**ID_Peça**, Nome, Valor)
- ITEM_OS (**OS**, **ID_Peça**)

(Em **negrito**, as chaves primárias; em *italico*, as chaves estrangeiras.)

Veja que **o telefone do Rui agora existe em um só lugar**. Corrigi-lo exige alterar uma linha. Outra observação: o valor da peça pode mudar com o tempo. Para preservar o histórico, é comum guardar também o **valor cobrado** em ITEM_OS, no momento do serviço.

A planilha de pedidos do Mercadinho

Planilha consolidada de pedidos (agosto de 2026)

Pe di do	Data	Cliente	Telefo ne	Produtos	Categoria	Fornecedor	Q t d	Preço
1	03/08 /2026	Ana Beatriz Souza	27 99900 -0001	Arroz tipo 1 - 5 kg; Feijão carioca - 1 kg	Mercearia; Mercearia	Distribuidora Grão Dourado Ltda; Distribuidora Grão Dourado Ltda	1; 3	27,90; 8,49
2	05/08 /2026	Bruno Carvalh o	27 99900 -0002	Refrigerante cola - 2 L; Leite integral - 1 L	Bebidas; Frios e Laticínios	Bebidas Litoral S.A.; Laticínios Serra Azul Ltda	6; 1 2	9,50; 5,49
3	07/08 /2026	Ana Beatriz Souza	27 99900 -0001	Sabonete - 85 g	Higiene e Limpeza	Limpeza Brilho Comércio Ltda	6	2,49
4	10/08 /2026	Carla Mende s	27 99900 -0003	Tomate - kg; Banana prata - kg; Pão francês - kg	Hortifruti; Hortifruti; Padaria	Hortifruti Vale Verde ME; Hortifruti Vale Verde ME; Panificadora Pão & Cia Ltda	2; 1; 1	8,99; 6,99; 16,90

Ação

Normalize a planilha do Mercadinho **da 1FN à 3FN**, escrevendo as tabelas resultantes em cada passo (use uma planilha ou editor de texto em [02_modelagem/normalizacao.md](#)).

1. **1FN**: separe as listas, deixando um produto por linha. Defina a chave da tabela resultante.
2. **Dependências**: escreva as dependências funcionais que existem (por exemplo, **Produto** → **Categoria**, **Produto** → **Fornecedor**, **Cliente** → **Telefone**, **Pedido** → **Data**).
3. **2FN**: separe o que depende só de parte da chave.
4. **3FN**: separe as dependências transitivas.
5. Para cada tabela final, indique PK e FK.

Dica

Pergunte, para cada coluna: "esse dado depende de **quê**?" Se depende só do pedido, do produto, do cliente ou do fornecedor, ele provavelmente pertence a uma tabela daquele assunto.

Reflexão

1. Compare a sua solução com a de outra dupla. Elas chegaram às mesmas tabelas? Se houver diferenças, quais são justificáveis?
2. Escolha uma anomalia de **inserção**, uma de **atualização** e uma de **exclusão** que existiam na planilha original e mostre como o modelo normalizado as elimina.
3. O preço do produto aparece na planilha. Se ele mudar amanhã, o que acontece com os pedidos antigos? Que decisão de modelagem preserva o histórico?
4. Em que situação você aceitaria **não normalizar** completamente? Que risco assumiria?

Nova ação

Aplique a normalização ao **DER da sua dupla**: compare as tabelas normalizadas com o DER v2 e ajuste o que for necessário. Registre cada alteração e a **justificativa** (qual forma normal ou anomalia motivou). Salve como `der_v3` em `02_modelagem/`.

Desafio

Um colega propõe guardar na tabela de pedidos uma coluna `total_do_pedido`, com a soma dos itens. Isso fere a normalização? Escreva dois argumentos a favor e dois contra e diga como o total poderia ser obtido **sem** guardá-lo (você vai fazer isso com SQL nas aulas 13 e 14).

Síntese e entrega

- Normalizar é guardar **cada fato uma só vez** para evitar anomalias.
- **1FN**: valores únicos por campo. **2FN**: sem dependência parcial da chave. **3FN**: sem dependência transitiva.
- Toda tabela é um **conjunto de linhas**: precisa de uma chave.

Evidência desta aula

Modelo normalizado e justificado (`normalizacao.md` e `der_v3`). Observação e feedback do professor, com checklist do indicador I1.

Aula 7 – Do modelo lógico ao modelo físico

Nesta aula

- **Etapas:** 1 – Compreender, preparar o ambiente e modelar
- **Indicador:** I1 (principal)
- **Conhecimentos:** C1 (tipos de dados); C2 (modelagem física, ferramentas de modelagem e manutenção, integridade referencial)
- **Habilidades:** H1, H2, H3 · **Atitudes/valores:** V2
- **Você vai entregar:** o modelo físico versionado

No caso do Mercadinho

O desenho está bom, mas o computador não entende desenhos: ele precisa de **tabelas com colunas de tipos definidos**. Chegou a hora de traduzir o modelo normalizado para o **PostgreSQL**, decidindo, por exemplo, como guardar o preço de um produto e o CPF de um cliente.

Conceito: do lógico ao físico

O **modelo lógico** organiza os dados em tabelas, chaves e relações, independentemente do SGBD. O **modelo físico** adiciona o que depende do SGBD escolhido: tipos de dados, tamanhos, restrições, nomes definitivos e índices. As regras de tradução são:

No modelo lógico	No modelo físico (PostgreSQL)
Entidade	Tabela
Atributo	Coluna, com tipo e tamanho
Identificador	Chave primária (PK)
Relacionamento 1:N	Chave estrangeira (FK) na tabela do lado N
Entidade associativa (N:N)	Tabela com chave composta e duas FKs
Atributo opcional / obrigatório	Coluna que aceita ou não valor vazio (NULL / NOT NULL)
Regra de negócio	Restrição (CHECK , UNIQUE)

Principais tipos de dados do PostgreSQL

Tipo	Uso	Exemplo
smallint, integer, bigint	Números inteiros de tamanhos crescentes	quantidade, identificadores
numeric(p, s)	Número decimal exato , com p dígitos no total e s após a vírgula	preço: numeric(10,2)
real, double precision	Número decimal aproximado	medidas científicas (não use para dinheiro)
char(n)	Texto de tamanho fixo	CPF (11), CNPJ (14)
varchar(n)	Texto de até n caracteres	nome, e-mail
text	Texto de tamanho livre	observações
boolean	Verdadeiro ou falso	produto ativo
date	Data	data do pedido

Tipo	Uso	Exemplo
timestamp, timestampz	Data e hora (sem / com fuso horário)	momento de uma operação

Chave gerada pelo banco

Para as chaves substitutas, o PostgreSQL oferece colunas de **identidade**: `integer GENERATED BY DEFAULT AS IDENTITY`. O banco numera as linhas automaticamente (1, 2, 3...). Você verá isso na prática na aula 9.

Nomes: uma convenção para toda a equipe

Padronize para facilitar a leitura e evitar erros: tudo em minúsculas, `snake_case` (palavras separadas por `_`), **sem acentos nem espaços**, tabelas no singular (`cliente`), PK como `id_<tabela>`, FK com o mesmo nome da PK referenciada, e restrições com prefixo (`pk_`, `fk_`, `uq_`, `ck_`).

Demonstração: escolhas de tipo na prática

Execute no SQL Editor do Supabase (ou no `psql`) estes experimentos. Eles mostram **por que** certas decisões de tipo são importantes.

1. Dinheiro: `numeric` ou decimal aproximado?

```
SELECT 0.1::double precision + 0.2::double precision = 0.3::double precision AS com_double,
       0.1::numeric + 0.2::numeric = 0.3::numeric AS com_numeric;
```

O resultado `false` para o tipo aproximado mostra que ele não é adequado para valores monetários.

2. CPF: número ou texto?

```
SELECT 00123456789::bigint AS cpf_como_numero,
       '00123456789'::char(11) AS cpf_como_texto;
```

Como número, o CPF **perde o zero à esquerda**. Ele é um identificador, não uma quantidade: não se soma nem se faz média de CPFs.

3. Que tipo o PostgreSQL atribui a cada valor?

```
SELECT pg_typeof(CURRENT_DATE) AS tipo_data,
       pg_typeof(now()) AS tipo_data_hora,
       pg_typeof(true) AS tipo_logico,
       pg_typeof(12.5) AS tipo_decimal;
```

4. O banco valida a data?

Este comando deve gerar erro (é o esperado)

```
SELECT DATE '2026-02-30';
```

O PostgreSQL recusa a data inexistente. É a integridade de **domínio** agindo.

Ação

Converta o seu **modelo normalizado (der_v3)** em **modelo físico** para o PostgreSQL, usando a ferramenta de modelagem indicada pelo professor.

1. Para cada tabela, defina **colunas, tipos e tamanhos** (use os experimentos acima para decidir).
2. Defina **NOT NULL** para os campos obrigatórios do dicionário de dados.
3. Marque **PK, FK e chaves únicas** e defina as ações de exclusão decididas na aula 5.

4. Anote as regras de negócio que viram **CHECK** (por exemplo, preço maior que zero, estoque não negativo, status limitado a valores permitidos).
5. Aplique a convenção de nomes e atualize o **dicionário de dados** com os tipos finais.
6. Exporte o diagrama físico como imagem e salve o arquivo editável em **02_modelagem/**.

Reflexão

Em dupla, justifique cada escolha em um quadro como este:

Campo	Tipo escolhido	Por quê?	Alternativa descartada
preco_venda	numeric(10,2)	Valores exatos, 2 casas decimais	double precision
cpf			
data_pedido			
ativo			
id_cliente			

Discuta também: o que acontece com a integridade se uma coluna que deveria ser obrigatória aceitar **NULL**? Que restrição você acrescentaria para impedir um estoque negativo?

Nova ação

Ajuste e versione o modelo físico: corrija o que a discussão apontou e salve como **modelo_fisico_v1.0**. No arquivo **02_modelagem/historico_versoes.md**, registre a versão, a data, quem alterou e o que mudou. A partir de agora, **toda mudança no modelo gera uma nova versão** (v1.1, v1.2...).

Desafio

Pesquise na documentação do PostgreSQL a diferença entre **char(n)**, **varchar(n)** e **text**. Em que situação **char(n)** é uma boa escolha? Existe ganho de desempenho em escolher um em vez do outro?

Síntese e entrega

- O modelo físico traduz o lógico para o SGBD: **tipos, tamanhos, restrições e nomes**.
- Use **numeric** para dinheiro e **texto** para identificadores como CPF.
- Padronizar nomes e **versionar** o modelo é parte da documentação do projeto.

Evidência desta aula

Modelo físico versionado (**modelo_fisico_v1.0**), dicionário atualizado e **historico_versoes.md**. Checklist do indicador I1.

Aula 8 – Checkpoint 1: apresentação e arguição do modelo

Nesta aula

- **Etapas:** 1 – Compreender, preparar o ambiente e modelar
- **Indicador:** I1 (principal) – **menção parcial**
- **Conhecimentos:** C1 e C2 integrados
- **Habilidades:** H1, H2, H4 · **Atitudes/valores:** V1
- **Você vai entregar:** a versão base do modelo de dados

No caso do Mercadinho

Antes de construir o banco, a equipe apresenta o modelo ao "cliente" (neste caso, outra dupla e o professor) e responde perguntas. É melhor descobrir um erro agora, no papel, do que depois, com dados já carregados. Este é o **primeiro checkpoint**: um momento para saber como você está e corrigir o rumo.

Como funciona o checkpoint

Cada dupla apresenta o seu modelo em cerca de **5 minutos** e responde a perguntas técnicas de outra dupla e do professor. Prepare:

- o **DER** com chaves e cardinalidades;
- o **modelo físico** (tipos e restrições);
- o **dicionário de dados**;
- o **registro de alterações**, mostrando como o modelo evoluiu.

Roteiro da apresentação

1. **Contexto:** o problema do Mercadinho em duas frases.
2. **Modelo:** as entidades e como se relacionam (cardinalidades).
3. **Decisões:** três escolhas importantes e o motivo (chaves, normalização, tipos).
4. **Integridade:** como o modelo impede dados errados.
5. **Evolução:** o que mudou da v1 para a versão atual e por quê.

Perguntas que podem surgir

- Por que essa relação é N:N e como foi resolvida?
- O que impede um pedido de existir sem cliente?
- Por que guardar o preço no item do pedido, e não só no produto?
- Que anomalia a sua normalização evita? Mostre em um exemplo.
- Por que esse campo é **numeric** / **char** / **varchar**?

CrITÉRIOS observÁveis (indicador I1)

O professor e a dupla avaliadora observam os critérios abaixo. O professor combina com a turma, antes da apresentação, como eles se traduzem em **A**, **PA** ou **NA**.

#	O que será observado	Evidência
1	Todas as entidades dos requisitos estão representadas.	DER
2	Cada tabela tem chave primária e as chaves estrangeiras são coerentes.	DER e modelo físico
3	As cardinalidades estão corretas e justificadas.	DER e fala da dupla
4	Relacionamentos N:N foram resolvidos com tabela associativa.	Modelo físico
5	O modelo está na 3FN, ou a exceção está justificada.	normalizacao.md
6	Os tipos de dados são adequados ao conteúdo (preço, CPF, datas).	Modelo físico
7	O dicionário de dados está completo e coerente com o diagrama.	Dicionário
8	O diagrama é legível e usa nomes padronizados.	Diagrama
9	A dupla explica as decisões com termos técnicos corretos.	Apresentação

Ação

1. **Apresente** o modelo de dados e o dicionário para a outra dupla, respondendo às perguntas técnicas.
2. **Avalie** a dupla que se apresentou, preenchendo a ficha de feedback abaixo.

Critério	Observação (o que está bom / o que melhorar)
Entidades, chaves e cardinalidades	
Normalização	
Tipos e restrições	
Dicionário de dados	
Clareza da apresentação	

Reflexão

Registre por escrito a sua **autoavaliação** e o feedback recebido:

1. Em quais critérios do quadro você acredita ter alcançado o esperado? Que evidência mostra isso?
2. Que pergunta você não soube responder bem? O que precisa estudar?
3. Que sugestão da outra dupla você vai adotar? Alguma você discorda? Por quê (com argumento técnico)?
4. De que forma a sua comunicação (clareza, uso de termos técnicos, cordialidade) ajudou ou atrapalhou?

Nova ação

Faça os **ajustes finais** apontados e **fixe a versão base do modelo**: salve o modelo físico final como [modelo_fisico_v1.1](#) (ou a versão correspondente), com dicionário e histórico de versões atualizados. A partir da aula 9 você vai construir o banco a partir desta versão.

Modelo de referência

Depois de concluir o seu modelo, consulte o **Apêndice B – Modelo de referência** e compare. Nas aulas 9 a 24, os exemplos de código usam o modelo de referência. Se o seu modelo for diferente, adapte nomes e colunas aos seus.

Desafio

Peça a um colega de outra dupla que escreva, **sem olhar o seu DER**, três frases sobre o Mercadinho usando só o seu dicionário de dados ("um pedido pertence a um ...", "um produto tem ..."). Se ele conseguir, a documentação está clara. Se não conseguir, descubra o que faltou.

Síntese e entrega

- Um modelo é **validado** quando pessoas diferentes o entendem e conseguem questioná-lo.
- Corrigir no desenho é muito mais barato do que corrigir no banco já populado.
- Documentação clara é parte do trabalho do técnico.

Evidência desta aula

Modelo de dados e dicionário finais, ficha de feedback e autoavaliação. **Menção parcial do indicador I1 (A/PA/NA)**, com feedback individual do professor.

Parte II – Construir e manipular

Aulas 9 a 18

Aula 9 – DDL I: criação das estruturas

Nesta aula

- **Etapa:** 2 – Construir e manipular
- **Indicadores:** I3 (principal) · I1 (aferição do banco em relação ao diagrama)
- **Conhecimentos:** C2 (criação, entidade, atributos, chaves, integridade referencial)
- **Habilidades:** H2, H3 · **Atitudes/valores:** V3
- **Você vai entregar:** os scripts de criação das tabelas (DDL), executados no Supabase

No caso do Mercadinho

O modelo foi aprovado no checkpoint. Agora a equipe vai transformar o desenho em **banco de dados de verdade**, escrevendo scripts SQL que criam cada tabela. Scripts são importantes porque podem ser **guardados, revisados e executados de novo** em qualquer ambiente.

Conceito: a linguagem SQL e suas partes

A SQL é dividida em grupos de comandos, de acordo com o que fazem:

Grupo	Nome	Para que serve	Comandos principais
DDL	Linguagem de Definição de Dados	Criar e alterar a estrutura	CREATE, ALTER, DROP
DML	Linguagem de Manipulação de Dados	Inserir, alterar e apagar dados	INSERT, UPDATE, DELETE
DQL	Linguagem de Consulta	Consultar dados	SELECT
DCL	Linguagem de Controle de Dados	Controlar acessos	GRANT, REVOKE
TCL	Linguagem de Controle de Transações	Confirmar ou desfazer operações	BEGIN, COMMIT, ROLLBACK

Nesta aula e na próxima você usa a **DDL**.

O comando CREATE TABLE

```
CREATE TABLE nome_da_tabela (
    coluna tipo [restrições],
    ...
    [restrições da tabela]
);
```

As principais restrições:

Restrição	Efeito
NOT NULL	A coluna não aceita valor vazio.
DEFAULT valor	Valor usado quando nada é informado.
PRIMARY KEY	Identifica a linha: único e não vazio.
UNIQUE	Não permite repetição.
CHECK (condição)	A condição precisa ser verdadeira.
FOREIGN KEY ... REFERENCES	A coluna deve existir na tabela referenciada.
ON DELETE ...	Define o que ocorre ao apagar a linha referenciada (RESTRICT, CASCADE, SET NULL).

A ordem de criação importa

Uma tabela só pode referenciar outra que **já existe**. Por isso criamos primeiro as tabelas "pai" (sem FK) e depois as que dependem delas: **categoria** e **fornecedor** → **produto** → ... Se a ordem estiver errada, o banco avisa:

Este comando deve gerar erro (é o esperado)

```
CREATE TABLE produto_teste (
    id_produto integer PRIMARY KEY,
    id_categoria integer REFERENCES categoria_inexistente (id_categoria)
);
```

Dica

Dê **nome** às restrições (**CONSTRAINT pk_cliente PRIMARY KEY ...**). Quando algo falhar, a mensagem de erro dirá exatamente qual regra foi violada.

Demonstração

Crie as duas primeiras tabelas do banco do Mercadinho no SQL Editor do Supabase. Observe como cada decisão do modelo físico (aula 7) virou uma linha de código.

```
CREATE TABLE categoria (
    id_categoria integer GENERATED BY DEFAULT AS IDENTITY,
    nome varchar(60) NOT NULL,
    CONSTRAINT pk_categoria PRIMARY KEY (id_categoria),
    CONSTRAINT uq_categoria_nome UNIQUE (nome)
);

CREATE TABLE fornecedor (
    id_fornecedor integer GENERATED BY DEFAULT AS IDENTITY,
    razao_social varchar(120) NOT NULL,
    cnpj char(14) NOT NULL,
    telefone varchar(20),
    email varchar(120),
    cidade varchar(60),
    CONSTRAINT pk_fornecedor PRIMARY KEY (id_fornecedor),
    CONSTRAINT uq_fornecedor_cnpj UNIQUE (cnpj)
);
```

Agora a tabela que **depende** das duas, **produto**:

```
CREATE TABLE produto (
    id_produto integer GENERATED BY DEFAULT AS IDENTITY,
    nome varchar(120) NOT NULL,
    id_categoria integer NOT NULL,
    id_fornecedor integer NOT NULL,
    preco_venda numeric(10,2) NOT NULL,
    estoque_atual integer NOT NULL DEFAULT 0,
    estoque_minimo integer NOT NULL DEFAULT 5,
    ativo boolean NOT NULL DEFAULT true,
    CONSTRAINT pk_produto PRIMARY KEY (id_produto),
    CONSTRAINT fk_produto_categoria FOREIGN KEY (id_categoria)
        REFERENCES categoria (id_categoria),
    CONSTRAINT fk_produto_fornecedor FOREIGN KEY (id_fornecedor)
        REFERENCES fornecedor (id_fornecedor),
    CONSTRAINT ck_produto_preco CHECK (preco_venda > 0),
    CONSTRAINT ck_produto_estoque CHECK (estoque_atual >= 0 AND estoque_minimo >= 0)
);
```

Leia o código lendo o modelo: o **NOT NULL** vem do dicionário de dados (campo obrigatório), as duas FKs vêm das relações 1:N, os **CHECK** vêm das regras de negócio ("preço maior que zero"; "estoque não negativo").

Ação

Salve os comandos em `03_scripts/01_criar_tabelas.sql` e execute-os no **Supabase** na ordem correta. Depois:

1. **Crie as tabelas restantes do seu modelo** (`cliente`, `pedido`, `item_pedido`), seguindo o seu modelo físico.
2. Execute o script, corrija os erros que aparecerem e **leia as mensagens**: elas dizem a linha e o motivo.
3. **Aferição**: confira se o banco criado corresponde ao DER usando estas consultas, que descrevem o que existe no banco.

```
SELECT table_name, column_name, data_type, is_nullable
FROM information_schema.columns
WHERE table_schema = 'public'
ORDER BY table_name, ordinal_position;
```

```
SELECT conrelid::regclass AS tabela, conname AS restricao, contype AS tipo
FROM pg_constraint
WHERE connamespace = 'public'::regnamespace
ORDER BY conrelid::regclass::text, contype, conname;
```

No resultado de restrições, o tipo **p** é chave primária, **f** chave estrangeira, **u** única e **c** **CHECK**. Compare, tabela por tabela, com o diagrama físico.

Script de referência (consulte depois de tentar)

Se precisar comparar com o modelo de referência, estas são as três tabelas restantes. **Tente escrever as suas antes de olhar.**

```
CREATE TABLE cliente (
  id_cliente integer GENERATED BY DEFAULT AS IDENTITY,
  nome varchar(120) NOT NULL,
  cpf char(11) NOT NULL,
  email varchar(120),
  telefone varchar(20),
  cidade varchar(60),
  data_cadastro date NOT NULL DEFAULT CURRENT_DATE,
  CONSTRAINT pk_cliente PRIMARY KEY (id_cliente),
  CONSTRAINT uq_cliente_cpf UNIQUE (cpf)
);

CREATE TABLE pedido (
  id_pedido integer GENERATED BY DEFAULT AS IDENTITY,
  id_cliente integer NOT NULL,
  data_pedido date NOT NULL DEFAULT CURRENT_DATE,
  status varchar(10) NOT NULL DEFAULT 'aberto',
  CONSTRAINT pk_pedido PRIMARY KEY (id_pedido),
  CONSTRAINT fk_pedido_cliente FOREIGN KEY (id_cliente)
    REFERENCES cliente (id_cliente),
  CONSTRAINT ck_pedido_status CHECK (status IN ('aberto', 'pago', 'cancelado'))
);

CREATE TABLE item_pedido (
  id_pedido integer NOT NULL,
  id_produto integer NOT NULL,
  quantidade integer NOT NULL,
  preco_unitario numeric(10,2) NOT NULL,
  CONSTRAINT pk_item_pedido PRIMARY KEY (id_pedido, id_produto),
  CONSTRAINT fk_item_pedido FOREIGN KEY (id_pedido)
    REFERENCES pedido (id_pedido) ON DELETE CASCADE,
  CONSTRAINT fk_item_produto FOREIGN KEY (id_produto)
    REFERENCES produto (id_produto),
  CONSTRAINT ck_item_quantidade CHECK (quantidade > 0),
  CONSTRAINT ck_item_preco CHECK (preco_unitario > 0)
);
```

Reflexão

1. Quais erros apareceram ao executar o script? Em que **ordem** corrigi-los? Que mensagem de erro foi mais útil?
2. A aferição encontrou **diferenças entre o banco e o DER**? Anote cada uma e diga quem está certo: o diagrama ou o script?
3. Por que **item_pedido** tem **chave primária composta (id_pedido, id_produto)**? O que isso impede?
4. Por que o **ON DELETE CASCADE** foi usado só em **item_pedido** e não nas demais chaves estrangeiras?

Nova ação

Corrija os scripts e/ou o DER para que fiquem coerentes entre si e **versione** as alterações (um novo arquivo de modelo físico ou uma nota no **historico_versoes.md**). Deixe o script **01_criar_tabelas.sql** com **comentários** no início:

```
-- Projeto: Banco de dados do Mercadinho Bom Preço  
-- Script: 01_criar_tabelas.sql   Versão: 1.0   Autores: ...  
-- Descrição: cria as tabelas do modelo físico v1.1
```

Desafio

Acrescente à tabela **produto** uma restrição que impeça que o **estoque mínimo seja maior que 1000** (regra inventada pelo dono para evitar erros de digitação). Escreva o comando como parte do **CREATE TABLE** e depois como um **ALTER TABLE** (você estuda **ALTER** na próxima aula).

Síntese e entrega

- A **DDL** (**CREATE**, **ALTER**, **DROP**) define a estrutura do banco.
- Tabelas são criadas **da mais independente para a mais dependente**.
- Dar nomes às restrições torna as mensagens de erro claras.
- **Aferir** é comparar o que foi construído com o que foi projetado.

Evidência desta aula

01_criar_tabelas.sql executado sem erros no Supabase, resultado das consultas de aferição e lista de diferenças encontradas entre o diagrama e o banco (indicadores I3 e I1).

Aula 10 – DDL II: alteração de estruturas e reprodução no ambiente local

Nesta aula

- **Etapa:** 2 – Construir e manipular
- **Indicadores:** I3 (principal) · I2 (apoio)
- **Conhecimentos:** C2 (criação de objetos, integridade referencial, ferramentas de manutenção)
- **Habilidades:** H1, H3 · **Atitudes/valores:** V3
- **Você vai entregar:** um script reproduzível, executado nos dois ambientes

No caso do Mercadinho

Na semana passada o dono descobriu que dois clientes foram cadastrados com o CPF "123", e que um produto ficou sem código de barras. O banco precisa de **regras mais rígidas**, mas ele já está criado. Em vez de apagar tudo, a equipe vai **alterar** a estrutura existente. E, como qualquer mudança deve ser ensaiada antes, vai também **reproduzir o banco no PostgreSQL local**.

Conceito: ALTER TABLE

O comando **ALTER TABLE** modifica uma tabela que já existe, sem perder os dados:

Operação	Comando
Adicionar coluna	<code>ALTER TABLE t ADD COLUMN c tipo;</code>
Remover coluna	<code>ALTER TABLE t DROP COLUMN c;</code>
Alterar o tipo	<code>ALTER TABLE t ALTER COLUMN c TYPE novo_tipo;</code>
Definir valor padrão	<code>ALTER TABLE t ALTER COLUMN c SET DEFAULT valor;</code>
Tornar obrigatória	<code>ALTER TABLE t ALTER COLUMN c SET NOT NULL;</code>
Adicionar restrição	<code>ALTER TABLE t ADD CONSTRAINT nome ...;</code>
Remover restrição	<code>ALTER TABLE t DROP CONSTRAINT nome;</code>
Renomear	<code>ALTER TABLE t RENAME COLUMN c TO novo;</code>

Atenção

DROP COLUMN e **DROP TABLE** **apagam dados**. Adicionar uma restrição a uma tabela que já tem dados só funciona se **todos os dados existentes já a respeitam**.

Scripts organizados e reproduzíveis

Um script é **reproduzível** quando, executado em outro ambiente, produz o **mesmo resultado**. Para isso:

- numere os arquivos pela ordem de execução (`01_criar_tabelas.sql`, `02_ajustes_estrutura.sql`, ...);
- comente cada script (objetivo, versão, autores);
- guarde tudo na pasta `03_scripts/`;
- evite alterações "feitas na mão" que não foram para um script.

Há dois estilos de manter a estrutura: **acumular** scripts de alteração (02_, 03_...), que registram a história; ou **consolidar** tudo em um só script de criação completo. Um script consolidado costuma começar limpando o que existe:

```
-- CUIDADO: apaga as tabelas e todos os dados. Use só em ambiente de teste.
DROP TABLE IF EXISTS item_pedido, pedido, cliente, produto, fornecedor, categoria CASCADE;
```

Demonstração

Veja **ALTER TABLE** em ação em uma tabela de ensaio, que será apagada ao final:

```
CREATE TABLE exemplo_alter (
    id integer PRIMARY KEY,
    nome varchar(30)
);

ALTER TABLE exemplo_alter ADD COLUMN observacao text;
ALTER TABLE exemplo_alter ADD COLUMN quantidade integer DEFAULT 0;
ALTER TABLE exemplo_alter ALTER COLUMN nome SET NOT NULL;
ALTER TABLE exemplo_alter ALTER COLUMN nome TYPE varchar(60);
ALTER TABLE exemplo_alter ADD CONSTRAINT ck_exemplo_qtd CHECK (quantidade >= 0);
ALTER TABLE exemplo_alter DROP COLUMN observacao;

DROP TABLE exemplo_alter;
```

Observe que o **CHECK** foi adicionado depois e valerá para os dados novos (e para os antigos, que foram verificados na hora).

Ação

Parte 1 – Aplicar os ajustes no Supabase. Salve em **03_scripts/02_ajustes_estrutura.sql** e execute:

```
-- Código de barras do produto (opcional, mas único quando informado)
ALTER TABLE produto ADD COLUMN codigo_barras varchar(14);
ALTER TABLE produto ADD CONSTRAINT uq_produto_codigo_barras UNIQUE (codigo_barras);

-- Regras de domínio: CPF e CNPJ só com 11 e 14 dígitos
ALTER TABLE cliente ADD CONSTRAINT ck_cliente_cpf CHECK (cpf ~ '^[0-9]{11}$');
ALTER TABLE fornecedor ADD CONSTRAINT ck_fornecedor_cnpj CHECK (cnpj ~ '^[0-9]{14}$');

-- E-mail, quando informado, precisa ter @
ALTER TABLE cliente ADD CONSTRAINT ck_cliente_email CHECK (email IS NULL OR email LIKE '%@%');
```

O operador **~** testa se o texto casa com uma **expressão regular**: **^[0-9]{11}\$** significa "exatamente 11 dígitos, do início ao fim".

Parte 2 – Reproduzir no PostgreSQL local. Em um terminal, na pasta **03_scripts/**, execute os dois scripts no banco **comercio** que você criou na aula 3:

```
No terminal
psql -h localhost -U comercio_app -d comercio -f 01_criar_tabelas.sql
psql -h localhost -U comercio_app -d comercio -f 02_ajustes_estrutura.sql
```

Confirme no **psql** com **\dt** (tabelas) e **\d produto** (estrutura de uma tabela). Aplique também as consultas de aferição da aula 9 no ambiente local.

Reflexão

1. Houve **diferença de comportamento** entre o Supabase e o PostgreSQL local? Liste (versão, mensagens, esquemas, papéis, extensões).
2. Por que é importante que o mesmo script gere o mesmo banco nos dois ambientes?
3. O que aconteceria se a alteração `ck_cliente_cpf` fosse aplicada a uma tabela que já tivesse o CPF "123"? Como você resolveria?
4. Qual a vantagem de **acumular** scripts de alteração? E de manter um script **consolidado**?

Nova ação

Consolide o trabalho em um **script reproduzível**: crie `03_scripts/00_recriar_banco_completo.sql`, que:

1. começa com o comando de limpeza (com o aviso em comentário);
2. cria todas as tabelas **já com os ajustes** (código de barras, `CHECK` de CPF, CNPJ e e-mail);
3. termina com a consulta de aferição de colunas.

Execute-o **nos dois ambientes**, em um banco de **teste** (nunca onde há dados que você precise manter), e confirme que o resultado é o mesmo.

Desafio

Escreva a restrição que garante que o **telefone** do cliente, quando informado, tenha pelo menos 10 caracteres. Teste com um valor curto para provar que ela funciona (volte aqui depois da aula 11, quando você já souber inserir dados).

Síntese e entrega

- `ALTER TABLE` evolui a estrutura **sem recriar** a tabela.
- Restrições novas só são aceitas se os dados existentes as respeitam.
- Scripts **numerados, comentados e reproduzíveis** permitem repetir o banco em qualquer ambiente.

Evidência desta aula

Script reproduzível executado no Supabase e no PostgreSQL local, e anotação das diferenças entre os dois ambientes. Observação e checklist dos indicadores I3 e I2.

Aula 11 – DML: inserção, atualização e exclusão de dados

Nesta aula

- **Etapa:** 2 – Construir e manipular
- **Indicador:** I3 (principal)
- **Conhecimentos:** C2 (manipulação, integridade referencial); transações básicas (complementação didática)
- **Habilidades:** H3, H4 · **Atitudes/valores:** V2, V3
- **Você vai entregar:** o script de carga de dados (DML), seguro e repetível

No caso do Mercadinho

Com as tabelas prontas, falta o que importa: **os dados**. O dono quer ver o banco com produtos, clientes e pedidos de verdade (fictícios, claro). Mas há um cuidado: um comando de alteração errado pode estragar milhares de registros em um segundo. Nesta aula você aprende a **manipular dados com segurança**.

Conceito: INSERT, UPDATE e DELETE

INSERT – adiciona linhas:

```
INSERT INTO tabela (coluna1, coluna2) VALUES (valor1, valor2);  
INSERT INTO tabela (coluna1, coluna2) VALUES (a, b), (c, d), (e, f); -- várias linhas
```

UPDATE – altera linhas **que atendem a uma condição**:

```
UPDATE tabela SET coluna = novo_valor WHERE condição;
```

DELETE – apaga linhas **que atendem a uma condição**:

```
DELETE FROM tabela WHERE condição;
```

Atenção: o WHERE protege você

UPDATE ou **DELETE** **sem WHERE** atingem **todas as linhas da tabela**. É um dos erros mais comuns (e mais caros) da profissão. Antes de rodar, faça um **SELECT** com a mesma condição para ver **quais linhas serão afetadas**.

Outros comandos úteis:

- **RETURNING**: devolve as linhas afetadas por **INSERT**, **UPDATE** ou **DELETE**, para você conferir o resultado.
- **TRUNCATE**: apaga **todas** as linhas de uma ou mais tabelas, rapidamente. Com **RESTART IDENTITY**, reinicia a numeração automática.

Integridade na prática

As restrições que você criou nas aulas 9 e 10 agora serão testadas. Cada tentativa de inserir dados inválidos é **recusada** pelo banco, com uma mensagem que identifica a restrição violada.

Transações (complementação didática)

Uma **transação** agrupa comandos para que tenham efeito **juntos ou nenhum**: **BEGIN** inicia, **COMMIT** confirma e **ROLLBACK** desfaz tudo desde o **BEGIN**. É um recurso excelente para **testar com segurança**: execute os comandos, confira o resultado e desfaça.

Complementação didática

Transações não constam como tópico do Plano de Curso. Elas entram aqui apenas para você **testar alterações sem risco** (indicador I3). No psql local o comportamento é o descrito acima; no SQL Editor do Supabase, confirme na documentação como as transações são tratadas antes de usar.

Demonstração: inserir, atualizar, apagar

```
INSERT INTO categoria (nome) VALUES ('Categoria de teste') RETURNING id_categoria, nome;
```

```
UPDATE categoria SET nome = 'Categoria de teste (editada)'
WHERE nome = 'Categoria de teste'
RETURNING id_categoria, nome;
```

```
DELETE FROM categoria WHERE nome = 'Categoria de teste (editada)' RETURNING id_categoria, nome;
```

Ação

Parte 1 – O banco recusa dados inválidos. Execute cada comando abaixo (um de cada vez) e **leia a mensagem de erro**. Anote qual restrição foi violada.

Primeiro, insira uma categoria e tente inserir **de novo**:

```
INSERT INTO categoria (nome) VALUES ('Categoria de teste');
```

Este comando deve gerar erro (é o esperado)

```
INSERT INTO categoria (nome) VALUES ('Categoria de teste');
```

Uma chave estrangeira que não existe, e uma coluna obrigatória em branco:

Este comando deve gerar erro (é o esperado)

```
INSERT INTO produto (nome, id_categoria, id_fornecedor, preco_venda)
VALUES ('Produto fantasma', 999, 1, 5.00);
```

Este comando deve gerar erro (é o esperado)

```
INSERT INTO produto (nome) VALUES ('Produto sem categoria');
```

Dados fora do domínio: CPF longo demais e CPF que não tem 11 dígitos:

Este comando deve gerar erro (é o esperado)

```
INSERT INTO cliente (nome, cpf) VALUES ('Fulano de Tal', '123456789012');
```

Este comando deve gerar erro (é o esperado)

```
INSERT INTO cliente (nome, cpf) VALUES ('Fulano de Tal', '123');
```

Limpe o dado de teste:

```
DELETE FROM categoria WHERE nome = 'Categoria de teste';
```

Parte 2 – Carga de dados. Salve o script do **Material de apoio** (no fim deste capítulo) como `03_scripts/03_carga_dados.sql` e execute-o. Ele começa com **TRUNCATE ... RESTART IDENTITY**, o

que o torna **repetível**: pode ser executado várias vezes e sempre deixa o banco no mesmo estado inicial. Depois, confira a carga:

```
SELECT 'categoria' AS tabela, count(*) AS linhas FROM categoria
UNION ALL SELECT 'fornecedor', count(*) FROM fornecedor
UNION ALL SELECT 'produto', count(*) FROM produto
UNION ALL SELECT 'cliente', count(*) FROM cliente
UNION ALL SELECT 'pedido', count(*) FROM pedido
UNION ALL SELECT 'item_pedido', count(*) FROM item_pedido;
```

O resultado esperado é: 7 categorias, 7 fornecedores, 25 produtos, 14 clientes, 24 pedidos e 70 itens de pedido.

Parte 3 – Alterações com segurança. Execute cada bloco e observe. Eles usam transação e terminam em **ROLLBACK**, então **não alteram o banco de verdade**.

O erro clássico, o **UPDATE** sem **WHERE**:

```
BEGIN;
UPDATE produto SET preco_venda = 0.01;
SELECT count(*) AS produtos_com_preco_errado FROM produto WHERE preco_venda = 0.01;
ROLLBACK;
SELECT count(*) AS depois_do_rollback FROM produto WHERE preco_venda = 0.01;
```

Uma baixa de estoque correta, com **WHERE** e **RETURNING**:

```
BEGIN;
UPDATE produto SET estoque_atual = estoque_atual - 2
WHERE id_produto = 1
RETURNING id_produto, nome, estoque_atual;
ROLLBACK;
```

Exclusão e integridade referencial. Não se pode apagar um cliente que tem pedidos, mas ao apagar um pedido seus itens saem juntos (**ON DELETE CASCADE**):

Este comando deve gerar erro (é o esperado)
 DELETE FROM cliente WHERE id_cliente = 1;

```
BEGIN;
DELETE FROM pedido WHERE id_pedido = 24;
SELECT count(*) AS itens_do_pedido_24 FROM item_pedido WHERE id_pedido = 24;
ROLLBACK;
```

Reflexão

1. Para cada erro da Parte 1, qual restrição foi violada e **por que é bom** que o banco a imponha, em vez de depender de quem digita?
2. O **UPDATE** sem **WHERE** mudou quantos produtos? O que teria acontecido sem a transação?
3. Por que foi possível apagar o pedido 24 (com seus itens), mas não o cliente 1?
4. Por que o script de carga usa **TRUNCATE ... RESTART IDENTITY** no início? O que aconteceria se executássemos a carga duas vezes **sem** essa linha?
5. Por que, na carga, os pedidos só podem ser inseridos depois dos clientes e os itens depois dos pedidos e produtos?

Nova ação

Torne o script de carga **seguro e repetível**:

1. Coloque **BEGIN**; no início e **COMMIT**; no final do script, para que a carga seja feita **por inteiro ou não seja feita**.
2. Execute o script **duas vezes seguidas** e prove, com a consulta de conferência, que o resultado é o mesmo.
3. Introduza **de propósito** um erro no meio do script (por exemplo, um produto com categoria 99) e verifique o que acontece com o banco. Corrija em seguida.
4. Execute o script também no PostgreSQL local.

Desafio

O dono quer **reajustar em 10%** o preço de todos os produtos **ativos** da categoria Bebidas. Escreva o **UPDATE**, antes de executar escreva o **SELECT** que mostra quais linhas serão afetadas, execute dentro de uma transação, confira o resultado e termine com **ROLLBACK**.

Síntese e entrega

- **INSERT** adiciona, **UPDATE** altera e **DELETE** apaga. **Sem WHERE, atingem todas as linhas**.
- O banco **recusa dados inválidos** graças às restrições; as mensagens dizem qual regra foi violada.
- Transações permitem **testar e desfazer**. Scripts de carga devem ser repetíveis.

Evidência desta aula

03_carga_dados.sql (com transação) executado e a conferência de contagens, mais o registro dos erros da Parte 1 e o que cada um ensinou. Observação com checklist (indicador I3).

Material de apoio: script de carga (03_carga_dados.sql)

Os dados são fictícios. O script limpa as tabelas, reinicia a numeração e insere tudo na ordem correta.

```
TRUNCATE item_pedido, pedido, produto, cliente, fornecedor, categoria RESTART IDENTITY;

INSERT INTO categoria (nome) VALUES
  ('Mercearia'),
  ('Bebidas'),
  ('Higiene e Limpeza'),
  ('Hortifruti'),
  ('Padaria'),
  ('Frios e Laticínios'),
  ('Pet Shop');

INSERT INTO fornecedor (razao_social, cnpj, telefone, email, cidade) VALUES
  ('Distribuidora Grão Dourado Ltda', '10000000000101', '27 3000-0101', 'vendas@graodourado.example', 'Serra'),
  ('Bebidas Litoral S.A.', '10000000000202', '27 3000-0202', 'pedidos@bebidaslitoral.example', 'Vila Velha'),
  ('Limpeza Brilho Comércio Ltda', '10000000000303', '27 3000-0303', 'contato@limpezabrilho.example', 'Cariacica'),
  ('Hortifruti Vale Verde ME', '10000000000404', '27 3000-0404', NULL, 'Vitória'),
  ('Laticínios Serra Azul Ltda', '10000000000505', '27 3000-0505', 'comercial@serraazul.example', 'Serra'),
  ('Panificadora Pão & Cia Ltda', '10000000000606', '27 3000-0606', 'atendimento@paoecia.example', 'Vitória'),
  ('Importadora Alfa Ltda', '10000000000707', '27 3000-0707', 'alfa@importadoraalfa.example', 'Guarapari');
```

```

INSERT INTO produto (nome, id_categoria, id_fornecedor, preco_venda, estoque_atual, estoque_minimo,
ativo) VALUES
('Arroz tipo 1 - 5 kg', 1, 1, 27.9, 40, 10, TRUE),
('Feijão carioca - 1 kg', 1, 1, 8.49, 55, 15, TRUE),
('Açúcar refinado - 1 kg', 1, 1, 4.99, 8, 15, TRUE),
('Óleo de soja - 900 ml', 1, 1, 7.29, 30, 10, TRUE),
('Macarrão espaguete - 500 g', 1, 1, 5.49, 3, 10, TRUE),
('Café torrado - 500 g', 1, 1, 18.9, 25, 8, TRUE),
('Refrigerante cola - 2 L', 2, 2, 9.5, 48, 12, TRUE),
('Suco de uva integral - 1 L', 2, 2, 12.9, 14, 6, TRUE),
('Água mineral - 1,5 L', 2, 2, 3.2, 90, 24, TRUE),
('Chá gelado - 1,5 L', 2, 2, 6.9, 0, 6, FALSE),
('Detergente líquido - 500 ml', 3, 3, 2.99, 70, 20, TRUE),
('Sabão em pó - 1 kg', 3, 3, 14.5, 18, 8, TRUE),
('Papel higiênico - 12 rolos', 3, 3, 19.9, 22, 10, TRUE),
('Sabonete - 85 g', 3, 3, 2.49, 6, 20, TRUE),
('Desinfetante - 2 L', 3, 3, 9.9, 25, 8, TRUE),
('Banana prata - kg', 4, 4, 6.99, 30, 10, TRUE),
('Tomate - kg', 4, 4, 8.99, 4, 8, TRUE),
('Batata - kg', 4, 4, 5.49, 35, 10, TRUE),
('Cebola - kg', 4, 4, 4.99, 28, 10, TRUE),
('Pão francês - kg', 5, 6, 16.9, 15, 5, TRUE),
('Bolo de fubá - 350 g', 5, 6, 9.9, 9, 4, TRUE),
('Leite integral - 1 L', 6, 5, 5.49, 80, 30, TRUE),
('Queijo mussarela - kg', 6, 5, 42.9, 7, 5, TRUE),
('Iogurte natural - 170 g', 6, 5, 2.99, 40, 12, TRUE),
('Manteiga - 200 g', 6, 5, 13.9, 12, 6, TRUE);

INSERT INTO cliente (nome, cpf, email, telefone, cidade, data_cadastro) VALUES
('Ana Beatriz Souza', '10000000001', 'ana.souza@email.example', '27 99900-0001', 'Vitória', '2025-03-10'),
('Bruno Carvalho', '10000000002', 'bruno.carvalho@email.example', '27 99900-0002', 'Vila Velha', '2025-03-22'),
('Carla Mendes', '10000000003', NULL, '27 99900-0003', 'Serra', '2025-04-05'),
('Diego Almeida', '10000000004', 'diego.almeida@email.example', '27 99900-0004', 'Cariacica', '2025-05-17'),
('Eduarda Lima', '10000000005', 'eduarda.lima@email.example', '27 99900-0005', 'Vitória', '2025-06-01'),
('Fábio Nogueira', '10000000006', NULL, '27 99900-0006', 'Serra', '2025-06-30'),
('Gabriela Torres', '10000000007', 'gabriela.torres@email.example', '27 99900-0007', 'Vila Velha', '2025-08-12'),
('Henrique Duarte', '10000000008', 'henrique.duarte@email.example', '27 99900-0008', 'Vitória', '2025-09-03'),
('Isabela Rocha', '10000000009', 'isabela.rocha@email.example', '27 99900-0009', 'Cariacica', '2025-10-21'),
('João Pedro Lacerda', '10000000010', 'joao.lacerda@email.example', '27 99900-0010', 'Serra', '2025-11-09'),
('Karina Fontes', '10000000011', 'karina.fontes@email.example', '27 99900-0011', 'Vitória', '2026-01-15'),
('Lucas Moreira', '10000000012', NULL, '27 99900-0012', 'Vila Velha', '2026-02-02'),
('Marina Teixeira', '10000000013', 'marina.teixeira@email.example', '27 99900-0013', 'Vitória', '2026-03-18'),
('Natan Freitas', '10000000014', 'natan.freitas@email.example', '27 99900-0014', 'Serra', '2026-04-27');

INSERT INTO pedido (id_cliente, data_pedido, status) VALUES
(1, '2026-08-03', 'pago'),
(2, '2026-08-05', 'pago'),
(3, '2026-08-07', 'pago'),
(1, '2026-08-10', 'pago'),
(4, '2026-08-12', 'pago'),
(5, '2026-08-14', 'pago'),
(2, '2026-08-18', 'pago'),
(6, '2026-08-21', 'cancelado'),
(7, '2026-08-24', 'pago'),
(8, '2026-08-27', 'pago'),
(1, '2026-08-31', 'pago'),
(9, '2026-09-01', 'pago'),
(2, '2026-09-03', 'pago'),
(10, '2026-09-05', 'pago'),
(1, '2026-09-08', 'pago'),

```

```
(3, '2026-09-10', 'pago'),
(11, '2026-09-12', 'pago'),
(12, '2026-09-15', 'pago'),
(5, '2026-09-17', 'pago'),
(2, '2026-09-19', 'cancelado'),
(4, '2026-09-22', 'pago'),
(7, '2026-09-24', 'pago'),
(1, '2026-09-26', 'aberto'),
(8, '2026-09-29', 'aberto');
```

```
INSERT INTO item_pedido (id_pedido, id_produto, quantidade, preco_unitario) VALUES
```

```
(1, 5, 1, 5.49),
(1, 14, 1, 2.49),
(1, 22, 12, 4.49),
(2, 13, 1, 19.9),
(2, 20, 3, 16.9),
(3, 2, 2, 8.49),
(3, 3, 1, 4.99),
(3, 15, 1, 9.9),
(4, 15, 1, 9.9),
(4, 19, 6, 4.99),
(5, 8, 4, 12.9),
(5, 22, 12, 4.49),
(6, 14, 1, 2.49),
(6, 20, 1, 16.9),
(7, 5, 2, 5.49),
(7, 19, 2, 4.99),
(8, 4, 2, 7.29),
(8, 19, 3, 4.99),
(8, 20, 6, 16.9),
(9, 4, 1, 7.29),
(9, 20, 2, 16.9),
(9, 22, 2, 4.49),
(10, 2, 3, 8.49),
(10, 20, 1, 16.9),
(11, 12, 2, 14.5),
(11, 15, 3, 9.9),
(11, 19, 2, 4.99),
(11, 23, 2, 42.9),
(12, 6, 6, 18.9),
(12, 8, 1, 12.9),
(12, 24, 1, 2.99),
(13, 12, 4, 14.5),
(13, 17, 2, 8.99),
(13, 18, 2, 5.49),
(14, 4, 2, 7.29),
(14, 18, 1, 5.49),
(15, 5, 1, 5.49),
(15, 15, 4, 9.9),
(15, 17, 1, 8.99),
(16, 12, 3, 14.5),
(16, 13, 2, 19.9),
(16, 24, 3, 2.99),
(17, 3, 4, 4.99),
(17, 9, 2, 3.2),
(17, 17, 1, 8.99),
(17, 24, 4, 2.99),
(18, 20, 6, 16.9),
(18, 22, 6, 5.49),
(18, 23, 2, 42.9),
(19, 1, 2, 27.9),
(19, 13, 1, 19.9),
(19, 16, 3, 6.99),
(19, 23, 1, 42.9),
(20, 2, 4, 8.49),
(20, 5, 1, 5.49),
(20, 7, 6, 9.5),
(20, 11, 2, 2.99),
(21, 3, 3, 4.99),
(21, 6, 2, 18.9),
(21, 14, 1, 2.49),
(21, 16, 6, 6.99),
```

```
(22, 9, 6, 3.2),  
(22, 15, 4, 9.9),  
(22, 19, 2, 4.99),  
(22, 24, 1, 2.99),  
(23, 3, 1, 4.99),  
(23, 5, 4, 5.49),  
(23, 6, 1, 18.9),  
(24, 17, 1, 8.99),  
(24, 20, 2, 16.9);
```

Aula 12 – Consultas I: seleção, filtros, ordenação e datas

Nesta aula

- **Etapa:** 2 – Construir e manipular
- **Indicador:** I3 (principal)
- **Conhecimentos:** C2 (listagens, ordenação, consultas, datas)
- **Habilidades:** H1, H4 · **Atitudes/valores:** V3
- **Você vai entregar:** a lista de consultas resolvidas e documentadas

No caso do Mercadinho

Banco carregado, chega a hora de perguntar. O dono manda mensagens ao longo do dia: "quais produtos estão acabando?", "quem comprou em setembro?", "me passa os clientes sem e-mail para eu pedir o contato". Cada pergunta de negócio vira uma **consulta SQL**.

Conceito: o comando SELECT

SELECT é o comando de **consulta**. Sua forma mais usada:

```
SELECT  colunas ou expressões
FROM    tabela
WHERE   condição
ORDER BY colunas
LIMIT  quantidade;
```

Atenção: a **ordem em que se escreve** é essa, mas o banco **processa** primeiro o **FROM**, depois o **WHERE**, depois o **SELECT** e, por fim, o **ORDER BY** e o **LIMIT**.

Recurso	Para que serve	Exemplo
*	Todas as colunas (use só para explorar)	<code>SELECT * FROM cliente</code>
Apelido (AS)	Dar nome a uma coluna ou expressão	<code>preco_venda * 1.1 AS novo_preco</code>
WHERE	Filtrar linhas	<code>WHERE ativo</code>
AND, OR, NOT	Combinar condições	<code>WHERE ativo AND preco_venda < 10</code>
BETWEEN	Faixa (inclui os extremos)	<code>preco_venda BETWEEN 5 AND 10</code>
IN	Lista de valores	<code>cidade IN ('Serra', 'Vitória')</code>
LIKE / ILIKE	Padrão de texto (% = qualquer trecho; ILIKE ignora maiúsculas)	<code>nome ILIKE '%leite%'</code>
IS NULL / IS NOT NULL	Testar valor vazio	<code>email IS NULL</code>
DISTINCT	Eliminar repetições	<code>SELECT DISTINCT cidade</code>
ORDER BY	Ordenar (ASC padrão, DESC decrescente)	<code>ORDER BY preco_venda DESC</code>
LIMIT	Limitar o número de linhas	<code>LIMIT 5</code>

Atenção: valor vazio não é igual a nada

NULL representa "desconhecido". Por isso `email = NULL` **nunca** é verdadeiro. Use sempre `email IS NULL`.

Datas

O PostgreSQL oferece recursos para trabalhar com datas:

Recurso	Resultado
<code>CURRENT_DATE</code>	A data de hoje.
<code>data + 30</code> ou <code>data - data</code>	Soma dias a uma data, ou calcula a diferença em dias.
<code>date_trunc('month', data)</code>	Primeiro dia do mês da data.
<code>extract(month FROM data)</code>	Parte da data (ano, mês, dia).
<code>to_char(data, 'DD/MM/YYYY')</code>	Data formatada como texto.
<code>age(data1, data2)</code>	Intervalo entre datas em anos, meses e dias.
<code>INTERVAL '1 month'</code>	Um intervalo de tempo, que pode ser somado a datas.

Dica

Para filtrar um mês inteiro, prefira `data >= '2026-09-01' AND data < '2026-10-01'`. É claro, funciona com `date` e `timestamp` e não depende de formato de exibição.

Demonstração

Veja como uma pergunta de negócio vira consulta.

"Quais produtos estão abaixo do estoque mínimo?"

```
SELECT id_produto, nome, estoque_atual, estoque_minimo
FROM produto
WHERE estoque_atual < estoque_minimo
ORDER BY estoque_atual;
```

O resultado esperado tem **5 linhas** (entre elas, Chá gelado, Macarrão espagete e Tomate).

"Quais pedidos foram feitos em setembro de 2026?"

```
SELECT id_pedido, data_pedido, status
FROM pedido
WHERE data_pedido >= DATE '2026-09-01'
AND data_pedido < DATE '2026-10-01'
ORDER BY data_pedido;
```

São **13 pedidos**. Compare com **BETWEEN**, que inclui os dois extremos:

```
SELECT count(*) FROM pedido WHERE data_pedido BETWEEN '2026-09-01' AND '2026-09-30';
```

"Quais clientes não têm e-mail?"

```
SELECT nome, cidade FROM cliente WHERE email IS NULL;
```

Expressões, apelidos, datas e texto:

```
SELECT id_pedido,
       data_pedido,
       to_char(data_pedido, 'DD/MM/YYYY') AS data_br,
       extract(month FROM data_pedido)    AS mes,
       DATE '2026-09-30' - data_pedido    AS dias_ate_fim_de_setembro
FROM pedido
ORDER BY data_pedido
LIMIT 3;
```

```
SELECT nome, round(preco_venda * 1.10, 2) AS preco_com_reajuste
FROM produto
WHERE id_categoria = 2
ORDER BY nome;
```

```
SELECT nome, preco_venda FROM produto WHERE nome ILIKE '%leite%' OR nome ILIKE 'sab%';
SELECT DISTINCT cidade FROM cliente ORDER BY cidade;
SELECT nome FROM cliente WHERE cidade IN ('Serra', 'Cariacica') ORDER BY nome;
SELECT nome, preco_venda FROM produto WHERE ativo ORDER BY preco_venda DESC LIMIT 5;
```

```
SELECT nome, data_cadastro, age(DATE '2026-10-01', data_cadastro) AS tempo_de_casa
FROM cliente
ORDER BY data_cadastro
LIMIT 3;
```

Ação

Resolva as demandas abaixo. Salve em `03_scripts/04_consultas_basicas.sql`, uma consulta por demanda, com um comentário descrevendo a pergunta de negócio antes de cada uma (`-- D1: ...`).

#	Demanda do dono
D1	Lista de produtos ativos , do mais caro para o mais barato, mostrando nome e preço.
D2	Produtos da categoria 3 (Higiene e Limpeza) com preço abaixo de R\$ 10,00 .
D3	Os três produtos mais baratos.
D4	Cientes cadastrados em 2026 .
D5	Cientes de Vitória ou Vila Velha , em ordem alfabética.
D6	Produtos cujo nome contém "pão" ou "pó" (cuidado com acentos e maiúsculas).
D7	Pedidos pagos de agosto de 2026 .
D8	Pedidos ainda abertos e há quantos dias foram feitos, tomando 30/09/2026 como "hoje".
D9	Produtos sem estoque (estoque zero) ou inativos .
D10	Fornecedores sem e-mail cadastrado.

Reflexão

- Compare duas formas de resolver D7 (**BETWEEN** e **>= / <**). Elas dão o mesmo resultado? Qual é mais segura se a coluna fosse **timestamp**?
- Em D6, o que acontece com **LIKE** e **ILIKE**? Que cuidado é necessário com acentos?
- Por que **ORDER BY** precisa ser explícito se você quer um resultado em uma ordem previsível?
- Escolha duas das suas consultas e compare com as de outra dupla quanto à **correção** (devolve só o que foi pedido?) e à **legibilidade** (alguém entende sem explicação?).

Nova ação

Reescreva duas consultas melhorando a legibilidade: use apelidos claros, indentação, uma cláusula por linha, comentários. Documente cada uma com o formato:

```
-- D?: <pergunta de negócio>
-- Tabelas: ... Filtro: ... Ordenação: ...
-- Resultado esperado: <nº de linhas>
```

Desafio

O dono pede: "os produtos que **não** estão abaixo do mínimo, mas estão a **menos de 5 unidades** do mínimo". Escreva a consulta e explique, em uma frase, a lógica.

Síntese e entrega

- Uma consulta é uma **pergunta de negócio** traduzida: `SELECT ... FROM ... WHERE ... ORDER BY`.
- Valores vazios (`NULL`) exigem `IS NULL`.
- Datas têm funções próprias; para intervalos, prefira `>=` e `<`.

Evidência desta aula

`04_consultas_basicas.sql` com D1 a D10 executadas e comentadas, e as duas consultas reescritas. Avaliação do indicador I3.

Aula 13 – Consultas II: agregação e agrupamento

Nesta aula

- **Etapa:** 2 – Construir e manipular
- **Indicador:** I3 (principal)
- **Conhecimentos:** C2 (operações em consultas, datas)
- **Habilidades:** H4 · **Atitudes/valores:** V3
- **Você vai entregar:** consultas de agregação com indicadores gerenciais

No caso do Mercadinho

O dono não quer ver 70 linhas de itens vendidos: quer **números que ajudem a decidir**. Quantos pedidos por mês? Qual o preço médio por categoria? Quanto o mercado fatura por pedido? Para isso, o SQL **resume** muitas linhas em poucas.

Conceito: funções de agregação

Funções de agregação calculam **um único valor** a partir de várias linhas:

Função	Resultado
<code>count(*)</code>	Número de linhas
<code>count(coluna)</code>	Número de linhas em que a coluna não é vazia
<code>count(DISTINCT coluna)</code>	Número de valores diferentes
<code>sum(coluna)</code>	Soma
<code>avg(coluna)</code>	Média
<code>min(coluna), max(coluna)</code>	Menor e maior valor

GROUP BY e HAVING

GROUP BY forma **grupos** de linhas com o mesmo valor, e a função de agregação é calculada **em cada grupo**.

HAVING filtra os **grupos** depois de formados.

	Filtra	Quando atua
WHERE	Linhas	Antes de agrupar
HAVING	Grupos	Depois de agrupar (pode usar funções de agregação)

Atenção

Em uma consulta com **GROUP BY**, toda coluna do **SELECT** precisa estar no **GROUP BY** ou dentro de uma função de agregação.

Demonstração

Totais gerais:

```
SELECT count(*) AS total_produtos,
       count(*) FILTER (WHERE ativo) AS ativos
FROM produto;
```

```
SELECT count(*) AS clientes, count(email) AS com_email FROM cliente;
```

`count(email)` só conta os e-mails preenchidos: aqui, 11 de 14 clientes.

Resumo por grupo:

```
SELECT id_categoria,
       count(*) AS qtd_produtos,
       round(avg(preco_venda), 2) AS preco_medio,
       min(preco_venda) AS menor_preco,
       max(preco_venda) AS maior_preco
FROM produto
GROUP BY id_categoria
ORDER BY id_categoria;
```

```
SELECT id_fornecedor, sum(estoque_atual) AS itens_em_estoque
FROM produto
GROUP BY id_fornecedor
ORDER BY id_fornecedor;
```

HAVING: só os grupos que atendem à condição

```
SELECT id_categoria, count(*) AS qtd_produtos
FROM produto
GROUP BY id_categoria
HAVING count(*) >= 5
ORDER BY id_categoria;
```

Vendas por período:

```
SELECT to_char(date_trunc('month', data_pedido), 'YYYY-MM') AS mes,
       status,
       count(*) AS pedidos
FROM pedido
GROUP BY 1, 2
ORDER BY 1, 2;
```

(`GROUP BY 1, 2` significa "agrupe pela primeira e pela segunda coluna do `SELECT`".)

Valor das vendas e ticket médio. O **ticket médio** é o valor médio por pedido: faturamento dividido pelo número de pedidos. Calculamos o valor de cada item multiplicando quantidade por preço unitário:

```
SELECT sum(quantidade * preco_unitario) AS faturamento_itens,
       count(DISTINCT id_pedido) AS pedidos,
       round(sum(quantidade * preco_unitario)
             / count(DISTINCT id_pedido), 2) AS ticket_medio
FROM item_pedido;
```

O resultado é 1.877,60 de faturamento em 24 pedidos, com ticket médio de **R\$ 78,23**. Note que esse cálculo inclui pedidos cancelados e abertos: na aula 14, com junções, você poderá filtrar por status.

Ação

Resolva as demandas gerenciais. Salve em `03_scripts/05_consultas_agregadas.sql`.

#	Demanda da gerência
G1	Quantos pedidos existem em cada status ?
G2	Quantos pedidos foram feitos em agosto e quantos em setembro ?

#	Demanda da gerência
G3	Preço médio , mínimo e máximo dos produtos ativos de cada categoria.
G4	Total de itens em estoque por fornecedor, mostrando só os fornecedores com mais de 140 itens .
G5	Quantidade vendida por produto (use o <code>id_produto</code>), dos mais vendidos aos menos vendidos; mostre os 5 primeiros.
G6	Total de cada pedido (soma dos itens), mostrando apenas os pedidos acima de R\$ 150,00.
G7	Quantos produtos têm preço acima de R\$ 10,00 em cada categoria?
G8	Quantos clientes há em cada cidade?

Reflexão

1. Qual a diferença entre `count(*)` e `count(email)`? E entre `count(id_pedido)` e `count(DISTINCT id_pedido)` na tabela `item_pedido`?
2. Em G4, o filtro foi feito com `WHERE` ou com `HAVING`? Por quê? O que aconteceria se você tentasse usar `WHERE sum(...) > 140`?
3. Em G6, por que não é possível obter o total do pedido sem agregar? O que a ausência da coluna `total` na tabela `pedido` (decisão da aula 6) ensina sobre dados **derivados**?
4. As consultas G5 e G6 mostram apenas números de identificação. Que informação falta para um gerente entender o resultado? (Você resolve isso na aula 14.)

Nova ação

Refine as consultas para novos pedidos da gerência:

1. Em G2, mostre também o **percentual** de cada mês sobre o total de pedidos.
2. Em G3, acrescente a **quantidade de produtos** e ordene pelo maior preço médio.
3. Crie uma nova consulta: **maior e menor quantidade comprada** em um único item, e o **número de itens por pedido** (em média).

Desafio

O dono pergunta: "qual o **valor médio de um item** vendido, e qual foi o item de maior valor?" Escreva as duas consultas, e discuta se a média de `preco_unitario` seria uma boa resposta à primeira pergunta ou se é preciso considerar as quantidades.

Síntese e entrega

- Funções de agregação **resumem** linhas; `GROUP BY` forma grupos; `HAVING` filtra grupos.
- `WHERE` age antes de agrupar; `HAVING`, depois.
- Dados **derivados** (totais, médias) são calculados nas consultas, e não guardados.

Evidência desta aula

`05_consultas_agregadas.sql` com G1 a G8 e as consultas refinadas. Avaliação do indicador I3.

Aula 14 – Junções

Nesta aula

- **Etapa:** 2 – Construir e manipular
- **Indicador:** I3 (principal)
- **Conhecimentos:** C2 (junção de dados); C1 (relacionamentos e chaves)
- **Habilidades:** H4 · **Atitudes/valores:** V3
- **Você vai entregar:** consultas com junções e a interpretação dos resultados

No caso do Mercadinho

Como o banco está normalizado, a informação do negócio está **espalhada**: o nome do cliente está em **cliente**, o pedido em **pedido**, os produtos em **produto**, as quantidades em **item_pedido**. Para responder "quanto a Ana Beatriz gastou?" é preciso **juntar** essas tabelas, usando as **chaves estrangeiras** como ponte. Esse é o **JOIN**.

Conceito: juntar tabelas

Um **JOIN** combina linhas de duas tabelas **quando uma condição é verdadeira**, normalmente "a FK de uma é igual à PK da outra":

```
SELECT ...
FROM pedido p
JOIN cliente c ON c.id_cliente = p.id_cliente;
```

p e **c** são **apelidos** de tabela, que evitam repetir nomes e resolvem colunas com o mesmo nome em tabelas diferentes.

Tipos de junção

Tipo	O que devolve	Quando usar
INNER JOIN (ou só JOIN)	Só as linhas que têm correspondência nas duas tabelas	Quando só interessa o que tem par
LEFT JOIN	Todas as linhas da tabela da esquerda, com a correspondência da direita ou vazio (NULL)	"Mostre tudo, mesmo que não tenha par"
RIGHT JOIN	O contrário do LEFT JOIN	Pouco usado: troque a ordem das tabelas e use LEFT
FULL JOIN	Todas as linhas das duas, com ou sem par	Comparar dois conjuntos
CROSS JOIN	Todas as combinações possíveis (produto cartesiano)	Casos especiais; cuidado com o tamanho

Um truque valioso: **LEFT JOIN ... WHERE tabela_da_direita.chave IS NULL** mostra o que **não tem correspondência** (clientes que nunca compraram, produtos nunca vendidos).

O cuidado com linhas duplicadas

Ao juntar uma tabela "pai" a uma "filha", cada linha do pai se repete **uma vez para cada filha**. Um pedido com 3 itens aparece 3 vezes. Se você contar linhas achando que são pedidos, **vai contar errado**.

Demonstração

Pedidos com o nome do cliente (**INNER JOIN**):

```
SELECT p.id_pedido, p.data_pedido, c.nome AS cliente, p.status
FROM pedido p
JOIN cliente c ON c.id_cliente = p.id_cliente
ORDER BY p.id_pedido
LIMIT 3;
```

Itens de um pedido, passando por três tabelas:

```
SELECT p.id_pedido,
       pr.nome AS produto,
       i.quantidade,
       i.preco_unitario,
       i.quantidade * i.preco_unitario AS subtotal
FROM item_pedido i
JOIN pedido p ON p.id_pedido = i.id_pedido
JOIN produto pr ON pr.id_produto = i.id_produto
WHERE p.id_pedido = 1
ORDER BY pr.nome;
```

Junção com agregação: pedidos por cliente

```
SELECT c.nome, count(*) AS pedidos
FROM cliente c
JOIN pedido p ON p.id_cliente = c.id_cliente
GROUP BY c.nome
ORDER BY pedidos DESC, c.nome;
```

Apenas **12 clientes** aparecem: os outros dois nunca fizeram pedido. Para listá-los, use o **LEFT JOIN**:

```
SELECT c.nome
FROM cliente c
LEFT JOIN pedido p ON p.id_cliente = c.id_cliente
WHERE p.id_pedido IS NULL
ORDER BY c.nome;
```

Do mesmo modo, estes **LEFT JOIN** encontram fornecedores sem produtos e produtos nunca vendidos:

```
SELECT f.razao_social
FROM fornecedor f
LEFT JOIN produto pr ON pr.id_fornecedor = f.id_fornecedor
WHERE pr.id_produto IS NULL;
```

```
SELECT pr.nome
FROM produto pr
LEFT JOIN item_pedido i ON i.id_produto = pr.id_produto
WHERE i.id_pedido IS NULL
ORDER BY pr.nome;
```

Faturamento por mês, só de pedidos pagos:

```
SELECT to_char(date_trunc('month', p.data_pedido), 'YYYY-MM') AS mes,
       sum(i.quantidade * i.preco_unitario) AS faturamento
FROM pedido p
JOIN item_pedido i ON i.id_pedido = p.id_pedido
WHERE p.status = 'pago'
GROUP BY 1
ORDER BY 1;
```

Faturamento por categoria (quatro tabelas):

```
SELECT cat.nome AS categoria,
       sum(i.quantidade * i.preco_unitario) AS faturamento
FROM item_pedido i
JOIN produto pr ON pr.id_produto = i.id_produto
JOIN categoria cat ON cat.id_categoria = pr.id_categoria
JOIN pedido p ON p.id_pedido = i.id_pedido
WHERE p.status = 'pago'
GROUP BY cat.nome
ORDER BY faturamento DESC;
```

A armadilha da duplicação. Veja que o mesmo **JOIN** conta 70 linhas, mas só 24 pedidos distintos:

```
SELECT count(*) FROM pedido p JOIN item_pedido i ON i.id_pedido = p.id_pedido;
SELECT count(DISTINCT p.id_pedido) FROM pedido p JOIN item_pedido i ON i.id_pedido = p.id_pedido;
```

LEFT JOIN com contagem: categorias e quantidade de produtos (incluindo as sem produto):

```
SELECT cat.nome AS categoria, count(pr.id_produto) AS produtos
FROM categoria cat
LEFT JOIN produto pr ON pr.id_categoria = cat.id_categoria
GROUP BY cat.nome
ORDER BY produtos, cat.nome;
```

Note que a categoria Pet Shop aparece com **zero** produtos. Com um **INNER JOIN**, ela sumiria do resultado. Observe também que usamos **count(pr.id_produto)** e não **count(*)**: **count(*)** contaria a linha do Pet Shop como 1.

Ação

Resolva as demandas. Salve em **03_scripts/06_consultas_junc.sql**.

#	Demanda
J1	Lista de produtos com o nome da categoria e do fornecedor , em ordem de categoria e nome.
J2	Itens do pedido 18 , com nome do produto, quantidade, preço e subtotal, e o total do pedido (use uma segunda consulta para o total).
J3	Total gasto por cliente , considerando só pedidos pagos , do maior para o menor, mostrando o nome.
J4	Quantidade vendida por produto (com o nome), considerando só pedidos pagos, os cinco mais vendidos.
J5	Cientes que nunca compraram .
J6	Produtos abaixo do estoque mínimo com o nome do fornecedor (para a gerência saber de quem comprar).
J7	Fornecedores com a quantidade de produtos que fornecem, incluindo os sem produto.
J8	Faturamento por cidade do cliente , só de pedidos pagos.

Reflexão

1. Em J3, por que somar sem filtrar o status daria um número **diferente**? Qual o impacto para o dono?
2. Em J7, o que muda se você trocar **LEFT JOIN** por **JOIN**? Mostre com o resultado.
3. Qual dos resultados poderia ter **linhas duplicadas** por causa de uma junção? Como perceber? Que consulta de verificação você escreveria?
4. Em quais casos o **LEFT JOIN** com **IS NULL** é a ferramenta certa?

Nova ação

Corrija as consultas com resultado inconsistente. Faça esta verificação cruzada: some o faturamento por mês (da Demonstração) e compare com a soma do faturamento por categoria. Os totais **devem ser iguais**. Se não forem, procure a causa (filtro faltando? junção errada?) e corrija.

Desafio

Escreva uma consulta que mostre, para cada **cliente**, o **valor do maior pedido** que ele já fez (só pedidos pagos), com o nome e a data do pedido de maior valor. Dica: pense em dois passos; na aula 16 você aprende uma forma elegante de combiná-los.

Síntese e entrega

- **JOIN** usa a **chave estrangeira** como ponte entre tabelas.
- **INNER JOIN** devolve só pares; **LEFT JOIN** mantém tudo da esquerda e permite achar o que **não tem par**.
- Junções multiplicam linhas: confira contagens com **count(DISTINCT ...)**.

Evidência desta aula

`06_consultas_junc.sql` (J1 a J8) com a verificação cruzada dos totais. Avaliação do indicador I3.

Aula 15 – União, interseção e diferença de dados

Nesta aula

- **Etapas:** 2 – Construir e manipular
- **Indicador:** I3 (principal)
- **Conhecimentos:** C2 (união, intersecção e junção de dados); C1 (teoria dos conjuntos)
- **Habilidades:** H4 · **Atitudes/valores:** V3
- **Você vai entregar:** consultas com operações de conjuntos e a justificativa

No caso do Mercadinho

A campanha de fidelidade precisa de listas: "clientes que compraram **em agosto e também em setembro**", "clientes de agosto que **não voltaram** em setembro", "todos os contatos do mercado, clientes e fornecedores, em uma lista só". Cada lista é uma **operação entre conjuntos** de linhas.

Conceito: operações de conjuntos

O resultado de uma consulta é um **conjunto de linhas**. A SQL oferece operadores que combinam dois resultados, como na teoria dos conjuntos:

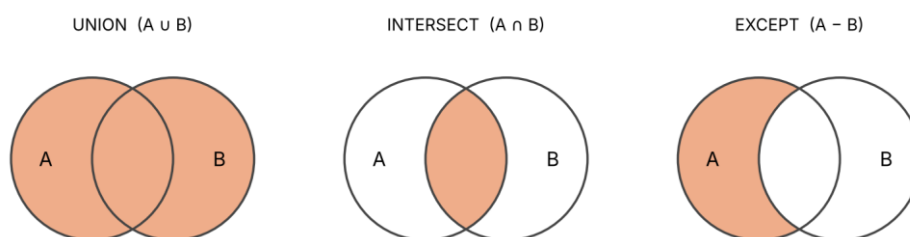


Figura 2 – Operações de conjuntos: união, interseção e diferença.

Operador	Resultado	Observação
UNION	Linhas que estão em A ou em B, sem repetir	Elimina duplicadas
UNION ALL	Todas as linhas de A e de B	Mantém duplicadas (mais rápido)
INTERSECT	Linhas que estão em A e em B	
EXCEPT	Linhas que estão em A e não estão em B	A ordem importa

Regras de compatibilidade: as duas consultas precisam ter o **mesmo número de colunas**, na mesma ordem e com **tipos compatíveis**. O nome das colunas do resultado vem da **primeira** consulta, e o **ORDER BY** final vale para o conjunto todo.

Conjuntos e junções

JOIN combina **colunas** de tabelas diferentes, lado a lado. Os operadores de conjuntos combinam **linhas** de consultas diferentes, uma sobre a outra.

Demonstração

Clientes que compraram em agosto (A) e em setembro (B):

```
SELECT c.nome FROM cliente c JOIN pedido p ON p.id_cliente = c.id_cliente
WHERE p.data_pedido BETWEEN '2026-08-01' AND '2026-08-31'
INTERSECT
SELECT c.nome FROM cliente c JOIN pedido p ON p.id_cliente = c.id_cliente
WHERE p.data_pedido BETWEEN '2026-09-01' AND '2026-09-30'
ORDER BY 1;
```

São **7 clientes**. Trocando o operador:

```
SELECT c.nome FROM cliente c JOIN pedido p ON p.id_cliente = c.id_cliente
WHERE p.data_pedido BETWEEN '2026-08-01' AND '2026-08-31'
EXCEPT
SELECT c.nome FROM cliente c JOIN pedido p ON p.id_cliente = c.id_cliente
WHERE p.data_pedido BETWEEN '2026-09-01' AND '2026-09-30'
ORDER BY 1;
```

Só **Fábio Nogueira** comprou em agosto e não voltou em setembro. Com **UNION**, a lista de quem comprou em pelo menos um dos dois meses tem 12 clientes:

```
SELECT c.nome FROM cliente c JOIN pedido p ON p.id_cliente = c.id_cliente
WHERE p.data_pedido BETWEEN '2026-08-01' AND '2026-08-31'
UNION
SELECT c.nome FROM cliente c JOIN pedido p ON p.id_cliente = c.id_cliente
WHERE p.data_pedido BETWEEN '2026-09-01' AND '2026-09-30'
ORDER BY 1;
```

Lista única de contatos, com a origem de cada um:

```
SELECT nome AS contato, 'cliente' AS origem, cidade FROM cliente
UNION ALL
SELECT razao_social, 'fornecedor', cidade FROM fornecedor
ORDER BY cidade, contato
LIMIT 6;
```

UNION e UNION ALL: a diferença (duplicadas):

```
SELECT cidade FROM cliente UNION SELECT cidade FROM fornecedor ORDER BY 1;
SELECT cidade FROM cliente UNION ALL SELECT cidade FROM fornecedor;
```

A primeira devolve 5 cidades distintas; a segunda, 21 linhas (14 clientes + 7 fornecedores), com repetições.

Cidades de fornecedores onde não há nenhum cliente:

```
SELECT cidade FROM fornecedor
EXCEPT
SELECT cidade FROM cliente;
```

Quando as consultas são incompatíveis: número de colunas diferente é erro.

```
Este comando deve gerar erro (é o esperado)
SELECT nome, cidade FROM cliente
UNION
SELECT razao_social FROM fornecedor;
```

Ação

Resolva e salve em `03_scripts/07_consultas_conjuntos.sql`.

#	Demanda
U1	Clientes que compraram em agosto e em setembro.

#	Demanda
U2	Clientes que compraram em setembro e não compraram em agosto (clientes "novos").
U3	Lista de todas as cidades em que o Mercadinho tem clientes ou fornecedores, sem repetição.
U4	Cidades em que há cliente e fornecedor ao mesmo tempo.
U5	Produtos que foram vendidos em agosto e em setembro.
U6	Produtos vendidos em agosto e não vendidos em setembro.
U7	Lista única de contatos (nome, tipo, e-mail), reunindo clientes e fornecedores com e-mail preenchido .

Para cada demanda, **desenhe** no caderno ou quadro os dois conjuntos (A e B) e escreva quais elementos entram em cada região do diagrama de Venn.

Reflexão

1. Qual a relação entre a **teoria dos conjuntos** e cada operador da SQL? Em que a **EXCEPT** difere da **INTERSECT** quanto à ordem das consultas?
2. Por que o **UNION ALL** é mais eficiente que o **UNION**? Em que situação você prefere **UNION ALL**?
3. O que aconteceria na demonstração se as duas consultas devolvessem os tipos **texto** e **número** na mesma posição?
4. Dá para obter U1 sem **INTERSECT**, usando apenas **JOIN**? Qual a vantagem de cada forma?

Nova ação

Reescreva duas das consultas usando uma alternativa (por exemplo, U1 com **JOIN** ou **IN**, e U2 com **LEFT JOIN/NOT EXISTS**) e **compare os resultados** com os originais. Registre se são idênticos e qual solução você acha mais clara.

Desafio

O dono quer um "painel de clientes" com três grupos em uma única consulta: **clientes novos** em setembro, **recorrentes** (compraram nos dois meses) e **perdidos** (só em agosto). Escreva uma consulta que devolva **nome** e **grupo**, usando operadores de conjuntos e um valor fixo como segunda coluna.

Síntese e entrega

- Resultados de consultas são **conjuntos de linhas**; **UNION**, **INTERSECT** e **EXCEPT** os combinam.
- As consultas combinadas precisam ter **colunas compatíveis** em número, ordem e tipo.
- **UNION ALL** mantém as duplicadas.

Evidência desta aula

07_consultas_conjuntos.sql (U1 a U7), os diagramas e a comparação entre as duas formas de resolver. Avaliação do indicador I3.

Aula 16 – Subconsultas

Nesta aula

- **Etapa:** 2 – Construir e manipular
- **Indicador:** I3 (principal)
- **Conhecimentos:** C2 (subconsultas SQL); revisão de SQL com IA sob supervisão (complementação didática)
- **Habilidades:** H4 · **Atitudes/valores:** V3
- **Você vai entregar:** consultas comparadas, com justificativa e validação

No caso do Mercadinho

Algumas perguntas do dono só se respondem em **dois passos**: "quais produtos custam **mais que a média**?" (primeiro é preciso saber a média) e "quais pedidos valem **mais que o ticket médio**?" (primeiro é preciso saber o ticket médio). Uma **subconsulta** é uma consulta dentro de outra, que resolve o primeiro passo.

Conceito: consulta dentro de consulta

Uma subconsulta fica entre parênteses e pode aparecer em vários lugares:

Onde	O que devolve	Exemplo de uso
No WHERE , com operador de comparação	Um valor (escalar)	<code>preco_venda > (SELECT avg(preco_venda) FROM produto)</code>
No WHERE , com IN / NOT IN	Uma lista de valores	<code>id_cliente IN (SELECT id_cliente FROM pedido)</code>
No WHERE , com EXISTS / NOT EXISTS	Existe ou não alguma linha	clientes para os quais existe pedido
No FROM	Uma tabela temporária (tabela derivada)	total por pedido, usado como se fosse uma tabela
No SELECT	Um valor por linha (subconsulta correlacionada)	quantidade de pedidos de cada cliente

Uma subconsulta é **correlacionada** quando usa colunas da consulta de fora e, por isso, é reavaliada para cada linha.

Atenção: o perigo do NOT IN com NULL

Se a lista do **NOT IN** contiver um **único NULL**, o resultado é **vazio**, mesmo quando deveria haver linhas. Por isso, para "não existe", prefira **NOT EXISTS** ou **LEFT JOIN ... IS NULL**.

Subconsulta ou junção?

Muitas perguntas podem ser resolvidas das duas formas. Em geral: use **junção** quando precisa **mostrar colunas** das duas tabelas; use **subconsulta** quando só precisa **filtrar** ou comparar com um valor calculado. A forma mais **legível** costuma ser a melhor. Há ainda a cláusula **WITH** (expressão de tabela comum), que dá um **nome** a uma subconsulta no início da consulta e deixa tudo mais fácil de ler.

Demonstração

Subconsulta escalar: produtos acima do preço médio

```
SELECT nome, preco_venda
FROM produto
WHERE preco_venda > (SELECT avg(preco_venda) FROM produto)
ORDER BY preco_venda DESC;
```

São **8 produtos**. Teste a subconsulta sozinha para ver o valor da média.

Cientes sem compras: três formas (todas devolvem os mesmos 2 clientes)

```
SELECT nome FROM cliente
WHERE id_cliente NOT IN (SELECT id_cliente FROM pedido)
ORDER BY nome;
```

```
SELECT nome FROM cliente c
WHERE NOT EXISTS (SELECT 1 FROM pedido p WHERE p.id_cliente = c.id_cliente)
ORDER BY nome;
```

```
SELECT c.nome FROM cliente c
LEFT JOIN pedido p ON p.id_cliente = c.id_cliente
WHERE p.id_pedido IS NULL
ORDER BY c.nome;
```

A armadilha do **NOT IN** com **NULL**: deveria devolver 14 clientes, mas devolve 0.

```
SELECT count(*) AS resultado
FROM cliente
WHERE id_cliente NOT IN (SELECT NULL::integer);
```

Lista de valores: produtos que já venderam 10 unidades ou mais

```
SELECT nome FROM produto
WHERE id_produto IN (SELECT id_produto FROM item_pedido GROUP BY id_produto HAVING sum(quantidade) >= 10)
ORDER BY nome;
```

Subconsulta no **FROM**: o ticket médio de verdade (média dos totais dos pedidos)

```
SELECT round(avg(total), 2) AS ticket_medio
FROM (SELECT id_pedido, sum(quantidade * preco_unitario) AS total
      FROM item_pedido
      GROUP BY id_pedido) t;
```

Pedidos acima do ticket médio (subconsulta no **FROM** e no **WHERE** juntas)

```
SELECT p.id_pedido, t.total
FROM pedido p
JOIN (SELECT id_pedido, sum(quantidade * preco_unitario) AS total
      FROM item_pedido GROUP BY id_pedido) t ON t.id_pedido = p.id_pedido
WHERE t.total > (SELECT avg(total)
                FROM (SELECT sum(quantidade * preco_unitario) AS total
                      FROM item_pedido GROUP BY id_pedido) x)
ORDER BY t.total DESC;
```

Essa consulta funciona, mas é difícil de ler. Compare com a mesma ideia usando **WITH**:

```
WITH total_pedido AS (
  SELECT id_pedido, sum(quantidade * preco_unitario) AS total
  FROM item_pedido
  GROUP BY id_pedido
)
SELECT round(avg(total), 2) AS ticket_medio, max(total) AS maior_pedido
FROM total_pedido;
```

Subconsulta correlacionada no SELECT:

```
SELECT nome,
       (SELECT count(*) FROM pedido p WHERE p.id_cliente = c.id_cliente) AS pedidos
FROM cliente c
ORDER BY pedidos DESC, nome
LIMIT 3;
```

Ação

Resolva as demandas com **subconsultas** e salve em `03_scripts/08_subconsultas.sql`.

#	Demanda
S1	Produtos mais baratos que a média da sua própria categoria (subconsulta correlacionada).
S2	Produtos que nunca foram vendidos , com NOT EXISTS .
S3	Cientes que fizeram algum pedido cancelado (EXISTS).
S4	Pedidos cujo total é maior que o ticket médio , mostrando o nome do cliente (reescreva a consulta da Demonstração com WITH e acrescente a junção).
S5	O produto mais vendido em quantidade (nome e total vendido), usando uma subconsulta para o máximo.
S6	Cientes cujo total gasto (pedidos pagos) é maior que a média dos totais gastos pelos clientes.
S7	Retome o desafio da aula 14 : para cada cliente, o valor do maior pedido pago.

Reflexão

1. Para S2, compare a solução com **NOT EXISTS** e com **LEFT JOIN ... IS NULL**. Os resultados são iguais? Qual é mais legível?
2. Quando é melhor usar **subconsulta** e quando é melhor usar **junção**? Dê um exemplo das suas consultas para cada caso.
3. Por que a consulta do ticket médio exige uma subconsulta **no FROM**? O que aconteceria com **avg(sum(...))**?
4. Em quais das suas consultas o **WITH** melhorou a leitura?

Revisão de SQL com IA sob supervisão**Complementação didática**

O uso de IA generativa não consta no Plano de Curso. Ele é incluído apenas como **apoio à revisão**, para você praticar o **pensamento crítico** sobre código (indicador I3). A responsabilidade pelo resultado continua sendo sua.

Siga este protocolo, com a autorização do professor:

1. Escolha uma consulta sua que funcione (por exemplo, S4 ou S6).
2. **Não envie** senhas, endereços de conexão nem dados reais. Envie só o **esquema** (tabelas e colunas) e a consulta, com dados fictícios.
3. Peça uma revisão, por exemplo: "Revise esta consulta PostgreSQL quanto à correção, legibilidade e desempenho. Explique cada sugestão."
4. **Valide tudo**: execute a consulta sugerida, **compare o resultado** com o da sua versão (mesmas linhas? mesmos totais?) e explique, com suas palavras, **o que cada mudança faz**.
5. Registre: o que a IA sugeriu, o que você **aceitou** e o que **recusou**, e **por quê**.

Uma sugestão que você não consegue explicar **não deve ser usada**.

Nova ação

Reescreva duas consultas usando a alternativa (de subconsulta para junção, ou de junção para subconsulta/**WITH**), **compare** os resultados e registre:

```
Consulta: S?      Forma original: ...      Forma alternativa: ...
Resultados iguais? ( ) Sim ( ) Não – Se não, por quê?
Qual é mais legível e por quê: ...
```

Desafio

O dono pede: "para cada categoria, qual é o produto **mais caro**?". Resolva de duas maneiras diferentes (por exemplo, com subconsulta correlacionada e com subconsulta no **FROM**) e confira se devolvem o mesmo resultado.

Síntese e entrega

- **Subconsulta** é uma consulta dentro de outra: devolve um valor, uma lista ou uma tabela.
- Prefira **NOT EXISTS** ou **LEFT JOIN ... IS NULL** a **NOT IN** quando houver chance de **NULL**.
- **WITH** dá nome a um passo e deixa a consulta legível.
- Sugestões de IA, como qualquer código de terceiros, **precisam ser validadas**.

Evidência desta aula

08_subconsultas.sql (S1 a S7), as comparações entre as duas formas de resolver e o registro da revisão com IA, com a validação. Avaliação do indicador I3.

Aula 17 – Views e índices

Nesta aula

- **Etapa:** 2 – Construir e manipular
- **Indicadores:** I3 (principal) · I1 (apoio)
- **Conhecimentos:** C2 (views, índices); C1 (visão, índices); plano de execução (complementação didática)
- **Habilidades:** H1, H2 · **Atitudes/valores:** V3
- **Você vai entregar:** views e índices criados e documentados

No caso do Mercadinho

O dono quer relatórios prontos: a gerente abre "estoque crítico" toda manhã, o financeiro consulta "vendas por categoria" toda semana. Digitar uma consulta de quatro junções toda vez é cansativo e arriscado. E, conforme os dados crescem, algumas consultas começam a ficar lentas. Para isso existem as **views** e os **índices**.

Conceito: views

Uma **view** é uma consulta com nome, guardada no banco. Ela se comporta como uma tabela virtual: você consulta com `SELECT ... FROM nome_da_view`. A view **não guarda dados**: ela executa a consulta toda vez que é usada, mostrando o estado atual das tabelas.

Para que serve:

- **Simplificar:** esconde junções complexas atrás de um nome simples.
- **Padronizar:** todos usam o mesmo cálculo (por exemplo, "subtotal" e "estoque crítico").
- **Proteger:** pode mostrar apenas algumas colunas (sem o CPF, por exemplo). Você vai usar isso na aula 21.

```
CREATE VIEW nome_da_view AS
SELECT ... ;
```

```
CREATE OR REPLACE VIEW nome_da_view AS SELECT ... ; -- altera a definição
DROP VIEW nome_da_view; -- remove a view (os dados ficam)
```

Existe também a **view materializada**, que guarda o resultado e precisa ser atualizada (**REFRESH**). Ela é útil para relatórios pesados, mas não é necessária neste projeto.

Conceito: índices

Um **índice** é uma estrutura auxiliar que permite ao banco **achar linhas sem ler a tabela inteira**, como o índice remissivo de um livro. Os tipos de índice mais comuns usam uma estrutura de árvore (B-tree).

Fato	Detalhe
Já existem índices	Para cada chave primária e cada restrição única , o PostgreSQL cria um índice automaticamente.
Chave estrangeira não gera índice	Colunas como <code>id_cliente</code> em <code>pedido</code> ou <code>id_produto</code> em <code>item_pedido</code> não são indexadas automaticamente. Se você as usa muito em junções e filtros, indexar ajuda.

Fato	Detalhe
Índices têm custo	Ocupam espaço e deixam INSERT , UPDATE e DELETE mais lentos, porque o índice precisa ser atualizado.
Nem sempre são usados	Em tabelas pequenas, o banco prefere ler a tabela toda: é mais rápido.

```
CREATE INDEX nome_do_indice ON tabela (coluna);
CREATE UNIQUE INDEX nome ON tabela (coluna);           -- também impede repetição
CREATE INDEX nome ON tabela (coluna1, coluna2);       -- índice composto
DROP INDEX nome_do_indice;
```

Demonstração

Três views para o Mercadinho:

```
CREATE VIEW vw_itens_vendidos AS
SELECT p.id_pedido,
       p.data_pedido,
       p.status,
       c.nome AS cliente,
       pr.nome AS produto,
       cat.nome AS categoria,
       i.quantidade,
       i.preco_unitario,
       i.quantidade * i.preco_unitario AS subtotal
FROM item_pedido i
JOIN pedido p ON p.id_pedido = i.id_pedido
JOIN cliente c ON c.id_cliente = p.id_cliente
JOIN produto pr ON pr.id_produto = i.id_produto
JOIN categoria cat ON cat.id_categoria = pr.id_categoria;
```

```
CREATE VIEW vw_total_pedido AS
SELECT p.id_pedido,
       p.data_pedido,
       p.status,
       c.nome AS cliente,
       sum(i.quantidade * i.preco_unitario) AS total
FROM pedido p
JOIN cliente c ON c.id_cliente = p.id_cliente
JOIN item_pedido i ON i.id_pedido = p.id_pedido
GROUP BY p.id_pedido, p.data_pedido, p.status, c.nome;
```

```
CREATE VIEW vw_estoque_critico AS
SELECT pr.id_produto,
       pr.nome,
       pr.estoque_atual,
       pr.estoque_minimo,
       pr.estoque_minimo - pr.estoque_atual AS falta,
       f.razao_social AS fornecedor,
       f.telefone AS telefone_fornecedor
FROM produto pr
JOIN fornecedor f ON f.id_fornecedor = pr.id_fornecedor
WHERE pr.ativo
AND pr.estoque_atual < pr.estoque_minimo;
```

Usando as views como se fossem tabelas:

```
SELECT nome, estoque_atual, estoque_minimo, falta, fornecedor
FROM vw_estoque_critico
ORDER BY falta DESC;
```

```
SELECT categoria, sum(subtotal) AS faturamento
FROM vw_itens_vendidos
WHERE status = 'pago'
GROUP BY categoria
ORDER BY faturamento DESC;
```

```
SELECT cliente, count(*) AS pedidos, round(avg(total), 2) AS ticket_medio
FROM vw_total_pedido
WHERE status = 'pago'
GROUP BY cliente
ORDER BY ticket_medio DESC
LIMIT 3;
```

Para **alterar** a definição de uma view (só é permitido acrescentar colunas no final):

```
CREATE OR REPLACE VIEW vw_estoque_critico AS
SELECT pr.id_produto,
       pr.nome,
       pr.estoque_atual,
       pr.estoque_minimo,
       pr.estoque_minimo - pr.estoque_atual AS falta,
       f.razao_social AS fornecedor,
       f.telefone AS telefone_fornecedor,
       f.email AS email_fornecedor
FROM produto pr
JOIN fornecedor f ON f.id_fornecedor = pr.id_fornecedor
WHERE pr.ativo
      AND pr.estoque_atual < pr.estoque_minimo;
```

Índices para as colunas de junção mais usadas:

```
CREATE INDEX idx_pedido_cliente      ON pedido (id_cliente);
CREATE INDEX idx_pedido_data        ON pedido (data_pedido);
CREATE INDEX idx_item_pedido_produto ON item_pedido (id_produto);
CREATE INDEX idx_produto_categoria  ON produto (id_categoria);
CREATE INDEX idx_produto_fornecedor ON produto (id_fornecedor);
```

Quais índices existem agora?

```
SELECT tablename, indexname, indexdef
FROM pg_indexes
WHERE schemaname = 'public'
ORDER BY tablename, indexname;
```

Na lista, aparecem tanto os índices que você criou quanto os criados automaticamente pelas chaves primárias e únicas.

Plano de execução: ver o índice trabalhando (complementação didática)

Complementação didática

O comando **EXPLAIN** não está listado nos conhecimentos do Plano. Ele é incluído para você **enxergar o benefício de um índice**, o que apoia a decisão de criá-lo (indicadores I3 e I1).

Com as tabelas pequenas do Mercadinho, o banco nem precisaria de índices. Para ver o efeito de verdade, crie uma tabela **de teste**, com 200 mil linhas, e depois apague:

```
CREATE TABLE exp_vendas AS
SELECT g AS id_venda,
       (g % 500) + 1 AS id_produto,
       DATE '2025-01-01' + (g % 365) AS data_venda,
       ((g * 7) % 100 + 1)::numeric(10,2) AS valor
FROM generate_series(1, 200000) AS g;

ANALYZE exp_vendas;
```

Veja **como o banco executa** uma busca sem índice:

```
EXPLAIN ANALYZE SELECT * FROM exp_vendas WHERE id_produto = 42;
```

No resultado, procure **Seq Scan** (leitura sequencial: a tabela inteira foi lida) e o tempo em **Execution Time**. Agora crie o índice e repita:

```
CREATE INDEX idx_exp_vendas_produto ON exp_vendas (id_produto);
ANALYZE exp_vendas;
EXPLAIN ANALYZE SELECT * FROM exp_vendas WHERE id_produto = 42;
```

O plano passa a usar o índice (**Index Scan** ou **Bitmap Index Scan**) e o tempo cai bastante. Os tempos variam de máquina para máquina; o que importa é a **mudança no plano**. Para terminar, remova a tabela de teste:

```
DROP TABLE exp_vendas;
```

O comando **ANALYZE** atualiza as estatísticas que o banco usa para escolher o plano. Em um banco real, essa e outras rotinas de manutenção são feitas com regularidade.

Ação

1. **Crie as views** da demonstração e uma **nova view** para a necessidade a seguir: **vw_vendas_mensais**, com o mês (YYYY-MM), o número de pedidos pagos e o faturamento. Salve tudo em **03_scripts/09_views.sql**.
2. **Use** cada view em pelo menos uma consulta (filtrando, ordenando e agrupando).
3. **Crie os índices** e liste os índices existentes. Salve em **03_scripts/10_indices.sql**.
4. **Experimente o EXPLAIN ANALYZE** com a tabela de teste e **registre** (copie) o plano antes e depois do índice, com os tempos.

Reflexão

1. Qual a diferença entre uma **view** e uma **tabela**? O que acontece com a view se um preço mudar na tabela **produto**?
2. Por que a chave estrangeira **não** gera índice automaticamente? Em quais das suas consultas das aulas 14 a 16 um índice em FK ajudaria?
3. Quais são o **custo** e o **benefício** de um índice? Em que tipo de tabela você **não** criaria índices extras?
4. Compare o plano **Seq Scan** com o plano com índice: o que mudou, e o que isso significa em tempo de resposta quando a tabela cresce?

Nova ação

Ajuste as estruturas e documente as decisões. Para cada view e cada índice, escreva uma linha no dicionário de dados (ou em **06_documentacao/objetos_do_banco.md**):

Objeto	Tipo	Finalidade	Quem usa	Observações
<code>vw_estoque_critico</code>	view	Produtos ativos abaixo do mínimo	Gerência (compras)	Inclui contato do fornecedor
<code>idx_pedido_cliente</code>	índice	Acelerar junção pedido x cliente	Consultas de pedidos por cliente	FK não é indexada automaticamente

Atualize o **diagrama** ou o modelo físico para registrar os objetos novos (indicador I1).

Desafio

Crie uma view `vw_clientes_publico` que mostre `id_cliente`, `nome` e `cidade`, **sem** o CPF, o e-mail e o telefone. Para quem esse tipo de view seria útil e que problema ela ajuda a evitar? (Você usa essa ideia na aula 21.)

Síntese e entrega

- **View** é uma consulta com nome: simplifica, padroniza e protege.
- **Índice** acelera buscas, mas tem custo; PK e **UNIQUE** já criam índices, FK **não**.
- **EXPLAIN ANALYZE** mostra **como** o banco executa a consulta.

Evidência desta aula

`09_views.sql`, `10_indices.sql` e `objetos_do_banco.md`, com os planos antes e depois do índice.
Avaliação dos indicadores I3 e I1.

Aula 18 – Checkpoint 2: mudança de requisito e manutenção do banco

Nesta aula

- **Etapa:** 2 – Construir e manipular
- **Indicadores:** I3 (principal) e I1 – **menção parcial**
- **Conhecimentos:** C1 e C2 integrados
- **Habilidades:** H1, H2, H4 · **Atitudes/valores:** V1, V2, V3
- **Você vai entregar:** o banco alterado, com scripts e documentação atualizados (e uma parte individual em SQL)

No caso do Mercadinho

Terça-feira, o dono chega com uma novidade: "agora aceitamos **Pix, cartão e dinheiro**, e vamos dar **descontos** em alguns itens. Preciso saber quanto entra por forma de pagamento e quanto foi dado de desconto". O banco já está em uso, com dados. Você precisa **mudar a estrutura sem perder nada**. Mudanças de requisito são parte normal da profissão e testam a **flexibilidade** da equipe.

O novo requisito

Código	Requisito
RD-N1	Cada pedido pago deve registrar a forma de pagamento : pix, cartão ou dinheiro .
RD-N2	Cada item de pedido pode ter um desconto percentual , de 0 a 100, com valor padrão 0.
RN-N1	Pedidos abertos ou cancelados podem não ter forma de pagamento. Pedidos pagos devem ter.
RN-N2	O total do pedido deve considerar o desconto: $\text{quantidade} \times \text{preço unitário} \times (1 - \text{desconto}/100)$.
RD-N3	A gerência quer um relatório do faturamento por forma de pagamento e do total de descontos concedidos por mês.

Conceito: manutenção evolutiva do banco

Alterar um banco em uso exige método:

1. **Entender** o requisito e **perguntar** o que está ambíguo (os pedidos antigos pagos não têm forma de pagamento registrada: o que fazer com eles?).
2. **Atualizar o modelo** (DER, modelo físico, dicionário) **antes** do código, e versionar (v1.2).
3. **Ensaiar** no PostgreSQL **local**, com cópia dos dados, antes de aplicar no Supabase.
4. **Alterar a estrutura** (**ALTER TABLE**), **migrar** os dados existentes (**UPDATE**) e só então **impor** as restrições.
5. **Ajustar** consultas e views que dependem do que mudou.
6. **Testar** com consultas de verificação (por exemplo, o faturamento total antes e depois deve ser o mesmo quando todos os descontos são zero).
7. **Documentar** a mudança.

Atenção

Para impor uma restrição **NOT NULL** ou **CHECK** a uma coluna nova em uma tabela com dados, primeiro **preencha os dados existentes**. Do contrário, o banco recusará a alteração.

Ação

Atenda ao novo requisito **em dupla**:

1. **Atualize o modelo:** DER, modelo físico e dicionário de dados, em nova versão (**v1.2**), e registre no **historico_versoes.md**.
2. **Decida e justifique** o que fazer com os pedidos pagos que já existem (por exemplo, uma forma de pagamento provisória, ou um valor que indique "não informado") e anote como isso será tratado nos relatórios.
3. **Escreva o script** **03_scripts/11_mudanca_pagamento_desconto.sql** com: o **ALTER TABLE** das duas colunas novas; a **migração** dos dados existentes; as **restrições** (valores permitidos da forma de pagamento; faixa do desconto; regra "pedido pago exige forma de pagamento").
4. **Ensaie primeiro no PostgreSQL local**; depois aplique no Supabase.
5. **Ajuste as views** (**vw_itens_vendidos**, **vw_total_pedido**) para considerar o desconto.
6. **Escreva as consultas** do relatório RD-N3.
7. **Verifique:** o faturamento total (de pedidos pagos) **antes e depois** da mudança, com descontos zerados, deve ser idêntico (R\$ 1.555,58 no conjunto de dados da apostila). Depois, aplique um desconto de teste a um item e confira que o total do pedido diminui na quantia esperada.

Parte individual (em SQL)

Em um momento definido pelo professor, **cada aluno, individualmente**, resolve um conjunto de tarefas em SQL no banco do projeto, sem consultar o parceiro. O conjunto cobre o que foi estudado nas aulas 9 a 17:

- alterar a estrutura de uma tabela com restrição (DDL);
- inserir, atualizar e apagar dados com segurança (DML);
- consultas com filtro, agrupamento e junção;
- uma consulta com subconsulta;
- uma view.

O objetivo é que cada um **demonstre autonomia**: o trabalho em dupla não substitui a evidência individual do indicador I3.

Reflexão

1. Preencha a **autoavaliação** abaixo, indicando se você alcançou cada critério e **qual evidência** prova isso:

Critério observável	Indicador	Alcancei? (S / Em parte / N)	Evidência
O modelo (DER, físico e dicionário) foi atualizado antes do script.	I1		
O banco construído corresponde ao modelo atualizado (aferição).	I1		

Critério observável	Indicador	Alcansei? (S / Em parte / N)	Evidência
Os scripts de alteração são executáveis, comentados e reproduzíveis.	I3		
Os dados existentes foram migrados sem perda .	I3		
As restrições impedem dados inválidos (testei com exemplos).	I3		
As consultas e views usam corretamente junções e agregações.	I3		
Resolvi a parte individual com autonomia.	I3		
Usei termos técnicos corretamente ao explicar o que fiz.	H4		

1. **Receba o feedback** do professor e da outra dupla. Que ponto você não percebeu sozinho?
2. O que foi **mais difícil**: mudar a estrutura, migrar os dados ou ajustar as consultas? Por quê?
3. Como a **documentação atualizada** ajudou (ou teria ajudado) nessa mudança?

Nova ação

Corrija os pontos apontados no feedback e **atualize a documentação**: modelo físico final da etapa 2, dicionário de dados, `objetos_do_banco.md` e o histórico de versões. Guarde também os scripts do banco **como estão agora**: eles são a base da etapa 3 (importação, segurança e backup).

Desafio

O dono agora quer saber **qual forma de pagamento é a mais usada em cada mês**. Escreva a consulta e discuta o que ela mostra caso haja um empate.

Síntese e entrega

- Uma mudança de requisito pede **método**: perguntar, atualizar o modelo, ensaiar, migrar, impor restrições, verificar e documentar.
- Testes de verificação (totais antes e depois) dão **segurança** à alteração.
- Autoavaliação por critérios observáveis ajuda a saber **onde você está**.

Evidência desta aula

Banco alterado (Supabase e local), script `11_mudanca_pagamento_desconto.sql`, consultas do relatório, documentação atualizada, autoavaliação e a **parte individual em SQL. Menção parcial dos indicadores I3 e I1 (A/PA/NA)**, com feedback do professor.

Parte III – Movimentar, proteger e entregar

Aulas 19 a 24

Aula 19 – Importação de dados

Nesta aula

- **Etapas:** 3 – Movimentar, proteger e entregar
- **Indicadores:** I4 (principal) · I3 (apoio)
- **Conhecimentos:** C2 (importação e exportação de dados; integridade referencial; tipos de dados); C1 (dicionário de dados)
- **Habilidades:** H3, H4 · **Atitudes/valores:** V3, V4
- **Você vai entregar:** importação de um arquivo CSV com validação, registro das linhas rejeitadas e conferência

No caso do Mercadinho

O fornecedor da **Importadora Alfa** mandou, por e-mail, uma planilha com produtos novos para o catálogo. A planilha foi salva como arquivo **CSV**. Digitar linha por linha seria lento e sujeito a erros. Mas importar o arquivo "às cegas" é pior: um preço escrito como **abc**, uma categoria que não existe ou um produto repetido poderiam sujar o banco, ou fazer a importação inteira falhar sem dizer por quê.

Nesta aula você vai importar com **método**: o arquivo entra primeiro numa área de preparação, é **validado**, e só o que está correto passa para as tabelas do projeto. O que for rejeitado é **registrado**, com o motivo.

Conceito: importar dados

Importar é levar dados de uma fonte externa (um arquivo, outro sistema, outro banco) para dentro do banco. O inverso, **exportar**, é a próxima aula.

O formato mais comum de troca é o **CSV** (*comma-separated values*), um arquivo de texto em que cada linha é um registro e os campos são separados por um caractere. No Brasil, planilhas costumam gravar o CSV com **ponto e vírgula** (;) como separador e **vírgula decimal** (4,89), porque a vírgula já é usada nos números.

Antes de importar, descubra, **olhando o arquivo**:

Característica	Pergunta
Separador	É ;, , ou tabulação?
Cabeçalho	A primeira linha tem os nomes das colunas?
Codificação	O arquivo está em UTF-8? (acentos errados indicam outra codificação)
Formato dos números e datas	4,89 ou 4.89? 05/10/2026 ou 2026-10-05?
Valores vazios	Campo vazio significa "sem valor" (NULL)?
Chaves	O arquivo traz códigos que já existem no banco?

O comando \copy do psql

O **psql** tem o comando **\copy**, que lê um arquivo **do seu computador** e o carrega em uma tabela. Ele é um comando do **psql** (não é SQL): vem com a barra invertida e é escrito em **uma única linha**.

```
\copy tabela (coluna1, coluna2, ...) FROM 'arquivo.csv'
WITH (FORMAT csv, DELIMITER ';', HEADER true, ENCODING 'UTF8')
```

Opção	Significado
FORMAT csv	O arquivo está no formato CSV.
DELIMITER ';'	O separador de campos.
HEADER true	Ignora a primeira linha (cabecalho).
ENCODING 'UTF8'	Codificação do arquivo. Se foi salvo em outra (por exemplo, WIN1252), informe a correta.

Dica

O SQL Editor do Supabase **não** executa comandos `\copy`: use o `psql` no seu computador (conectado ao Supabase, como na aula 2, ou ao PostgreSQL local). O painel do Supabase também oferece importação de CSV pela interface; confira como funciona na sua conta. Qualquer que seja o caminho, vale o método desta aula: **validar antes de gravar nas tabelas finais**.

A área de preparação (*staging*)

A estratégia profissional é carregar o arquivo em uma **tabela de preparação**, na qual **todas as colunas são texto**. Assim, nenhuma linha é recusada por causa de tipo: tudo entra. Depois, com SQL, você **consulta** o que está certo e o que está errado, e só então copia as linhas válidas para as tabelas definitivas.

```
arquivo CSV → stg_produto (tudo texto) → validação (SQL) → produto
                                   ↓
                               linhas rejeitadas + motivo
```

Demonstração

Passo 1 – o arquivo. Crie, em um editor de texto, o arquivo `novos_produtos.csv` com o conteúdo abaixo e salve em **UTF-8**, na pasta `bom-preco-db/04_dados/` (a pasta onde você abrirá o `psql`). Observe-o antes de importar: ele tem 14 linhas de dados, e **nem todas estão corretas**.

Arquivo CSV (salve em UTF-8)

```
codigo_barras;nome;categoria;cnj_fornecedor;preco_venda;estoque_inicial
7890000000011;Macarrão espagete 500 g;Mercearia;10000000000101;4,89;60
7890000000028;Molho de tomate 340 g;Mercearia;10000000000101;3,59;80
7890000000035;Água com gás 500 ml;Bebidas;10000000000202;3,20;48
7890000000042;Detergente neutro 500 ml;Higiene e Limpeza;10000000000303;2,79;70
7890000000059;Banana prata (kg);Hortifruti;10000000000404;6,90;25
7890000000066;Iogurte natural 170 g;Frios e Laticínios;10000000000505;2,99;40
7890000000073;Pão francês (kg);Padaria;10000000000606;16,90;15
7890000000080;Ração para cães 1 kg;Pet Shop;10000000000707;18,50;20
7890000000028;Molho de tomate 340 g;Mercearia;10000000000101;3,59;80
7890000000097;Tempero completo 100 g;Condimentos;10000000000101;5,49;30
7890000000103;Biscoito recheado 140 g;Mercearia;10000000000999;2,49;50
7890000000110;Suco de uva 1 L;Bebidas;10000000000202;abc;24
7890000000127;Esponja de aço;Higiene e Limpeza;10000000000303;-5,00;40
7890000000134;Leite integral 1 L;Frios e Laticínios;10000000000505;5,19;
```

Atenção

Todos os códigos de barras, CNPJs e nomes do arquivo são **fictícios**. Em um caso real, arquivos de fornecedores e clientes podem conter dados pessoais ou comerciais sigilosos: guarde-os apenas onde tiver permissão e não os envie a terceiros (valor V4).

Passo 2 – a tabela de preparação. Todas as colunas são **text**. A coluna **linha** numera as linhas do arquivo e ajuda a localizar problemas.

```
CREATE TABLE stg_produto (
    linha          integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    codigo_barras  text,
    nome           text,
    categoria      text,
    cnpj_fornecedor text,
    preco_venda    text,
    estoque_inicial text
);
```

Passo 3 – carregar o arquivo. No **psql**, na pasta onde está o arquivo:

No **psql**

```
\copy stg_produto (codigo_barras, nome, categoria, cnpj_fornecedor, preco_venda, estoque_inicial)
FROM 'novos_produtos.csv' WITH (FORMAT csv, DELIMITER ';', HEADER true, ENCODING 'UTF8')
```

O **psql** responde **COPY 14**. Confira:

```
SELECT * FROM stg_produto ORDER BY linha;
```

Se os acentos aparecerem trocados por símbolos estranhos, o arquivo não está em UTF-8: salve novamente como "CSV UTF-8" ou use a codificação correta em **ENCODING**.

Passo 4 – a validação. Em vez de repetir a mesma lógica várias vezes, vamos guardá-la em uma **view** (aula 17), que classifica cada linha. O **CASE** verifica as regras **na ordem**: a primeira que falhar define o motivo.

```
CREATE VIEW vw_stg_produto_situacao AS
SELECT s.*,
CASE
    WHEN s.nome IS NULL OR trim(s.nome) = ''
    THEN 'nome vazio'
    WHEN s.codigo_barras IS NULL OR trim(s.codigo_barras) !~ '^[0-9]{8,14}$'
    THEN 'código de barras inválido'
    WHEN EXISTS (SELECT 1 FROM stg_produto o
        WHERE trim(o.codigo_barras) = trim(s.codigo_barras)
        AND o.linha < s.linha)
    THEN 'duplicada no arquivo'
    WHEN NOT EXISTS (SELECT 1 FROM categoria c WHERE c.nome = trim(s.categoria))
    THEN 'categoria inexistente'
    WHEN NOT EXISTS (SELECT 1 FROM fornecedor f WHERE f.cnpj = trim(s.cnpj_fornecedor))
    THEN 'fornecedor inexistente'
    WHEN trim(s.preco_venda) !~ '^-[0-9]+([0-9]{1,2})?$',
    THEN 'preço não numérico'
    WHEN replace(trim(s.preco_venda), ',', '.')::numeric <= 0
    THEN 'preço deve ser maior que zero'
    WHEN s.estoque_inicial IS NULL OR trim(s.estoque_inicial) !~ '^[0-9]+$'
    THEN 'estoque inválido'
    WHEN EXISTS (SELECT 1 FROM produto p WHERE p.codigo_barras = trim(s.codigo_barras))
    THEN 'já cadastrado'
    ELSE 'ok'
END AS situacao
FROM stg_produto s;
```

Pontos de atenção neste código:

- O operador **!~** significa "**não** combina com a expressão regular". **'^[0-9]+\$'** combina com texto formado só por dígitos. Expressões regulares foram usadas nas restrições da aula 10.
- A conversão **::numeric** só é feita **depois** de a regra anterior garantir que o texto é um número. Como o **CASE** avalia na ordem, **abc** nunca chega a ser convertido.
- A regra "**duplicada no arquivo**" compara cada linha com as linhas **anteriores** do arquivo (**o.linha < s.linha**): a primeira ocorrência vale, a repetição é rejeitada.
- A regra "**já cadastrado**" compara com os produtos que **já estão** na tabela **produto**. Ela usa a coluna **codigo_barras**, criada na aula 10.

Veja o resultado, primeiro o resumo e depois linha a linha:

```
SELECT situacao, count(*) AS linhas
FROM vw_stg_produto_situacao
GROUP BY situacao
ORDER BY situacao;
```

```
SELECT linha, nome, situacao
FROM vw_stg_produto_situacao
ORDER BY linha;
```

Passo 5 – guardar as rejeitadas. Antes de importar, registre o que **não** vai entrar, com o motivo. Isso precisa ser feito **antes**, porque, depois da importação, as linhas boas passarão a ser classificadas como "já cadastrado".

```
CREATE TABLE stg_produto_rejeitado AS
SELECT linha, codigo_barras, nome, categoria, cnpj_fornecedor,
       preco_venda, estoque_inicial, situacao
FROM vw_stg_produto_situacao
WHERE situacao <> 'ok';
```

Passo 6 – importar o que é válido. `INSERT ... SELECT` copia o resultado de uma consulta para a tabela. Aqui, as junções traduzem o **nome da categoria** e o **CNPJ** para as chaves (`id_categoria`, `id_fornecedor`), e as conversões transformam texto em número.

```
INSERT INTO produto (codigo_barras, nome, id_categoria, id_fornecedor,
                    preco_venda, estoque_atual)
SELECT trim(s.codigo_barras),
       trim(s.nome),
       c.id_categoria,
       f.id_fornecedor,
       replace(trim(s.preco_venda), ',', '.'):numeric,
       trim(s.estoque_inicial)::integer
FROM vw_stg_produto_situacao s
JOIN categoria c ON c.nome = trim(s.categoria)
JOIN fornecedor f ON f.cnpj = trim(s.cnpj_fornecedor)
WHERE s.situacao = 'ok';
```

Passo 7 – conferir. O número de produtos deve ter aumentado exatamente pelo número de linhas "ok", e a situação das linhas mudou:

```
SELECT count(*) AS produtos FROM produto;
```

```
SELECT situacao, count(*) AS linhas
FROM vw_stg_produto_situacao
GROUP BY situacao
ORDER BY situacao;
```

Repare que as 8 linhas válidas agora aparecem como "**já cadastrado**". Isso mostra que a importação é **repetível sem duplicar dados**: se alguém executar o `INSERT` do passo 6 de novo, nada entra pela segunda vez.

Passo 8 – corrigir e importar de novo. A categoria `Condimentos` não existia. O dono decidiu criá-la. Depois disso, a linha 10 passa a ser válida:

```
INSERT INTO categoria (nome) VALUES ('Condimentos');
```

Execute **novamente** o `INSERT ... SELECT` do passo 6 e confira: entra **uma** linha nova (o "Tempero completo"). As outras rejeições continuam, porque o problema está **no arquivo**, e não no banco: preço `abc`, preço negativo, estoque vazio e CNPJ inexistente precisam ser corrigidos **pelo fornecedor**.

```
SELECT nome, preco_venda, estoque_atual
FROM produto
WHERE codigo_barras LIKE '78900000%'
ORDER BY id_produto;
```

Complementação didática

Por que não importar direto na tabela `produto`? O `\copy` direto para uma tabela com restrições **falha por inteiro** na primeira linha inválida: nenhuma linha é gravada e a mensagem aponta só um problema por vez. A área de preparação permite ver **todos** os problemas de uma vez e decidir o que fazer. É uma prática comum de rotinas de carga de dados.

Ação

1. **Crie o arquivo `novos_produtos.csv`** exatamente como na demonstração e **registre** (em `06_documentacao/importacao.md`): separador, codificação, cabeçalho e formato de números.
2. **Execute os passos 2 a 7** em um banco de ensaio (o PostgreSQL local) e, depois, no Supabase. Salve os comandos em `03_scripts/12_importacao_produtos.sql`.
3. **Liste as linhas rejeitadas** com o motivo e **anote o que o fornecedor precisa corrigir**.
4. **Execute o passo 8** e confira o resultado final: **34 produtos e 8 categorias**.
5. **Execute o `INSERT` do passo 6 uma segunda vez** e confira que a contagem de produtos **não** mudou.

Reflexão

1. Por que a tabela de preparação tem **todas as colunas em texto**? O que aconteceria se `preco_venda` fosse `numeric` e o arquivo tivesse `abc`?
2. Por que as linhas rejeitadas precisaram ser guardadas **antes** da importação?
3. Qual a vantagem de ter a validação em uma **view** e não em uma consulta copiada várias vezes?
4. Das regras de validação, qual **protege a integridade referencial** (aula 5)? Qual protege a **integridade de domínio** (tipos e restrições, aula 7)?
5. A linha 14 tem estoque vazio. Seria correto assumir `0`? Quem deveria decidir isso, e por quê?
6. Que cuidado você deve ter com um arquivo recebido por e-mail que contém dados de clientes?

Nova ação

1. **Crie sua própria versão do arquivo**: 10 linhas, com **pelo menos quatro tipos diferentes de erro** (diferentes dos do exemplo, se puder). Troque o arquivo com outra dupla.
2. **Importe o arquivo recebido** com o método da aula, sem alterar a view. Registre o resumo por situação e devolva à outra dupla uma lista do que foi rejeitado e por quê.
3. **Melhore a validação**: acrescente uma regra que rejeite nomes com mais de 120 caracteres (o limite da coluna `nome`) e teste com uma linha criada por você.
4. Para encerrar o ensaio, **remova** os objetos de preparação, mantendo `stg_produto_rejeitado` como registro, se quiser:

```
DROP VIEW vw_stg_produto_situacao;
DROP TABLE stg_produto;
```

Desafio

O fornecedor agora quer enviar **atualizações de preço** de produtos que **já existem**, identificados pelo código de barras. Projete a área de preparação, as regras de validação (por exemplo, rejeitar reajustes acima de 30%) e o **UPDATE** que aplica os preços válidos. Execute-o dentro de uma transação, confira com um **SELECT** e termine com **ROLLBACK** enquanto estiver testando.

Síntese e entrega

- Importar com método: **conhecer o arquivo → carregar em preparação → validar → importar o válido → registrar o rejeitado → conferir**.
- O **\copy** do **psql** lê arquivos do seu computador; o separador, o cabeçalho e a codificação precisam ser informados.
- Importações bem feitas são **repetíveis** e **auditáveis**: sabe-se o que entrou, o que não entrou e por quê.

Evidência desta aula

novos_produtos.csv, **12_importacao_produtos.sql**, tabela de rejeitadas com motivos, conferência de contagens (34 produtos; 8 categorias) e **importacao.md**. Observação com checklist (**indicador I4**, em construção).

Aula 20 – Exportação de dados e migração

Nesta aula

- **Etapa:** 3 – Movimentar, proteger e entregar
- **Indicadores:** I4 (principal) · I3 (apoio)
- **Conhecimentos:** C2 (importação e exportação de dados; views); C3 (sigilo e integridade dos dados)
- **Habilidades:** H3, H4 · **Atitudes/valores:** V2, V4
- **Você vai entregar:** arquivos exportados para uso externo e uma migração entre bancos, conferida

No caso do Mercadinho

A contadora do Mercadinho pede o **faturamento por mês** em uma planilha. A gerente quer enviar ao fornecedor a **lista de linhas rejeitadas** da aula passada. E o dono decidiu **testar uma cópia completa do banco** em outro lugar antes de confiar no sistema. São três tipos de saída de dados:

Necessidade	Formato adequado
Entregar o resultado de uma consulta a uma pessoa (planilha)	CSV , pensado para quem vai abrir no Excel/LibreOffice
Entregar dados a outro sistema	CSV com formato previsível (separador, decimais, codificação combinados)
Levar o banco todo (estrutura e dados) a outro servidor	Arquivo de dump em SQL

Conceito: exportar dados

Exportar é gravar dados do banco em um arquivo ou em outro banco. Antes de exportar, responda:

1. **Para quem** vai o arquivo? (uma pessoa abre em planilha; um sistema lê por programa)
2. **Que dados** devem ir, e quais **não podem** ir? (sigilo: valor **V4**)
3. **Que formato** o destino espera? (separador, decimal, datas, codificação)
4. **Como conferir** que o que saiu é igual ao que está no banco?

\copy ... TO

O mesmo **\copy** da aula 19 funciona no sentido inverso. Você pode exportar uma tabela inteira ou o resultado de **qualquer consulta**, escrita entre parênteses:

```
\copy tabela TO 'arquivo.csv' WITH (FORMAT csv, DELIMITER ';', HEADER true, ENCODING 'UTF8')
\copy (SELECT ...) TO 'arquivo.csv' WITH (FORMAT csv, DELIMITER ';', HEADER true, ENCODING 'UTF8')
```

Exportar uma **consulta** (e não a tabela inteira) é a forma mais segura de controlar **quais colunas e quais linhas** saem.

Dump: levar o banco inteiro

O **pg_dump** é um programa (executado no terminal, e não no **psql**) que gera um arquivo com os comandos SQL necessários para **recriar** o banco: estrutura (**CREATE TABLE**, restrições, views) e dados (**COPY/INSERT**). Nesta aula você usa o formato **SQL puro**, que pode ser lido em um editor de texto e executado com o **psql**. Na aula 22, você conhecerá o formato próprio para backups.

Demonstração

Parte 1 – Uma view para o relatório da contadora. Se você criou `vw_vendas_mensais` na aula 17, ela já existe. O bloco abaixo mostra a versão básica (sem desconto):

Atenção

Se você já aplicou a mudança de requisito da aula 18, a sua versão da view **considera o desconto**. Nesse caso, **não execute** o bloco abaixo, porque ele a substituiria pela versão sem desconto. Use a sua versão nas exportações.

```
CREATE OR REPLACE VIEW vw_vendas_mensais AS
SELECT to_char(p.data_pedido, 'YYYY-MM')      AS mes,
       count(DISTINCT p.id_pedido)            AS pedidos,
       sum(i.quantidade * i.preco_unitario)    AS faturamento
FROM pedido p
JOIN item_pedido i ON i.id_pedido = p.id_pedido
WHERE p.status = 'pago'
GROUP BY to_char(p.data_pedido, 'YYYY-MM');
```

```
SELECT * FROM vw_vendas_mensais ORDER BY mes;
```

Parte 2 – Exportar para uma pessoa abrir na planilha. No `psql`, em uma linha:

No psql

```
\copy (SELECT mes, pedidos, round(faturamento, 2) AS faturamento FROM vw_vendas_mensais ORDER BY mes)
TO 'vendas_mensais.csv' WITH (FORMAT csv, DELIMITER ';', HEADER true, ENCODING 'UTF8')
```

Abra o arquivo em um editor de texto. O valor sai como `596.92`, com **ponto** decimal (o `round(..., 2)` garante duas casas; sem ele, a view com desconto da aula 18 devolveria um número com muitas casas): o formato do banco. Uma planilha configurada em português do Brasil pode tratar `596.92` como texto ou como data. Para quem vai abrir em planilha brasileira, exporte o número já com **vírgula**:

No psql

```
\copy (SELECT mes, pedidos, replace(round(faturamento, 2)::text, '.', ',') AS faturamento FROM
vw_vendas_mensais ORDER BY mes) TO 'vendas_mensais_br.csv' WITH (FORMAT csv, DELIMITER ';', HEADER
true, ENCODING 'UTF8')
```

Dica

Para pessoas, formate como elas esperam (vírgula decimal). **Para sistemas**, mantenha o formato do banco (ponto decimal, datas `AAAA-MM-DD`) e combine o formato por escrito. O importante é que o formato esteja **documentado**.

Parte 3 – Exportar sem expor dados sigilosos. O cadastro de clientes tem CPF, e-mail e telefone. A área de marketing só precisa de nome, cidade e data de cadastro:

No psql

```
\copy (SELECT id_cliente, nome, cidade, data_cadastro FROM cliente ORDER BY id_cliente) TO
'clientes_marketing.csv' WITH (FORMAT csv, DELIMITER ';', HEADER true, ENCODING 'UTF8')
```

O princípio é o do **mínimo necessário**: exporte apenas o que o destinatário precisa para o trabalho dele.

Parte 4 – Exportar as linhas rejeitadas da aula 19 para devolver ao fornecedor:

No psql

```
\copy (SELECT * FROM stg_produto_rejeitado ORDER BY linha) TO 'rejeitadas.csv' WITH (FORMAT csv,
DELIMITER ';', HEADER true, ENCODING 'UTF8')
```

Parte 5 – Migrar o banco inteiro com `pg_dump`. Gere o dump do esquema `public` (as tabelas do projeto) a partir do banco de origem. As opções `--no-owner` e `--no-privileges` evitam levar donos e permissões do ambiente de origem, que não existem no destino.

Do PostgreSQL local:

No terminal

```
pg_dump -h localhost -U comercio_app -d comercio -n public --no-owner --no-privileges -f
dump_comercio.sql
```

Do Supabase (use a conexão do *session pooler* da aula 2):

No terminal

```
pg_dump "postgresql://postgres.[REF]@[HOST-DO-POOLER]:5432/postgres" -n public --no-owner --no-
privileges -f dump_supabase.sql
```

Atenção

- Para o `pg_dump` funcionar, a **versão do programa no seu computador não pode ser mais antiga** que a versão do servidor. Se aparecer um erro de "server version mismatch", instale uma versão igual ou mais nova do cliente.
- O arquivo de dump contém **todos os dados**, inclusive CPFs. Trate-o como um documento sigiloso: não envie por e-mail nem o publique em repositórios.

Abra o arquivo no editor de texto e **leia** o início: você verá os comandos `CREATE TABLE`, as restrições e, mais adiante, os dados.

Parte 6 – Restaurar o dump em um banco novo e conferir. Crie um banco de destino (como administrador, no banco `postgres`):

Executar como administrador do servidor

```
CREATE DATABASE comercio_migracao OWNER comercio_app;
```

Execute o dump no novo banco:

No terminal

```
psql -h localhost -U comercio_app -d comercio_migracao -f dump_comercio.sql
```

Você verá a mensagem `ERROR: schema "public" already exists`. Ela é **esperada e inofensiva**: o dump pede para criar o esquema `public`, que já existe em todo banco novo. Os demais comandos são executados normalmente.

Parte 7 – A conferência. Uma migração só termina quando você **prova** que o destino é igual à origem. Execute a consulta abaixo **nos dois bancos** e compare:

```
SELECT 'categoria' AS tabela, count(*) AS linhas FROM categoria
UNION ALL SELECT 'fornecedor', count(*) FROM fornecedor
UNION ALL SELECT 'produto', count(*) FROM produto
UNION ALL SELECT 'cliente', count(*) FROM cliente
UNION ALL SELECT 'pedido', count(*) FROM pedido
UNION ALL SELECT 'item_pedido', count(*) FROM item_pedido;
```

Compare também um **total de negócio**, que testa os dados e não só a quantidade:

```
SELECT sum(i.quantidade * i.preco_unitario) AS faturamento_pago
FROM item_pedido i
JOIN pedido p ON p.id_pedido = i.id_pedido
WHERE p.status = 'pago';
```

Por fim, verifique se as **views** e as **restrições** também foram:

```
SELECT table_name FROM information_schema.views WHERE table_schema = 'public' ORDER BY table_name;
```

```
SELECT conrelid::regclass AS tabela, conname AS restricao, contype AS tipo
FROM pg_constraint
WHERE connamespace = 'public'::regnamespace
ORDER BY conrelid::regclass::text, conname;
```

Ação

1. **Gere os quatro arquivos CSV** da demonstração (partes 2 a 4). Guarde-os em `04_dados/exportados/` e abra cada um em um editor de texto e em uma planilha. Anote o que mudou entre os dois arquivos de faturamento (`vendas_mensais.csv` e `vendas_mensais_br.csv`).
2. **Registre em `06_documentacao/exportacao.md`**, para cada arquivo: finalidade, destinatário, colunas exportadas, formato e **por que as colunas sigilosas ficaram de fora**.
3. **Gere o dump** do banco local e (se possível) do Supabase. Abra o arquivo e **identifique** no texto onde estão: uma tabela, uma restrição, uma view e os dados de uma tabela.
4. **Restaurar o dump** em `comercio_migracao` e **execute a conferência** (parte 7). Compare os resultados e registre.

Reflexão

1. Por que o arquivo para a contadora e o arquivo para outro sistema podem ter formatos diferentes?
2. Qual a diferença entre exportar uma **tabela** e exportar o resultado de uma **consulta**? Em que situação a segunda é mais segura?
3. Por que a conferência da migração inclui um **total de negócio** (faturamento) e não só a contagem de linhas?
4. O que há de risco em guardar um arquivo de dump no seu computador, no e-mail ou em um repositório público? O que você faria para reduzir esse risco?
5. A importação (aula 19) e a exportação (aula 20) são operações inversas. O que você aprendeu em uma que ajudou na outra?

Nova ação

1. **Altere o banco de origem** (por exemplo, insira um cliente e um pedido de teste), gere um **novo dump** e restaure-o em um banco **novo** (`comercio_migracao2`). Confira os totais.
2. **Teste o formato**: exporte `vw_vendas_mensais` com `DELIMITER ','` e abra no Excel/LibreOffice. O que acontece com os valores decimais? Registre o aprendizado em `exportacao.md`.
3. **Crie um script de exportação** (`03_scripts/13_exportacoes.sql`) que reúna todos os `\copy` da aula, comentados, para que qualquer colega consiga repetir o processo.
4. Após terminar, apague do computador os arquivos de teste que contenham dados pessoais e que você não precisa guardar.

Desafio

A contadora pediu, além do faturamento por mês, o **faturamento por categoria em cada mês**. Escreva a consulta (use a view `vw_itens_vendidos`, se ela existir), exporte para CSV pronto para planilha brasileira e documente o formato.

Síntese e entrega

- Exportar exige decisões: **para quem, o quê, em qual formato e com qual proteção.**
- `\copy (consulta) TO` controla colunas e linhas; `pg_dump` leva a estrutura e os dados.
- Migração só termina com **conferência** entre origem e destino.

Evidência desta aula

Os arquivos CSV exportados, `exportacao.md`, os dumps, o `13_exportacoes.sql` e o registro da conferência da migração. **Menção parcial do indicador I4 (A/PA/NA)**, com feedback do professor.

Aula 21 – Segurança do banco de dados

Nesta aula

- **Etapas:** 3 – Movimentar, proteger e entregar
- **Indicadores:** I5 (apoio: planejamento de proteção) · I3 (apoio: scripts SQL)
- **Conhecimentos:** C3 (ameaças, controle de acesso, autenticação, criptografia, integridade e consistência, normas); C2 (views)
- **Habilidades:** H1, H4 · **Atitudes/valores:** V3, V4
- **Você vai entregar:** plano de controle de acesso implementado em SQL, matriz de ameaças e controles, e demonstração de criptografia

No caso do Mercadinho

Hoje, na planilha, qualquer pessoa que recebe o arquivo vê **tudo**: CPF dos clientes, preços de custo, telefones. No banco de dados será diferente, mas só se você **decidir quem pode fazer o quê**. A equipe do Mercadinho tem três perfis:

Perfil	O que precisa fazer
Estoquista	Consultar produtos e fornecedores; ajustar a quantidade em estoque. Não pode mudar preços.
Vendedor	Consultar produtos; registrar pedidos e itens; ver dados de contato dos clientes. Não precisa ver o CPF.
Gerência	Consultar tudo para os relatórios.

Esta aula transforma esse quadro em regras que o próprio banco faz cumprir.

Conceito: o que proteger e de quê

A segurança de dados busca preservar três propriedades, conhecidas pelas iniciais **CID**:

Propriedade	Significa	Exemplo no Mercadinho
Confidencialidade (sigilo)	Só quem tem permissão acessa o dado.	O CPF do cliente não pode ser visto por quem não precisa.
Integridade	O dado está correto e só muda de forma autorizada.	Um vendedor não pode alterar o preço de um produto.
Disponibilidade	O dado está acessível quando necessário.	O sistema de vendas precisa funcionar na hora do pico; e, se houver falha, o banco precisa ser recuperável (aulas 22 e 23).

A **consistência** é a garantia de que o banco passa sempre de um estado válido a outro estado válido. Você já usou três ferramentas para isso: **restrições** (aulas 7 e 10), **integridade referencial** (aula 5) e **transações** (aula 11).

Ameaças e controles

Uma **ameaça** é algo que pode causar dano aos dados; um **controle** é a medida que reduz esse risco. Falhas e ameaças podem ser técnicas, humanas ou de processo.

Ameaça	Propriedade afetada	Controle
Funcionário vê dados que não precisa (CPF)	Confidencialidade	Privilégios mínimos; views com dados mascarados
Funcionário altera preço sem autorização	Integridade	Privilégios por coluna; papéis separados
Digitação errada ou UPDATE sem WHERE	Integridade	Restrições; transações; privilégios limitados
Senha fraca ou compartilhada	Confidencialidade	Autenticação individual; senhas fortes; não compartilhar credenciais
Vazamento de arquivo de dump ou CSV	Confidencialidade	Exportar o mínimo; guardar em local controlado; criptografia
Falha de disco, erro de operação ou exclusão acidental	Disponibilidade	Backup e restore testados (aulas 22 e 23)
Alguém lê dados de outro usuário (no mesmo sistema)	Confidencialidade	Segurança em nível de linha (RLS)

Complementação didática

Normas. O Plano de Curso cita "normas" como conhecimento de segurança, sem indicar quais. Duas referências costumam ser usadas: a **Lei Geral de Proteção de Dados (LGPD, Lei nº 13.709/2018)**, que trata de dados pessoais, como nome, CPF e e-mail, e a família de normas **ISO/IEC 27001**, sobre gestão da segurança da informação. Elas são apresentadas aqui apenas como **contexto**, para você perceber por que sigilo e mínimo necessário não são opcionais. O estudo detalhado delas não faz parte desta UC.

Conceito: controle de acesso no PostgreSQL

No PostgreSQL, quem acessa o banco é representado por **papéis** (*roles*). Um papel pode ser:

- uma **pessoa ou aplicação** que faz login (tem senha); ou
- um **grupo** de permissões (sem login), do qual pessoas fazem parte.

Boa prática: **crie papéis de grupo com as permissões e associe as pessoas aos grupos**. Quando alguém muda de função, troca-se o grupo, sem refazer as permissões.

Comando	Para que serve
<code>CREATE ROLE nome NOLOGIN;</code>	Cria um papel de grupo.
<code>CREATE ROLE nome LOGIN PASSWORD '...' IN ROLE grupo;</code>	Cria uma pessoa/aplicação com login, já dentro de um grupo.
<code>GRANT privilégio ON objeto TO papel;</code>	Concede um privilégio.
<code>REVOKE privilégio ON objeto FROM papel;</code>	Retira um privilégio.
<code>SET ROLE papel; / RESET ROLE;</code>	Passa a agir como outro papel (para testar) e volta.

Privilégios comuns em tabelas: **SELECT** (ler), **INSERT** (inserir), **UPDATE** (alterar), **DELETE** (apagar). Eles podem ser concedidos na tabela inteira ou **apenas em colunas**: `GRANT SELECT (coluna1, coluna2) ON tabela TO papel`.

Princípio do privilégio mínimo

Cada papel recebe **apenas** as permissões necessárias para o trabalho. Começa-se sem nada e **acrescenta-se** o que for preciso, em vez de dar tudo e tentar retirar depois.

Autenticação

Autenticação é provar quem você é, normalmente com **usuário e senha**. No PostgreSQL, as senhas são guardadas em **forma de hash** (nunca em texto), por padrão com o método **scram-sha-256**:

```
SHOW password_encryption;
```

Regras práticas: uma conta **por pessoa**, senha **forte e individual**, **nunca** compartilhada em chats, planilhas ou no código. As senhas escritas nesta apostila (**troque_esta_senha**) são exemplos do **ambiente local de estudo**: troque-as.

Demonstração

Atenção

Os comandos de **papéis** são executados como **administrador** do banco (no PostgreSQL local, o usuário administrador; no Supabase, o usuário **postgres** no SQL Editor). Papéis valem para o **servidor inteiro**, e não para um banco apenas. Execute os experimentos primeiro no **PostgreSQL local**.

Parte 1 – Os três grupos de permissões.

```
CREATE ROLE papel_estoquista NOLOGIN;
CREATE ROLE papel_vendedor NOLOGIN;
CREATE ROLE papel_gerencia NOLOGIN;

-- permite que o administrador "vista" cada papel para testar
GRANT papel_estoquista, papel_vendedor, papel_gerencia TO CURRENT_USER;
```

Parte 2 – Dando permissões (do menor para o maior).

```
-- Estoquista: lê produtos/fornecedores e só altera as colunas de estoque
GRANT SELECT ON categoria, fornecedor, produto TO papel_estoquista;
GRANT UPDATE (estoque_atual, estoque_minimo) ON produto TO papel_estoquista;

-- Vendedor: lê produtos, lê o cliente SEM o CPF, registra pedidos e itens
GRANT SELECT ON categoria, produto TO papel_vendedor;
GRANT SELECT (id_cliente, nome, email, telefone, cidade, data_cadastro)
ON cliente TO papel_vendedor;
GRANT SELECT, INSERT, UPDATE ON pedido TO papel_vendedor;
GRANT SELECT, INSERT ON item_pedido TO papel_vendedor;

-- Gerência: lê tudo
GRANT SELECT ON ALL TABLES IN SCHEMA public TO papel_gerencia;
```

Parte 3 – Testando como cada papel. Um teste de segurança precisa mostrar o que **funciona** e o que é **negado**. Execute cada bloco inteiro; ao final, **RESET ROLE** volta a ser o administrador. (Se um erro interromper o bloco antes disso, execute **RESET ROLE**; sozinho.)

O vendedor lê o cliente, **sem o CPF**:

```
SET ROLE papel_vendedor;
SELECT current_user;
SELECT nome, cidade FROM cliente ORDER BY id_cliente LIMIT 3;
RESET ROLE;
```

O vendedor tenta ver o CPF (negado), e tenta **SELECT *** (também negado, porque ***** inclui o CPF):

```
Este comando deve gerar erro (é o esperado)
SET ROLE papel_vendedor;
SELECT cpf FROM cliente LIMIT 1;
```

Este comando deve gerar erro (é o esperado)

```
SET ROLE papel_vendedor;
SELECT * FROM cliente LIMIT 1;
```

O vendedor tenta mudar o preço de um produto (negado) e apagar um pedido (negado):

Este comando deve gerar erro (é o esperado)

```
SET ROLE papel_vendedor;
UPDATE produto SET preco_venda = 1 WHERE id_produto = 1;
```

Este comando deve gerar erro (é o esperado)

```
SET ROLE papel_vendedor;
DELETE FROM pedido WHERE id_pedido = 24;
```

O vendedor registra um pedido (permitido). A transação termina em **ROLLBACK** para não mudar o banco:

```
SET ROLE papel_vendedor;
BEGIN;
INSERT INTO pedido (id_cliente) VALUES (1) RETURNING id_pedido, status;
ROLLBACK;
RESET ROLE;
```

O estoquista ajusta o estoque (permitido) e tenta mudar o preço (negado):

```
SET ROLE papel_estoquista;
BEGIN;
UPDATE produto SET estoque_atual = estoque_atual + 1
WHERE id_produto = 1
RETURNING nome, estoque_atual;
ROLLBACK;
RESET ROLE;
```

Este comando deve gerar erro (é o esperado)

```
SET ROLE papel_estoquista;
UPDATE produto SET preco_venda = 1 WHERE id_produto = 1;
```

Execute **RESET ROLE**; e prossiga.

Parte 4 – Auditoria: quem pode o quê? Os privilégios podem ser consultados. A consulta abaixo monta uma **matriz de permissões** das quatro tabelas principais:

```
SELECT r.rolname AS papel,
       t.tablename AS tabela,
       has_table_privilege(r.rolname, format('public.%I', t.tablename), 'SELECT') AS ler,
       has_table_privilege(r.rolname, format('public.%I', t.tablename), 'INSERT') AS inserir,
       has_table_privilege(r.rolname, format('public.%I', t.tablename), 'UPDATE') AS alterar,
       has_table_privilege(r.rolname, format('public.%I', t.tablename), 'DELETE') AS apagar
FROM pg_roles r
CROSS JOIN pg_tables t
WHERE r.rolname LIKE 'papel\_%'
      AND t.schemaname = 'public'
      AND t.tablename IN ('cliente', 'produto', 'pedido', 'item_pedido')
ORDER BY r.rolname, t.tablename;
```

Observe que o vendedor aparece com **ler = f** em **cliente**: o privilégio é **por coluna**, não por tabela. Consulte as colunas:

```
SELECT has_column_privilege('papel_vendedor', 'cliente', 'nome', 'SELECT') AS nome,
       has_column_privilege('papel_vendedor', 'cliente', 'cpf', 'SELECT') AS cpf;
```

Parte 5 – Mostrar o dado de forma protegida: view com máscara. A gerência precisa conferir cadastros por CPF, mas o vendedor só precisa **reconhecer** o cliente. Uma view pode mostrar o CPF **parcialmente oculto**. A view roda com os privilégios do **seu dono**, então o vendedor pode usá-la mesmo sem acesso à coluna **cpf**.

```
CREATE VIEW vw_cliente_mascarado AS
SELECT id_cliente,
       nome,
       substr(cpf, 1, 3) || '.*.*.*-' || substr(cpf, 10, 2) AS cpf_mascarado,
       cidade
FROM cliente;

GRANT SELECT ON vw_cliente_mascarado TO papel_vendedor;
```

```
SET ROLE papel_vendedor;
SELECT * FROM vw_cliente_mascarado ORDER BY id_cliente LIMIT 3;
RESET ROLE;
```

Parte 6 – Pessoas dentro dos grupos. Cada pessoa tem **conta própria** e pertence a um grupo:

```
CREATE ROLE ana_vendas LOGIN PASSWORD 'troque_esta_senha' IN ROLE papel_vendedor;
CREATE ROLE bruno_vendas LOGIN PASSWORD 'troque_esta_senha' IN ROLE papel_vendedor;
```

```
SELECT r.rolname AS membro, g.rolname AS grupo
FROM pg_auth_members m
JOIN pg_roles r ON r.oid = m.member
JOIN pg_roles g ON g.oid = m.roleid
WHERE g.rolname LIKE 'papel\_%'
      AND r.rolname <> current_user
ORDER BY r.rolname;
```

Para retirar uma permissão, usa-se **REVOKE**. Por exemplo, **REVOKE INSERT ON item_pedido FROM papel_vendedor;** impede que o grupo continue inserindo itens. Cada decisão de acesso vira um comando e uma linha no seu plano de controle de acesso.

Conceito: criptografia e hash

Técnica	O que faz	Reversível ?	Uso típico
Hash	Transforma um texto em um código de tamanho fixo.	Não	Guardar senhas ; verificar integridade.
Criptografia (simétrica)	Embaralha o dado com uma chave ; com a chave, desembaralha.	Sim, com a chave	Proteger um dado que precisa ser lido depois (por exemplo, um telefone).

No PostgreSQL, a extensão **pgcrypto** oferece essas funções:

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;
```

Hash simples: o mesmo texto sempre dá o mesmo resultado.

```
SELECT encode(sha256('senha123'::bytea), 'hex') AS hash_sem_sal;
```

Isso **não é seguro para senhas**: duas pessoas com a mesma senha teriam o mesmo hash, e senhas comuns podem ser descobertas por tabelas prontas. Para senhas, usa-se hash **com sal** (um valor aleatório adicionado), como o que as funções **crypt** e **gen_salt** fazem:

```
CREATE TABLE exp_usuario (
  login      text PRIMARY KEY,
  senha_hash text NOT NULL
);

INSERT INTO exp_usuario VALUES ('ana', crypt('minha_senha_forte', gen_salt('bf')));
INSERT INTO exp_usuario VALUES ('bruno', crypt('minha_senha_forte', gen_salt('bf')));
```

Duas pessoas, mesma senha, **hashes diferentes**:

```
SELECT login,
       senha_hash,
       senha_hash = (SELECT senha_hash FROM exp_usuario WHERE login = 'ana') AS igual_ao_da_ana
FROM exp_usuario;
```

Para **verificar** uma senha, recalcula-se o hash usando o hash guardado como sal:

```
SELECT login,
       senha_hash = crypt('minha_senha_forte', senha_hash) AS senha_certa,
       senha_hash = crypt('outra_senha',      senha_hash) AS senha_errada
FROM exp_usuario
WHERE login = 'ana';
```

Criptografia com chave: o dado é guardado ilegível e só é recuperado com a chave.

```
SELECT pg_typeof(pgp_sym_encrypt('27 99999-0001', 'chave_de_estudo')) AS tipo_guardado;

SELECT pgp_sym_decrypt(pgp_sym_encrypt('27 99999-0001', 'chave_de_estudo'),
                       'chave_de_estudo') AS telefone_recuperado;
```

Atenção

Neste exercício a chave aparece escrita no comando. Em um sistema real, a **chave nunca fica no código nem no histórico de comandos**: ela é guardada separadamente, em um serviço apropriado. Além disso, o banco já **protege os dados em trânsito e em disco** conforme o ambiente (por exemplo, conexão cifrada no Supabase); confira a documentação do serviço que você usa.

Conceito: segurança em nível de linha (RLS)

Até aqui, as permissões são sobre **tabelas e colunas**. A **segurança em nível de linha** (*Row Level Security*, RLS) decide **quais linhas** cada usuário enxerga. Ela é ativada na tabela e funciona com **políticas**.

Experimento: uma tabela de anotações em que cada pessoa só vê as **suas**.

```
CREATE TABLE exp_anotacao (
  id      integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  autor text NOT NULL DEFAULT current_user,
  texto text NOT NULL
);
```

```
GRANT SELECT, INSERT ON exp_anotacao TO papel_vendedor;
```

```
ALTER TABLE exp_anotacao ENABLE ROW LEVEL SECURITY;
```

```
CREATE POLICY ver_proprias ON exp_anotacao
  FOR SELECT USING (autor = current_user);
```

```
CREATE POLICY inserir_proprias ON exp_anotacao
  FOR INSERT WITH CHECK (autor = current_user);
```

```
SET ROLE ana_vendas;
INSERT INTO exp_anotacao (texto) VALUES ('Cliente pediu desconto no pão.');
```

```
SET ROLE bruno_vendas;
INSERT INTO exp_anotacao (texto) VALUES ('Conferir validade do iogurte.');
```

```
SELECT autor, texto FROM exp_anotacao;

RESET ROLE;
SELECT autor, texto FROM exp_anotacao ORDER BY id;
```

O Bruno vê **só a sua** anotação. O administrador, que é dono da tabela, vê as duas (o dono não é limitado pela política, salvo configuração especial). Por fim, o Bruno tenta escrever em nome da Ana:

Este comando deve gerar erro (é o esperado)

```
SET ROLE bruno_vendas;
INSERT INTO exp_anotacao (autor, texto) VALUES ('ana_vendas', 'Tentando me passar pela Ana.');
```

Dica: RLS no Supabase

No Supabase, as tabelas do esquema **public** podem ser acessadas também pela **API automática** do projeto. Por isso, o painel recomenda **ativar o RLS em todas as tabelas** do esquema **public**: com o RLS ativo e **nenhuma política**, os acessos pela API ficam **negados por padrão**. A conexão direta com o usuário **postgres** (como a que você usa no **psql**) não é afetada. Para as tabelas do Mercadinho:

```
ALTER TABLE categoria    ENABLE ROW LEVEL SECURITY;
ALTER TABLE fornecedor   ENABLE ROW LEVEL SECURITY;
ALTER TABLE produto      ENABLE ROW LEVEL SECURITY;
ALTER TABLE cliente      ENABLE ROW LEVEL SECURITY;
ALTER TABLE pedido       ENABLE ROW LEVEL SECURITY;
ALTER TABLE item_pedido  ENABLE ROW LEVEL SECURITY;
```

Confira no painel do seu projeto (Table Editor e Security Advisor) como o Supabase mostra o estado do RLS; os detalhes da plataforma podem mudar com o tempo.

Limpeza do experimento

Para repetir o experimento, ou deixar o ambiente organizado, remova os objetos de teste. Mantenha os papéis e as views se quiser usá-los na entrega final.

```
DROP TABLE IF EXISTS exp_anotacao;
DROP TABLE IF EXISTS exp_usuario;
```

Ação

Em dupla, elabore o **Plano de Controle de Acesso** do Mercadinho:

1. **Preencha o quadro de permissões** (papel × tabela × permissões) com base no quadro "O que precisa fazer", do início da aula. Inclua um quarto perfil, "**Caixa**", que só pode consultar produtos e registrar itens de pedido, **definindo você mesmos** as permissões.
2. **Implemente** em SQL os papéis e privilégios (**03_scripts/14_seguranca.sql**), no PostgreSQL local e depois no Supabase.
3. **Teste** cada papel com **SET ROLE**: dois testes que **funcionam** e dois **negados** por papel. Registre o resultado em uma tabela.
4. **Gere a matriz de permissões** (parte 4) e anexe ao plano.
5. **Preencha a matriz de ameaças e controles** para o Mercadinho, com pelo menos cinco linhas. Marque quais controles são do banco e quais são de **processo** (por exemplo, "não compartilhar senhas").
6. **No Supabase**, ative o RLS nas tabelas do projeto e anote o que mudou no painel.

Reflexão

1. O que acontece quando o vendedor executa **SELECT *** em **cliente**? Por que isso é bom do ponto de vista da segurança, mesmo sendo incômodo?
2. Por que criar **grupos** (papéis) e colocar pessoas neles, em vez de dar permissões a cada pessoa?

3. Qual a diferença entre **hash** e **criptografia**? Para o campo CPF, qual dos dois você usaria? E para a senha de um usuário? Por quê?
4. A view mascarada e o privilégio por coluna resolvem o mesmo problema? Quando você escolheria um ou outro?
5. Qual controle da sua matriz de ameaças é **mais difícil de garantir** só com o banco? Quem precisa participar?
6. Que dados do Mercadinho são **dados pessoais**? O que mudaria no seu plano se um cliente pedisse para ver ou excluir os dados dele?

Nova ação

1. **Revise o plano** com o que a Reflexão revelou e a avaliação do professor (por exemplo, um privilégio a mais do que o necessário).
2. **Acrescente ao script** o **REVOKE** de uma permissão e teste de novo, comprovando que ela deixou de funcionar.
3. **Ajuste a exportação da aula 20**: a exportação de clientes para o marketing também precisa de controle de acesso? Descreva (em **exportacao.md**) quem pode gerá-la e onde ela é guardada.
4. Escreva uma **regra de senhas e credenciais** de uma página para a equipe do Mercadinho, com linguagem clara para quem não é da área técnica.

Desafio

O dono quer que a vendedora **Ana** só veja os **pedidos que ela mesma registrou**. Descreva o que precisa mudar na tabela **pedido** (qual coluna, qual política) e escreva o script, testando com **SET ROLE**. Se algo do modelo precisar mudar, atualize o modelo e o dicionário de dados antes.

Síntese e entrega

- Segurança protege **confidencialidade, integridade e disponibilidade**; e começa em perguntar **de quê** e **de quem** os dados precisam ser protegidos.
- Controle de acesso: **papéis, privilégio mínimo**, privilégios por **coluna, views** e **RLS**.
- Senhas: **hash com sal**; dados que precisam ser lidos de novo: **criptografia**, com gestão cuidadosa da chave.
- Todo controle precisa ser **testado**: mostre o que funciona e o que é negado.

Evidência desta aula

Plano de Controle de Acesso, **14_seguranca.sql**, registro dos testes por papel, matriz de permissões e matriz de ameaças e controles. Observação com checklist; contribui para o indicador I5 (planejamento de proteção) e para o valor V4 (sigilo).

Aula 22 – Backup: planejamento e execução

Nesta aula

- **Etapa:** 3 – Movimentar, proteger e entregar
- **Indicadores:** I5 (principal)
- **Conhecimentos:** C3 (redundância, disponibilidade, falhas, backup e restore)
- **Habilidades:** H1, H3, H4 · **Atitudes/valores:** V3, V4
- **Você vai entregar:** o Plano de Backup do Mercadinho e um backup gerado, nomeado e verificado

No caso do Mercadinho

Na semana passada, um estagiário apagou a planilha de preços e ninguém tinha cópia. O dono pergunta: "Se o banco de dados quebrar, ou se alguém apagar uma tabela por engano, **em quanto tempo** voltamos ao normal, e **quanto do trabalho** perdemos?"

Para responder, não basta saber o comando de backup. É preciso **planejar**: o que copiar, com que frequência, para onde, por quanto tempo guardar, quem faz e como se sabe que a cópia **funciona**. Esse é o conteúdo do indicador I5: **planejar rotinas de backup e restore**.

Conceito: falhas e backup

Os dados podem ser perdidos por muitos motivos. Cada tipo de falha pede uma proteção:

Falha	Exemplo	Proteção
Erro humano	DELETE sem WHERE ; tabela apagada	Backup + restore; transações; permissões mínimas (aula 21)
Falha de equipamento	Disco queimado; computador roubado	Cópia em outro equipamento ou local
Falha de software	Atualização que corrompe dados	Backup anterior à atualização
Ataque	Sequestro de dados (<i>ransomware</i>)	Cópias isoladas da rede principal
Desastre	Incêndio, enchente no escritório	Cópia fora do local

Três ideias do conteúdo de segurança (C3) aparecem aqui:

- **Redundância:** manter mais de uma cópia dos dados.
- **Disponibilidade:** o dado precisa estar acessível quando necessário; o backup permite **recuperar** o acesso após uma falha.
- **Falhas:** o plano precisa prever o que fazer **quando** (e não se) algo der errado.

Tipos de backup

Tipo	Como funciona	Observação
Lógico	Gera um arquivo com os comandos/dados para recriar o banco (é o que o pg_dump faz).	Simples, portátil entre máquinas e versões próximas; é o tipo usado nesta UC.
Físico	Copia os arquivos do servidor de banco de dados.	Mais rápido para bancos grandes; exige conhecimento de administração do servidor.

E, quanto à **quantidade** de dados copiados em cada rodada:

Estratégia	Cópia	Vantagem	Desvantagem
Completo	Tudo, toda vez	Restauração simples	Ocupa mais espaço e tempo
Incremental	Só o que mudou desde a última cópia (completa ou incremental)	Rápido e pequeno	Restauração exige toda a cadeia
Diferencial	Só o que mudou desde a última completa	Restauração mais simples que a incremental	Cresce a cada dia

Complementação didática

O `pg_dump` gera sempre um backup **completo** (lógico). As estratégias incremental e diferencial dependem de ferramentas de backup físico e arquivamento contínuo, que **não** são estudadas aqui; elas aparecem apenas para você entender os termos que encontrará no mercado. Para um banco do tamanho do Mercadinho, o backup lógico completo diário é adequado.

Dois números que guiam o plano: RPO e RTO

Complementação didática

Os termos abaixo não constam do Plano de Curso, mas são usados por profissionais da área para transformar a pergunta do dono em metas mensuráveis.

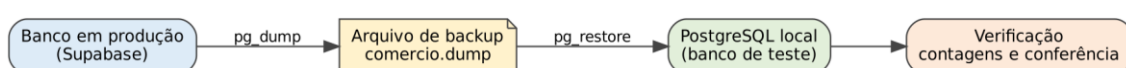
Sigla	Pergunta	Exemplo
RPO (<i>Recovery Point Objective</i>)	Quanto de dado podemos perder? É o intervalo máximo aceitável entre o último backup e a falha.	"Podemos perder no máximo as vendas de 1 dia."
RTO (<i>Recovery Time Objective</i>)	Em quanto tempo o banco precisa voltar?	"O sistema deve voltar em até 2 horas."

O **RPO** define a **frequência** do backup: se o RPO é de 24 horas, é preciso um backup pelo menos diário. O **RTO** define a **estratégia de restauração** e o quanto ela precisa ser ensaiada e rápida. Metas rígidas custam mais; por isso o plano deve ser **combinado com o dono do negócio**.

Regra 3-2-1

Uma regra simples e muito usada:

- **3** cópias dos dados (a original e duas de backup);
- em **2** tipos diferentes de armazenamento (por exemplo, disco do computador e disco externo);
- com **1** cópia **fora do local** (por exemplo, em nuvem).



Rota das cópias de segurança do Mercadinho

Um backup que nunca foi testado não é backup

A parte mais importante do plano é **testar a restauração** (aula 23). Muita gente só descobre que o backup estava corrompido, incompleto ou protegido por senha esquecida **no dia do desastre**.

Demonstração

Parte 1 – O backup lógico com `pg_dump`. O formato **custom** (`-Fc`) gera um arquivo **compactado**, que não se lê em editor de texto, mas permite restaurar **seletivamente** com o `pg_restore`.

No terminal

```
pg_dump -h localhost -U comercio_app -d comercio -n public -Fc --no-owner --no-privileges -f comercio_backup.dump
```

Opção	Significado
<code>-h, -U, -d</code>	Servidor, usuário e banco (como no <code>psql</code>).
<code>-n public</code>	Apenas o esquema <code>public</code> (as tabelas do projeto).
<code>-Fc</code>	Formato custom (compactado).
<code>--no-owner, --no-privileges</code>	Não leva donos e permissões do ambiente de origem.
<code>-f</code>	Nome do arquivo de saída.

Para o **Supabase**, use a conexão do *session pooler* (aula 2). A versão do `pg_dump` no seu computador não pode ser mais antiga que a do servidor:

No terminal

```
pg_dump "postgresql://postgres.[REF]@[HOST-DO-POOLER]:5432/postgres" -n public -Fc --no-owner --no-privileges -f supabase_backup.dump
```

Parte 2 – Verificar o arquivo gerado. O `pg_restore --list` lê o índice do backup **sem restaurar nada**. Se o arquivo estiver corrompido, o comando acusa erro:

No terminal

```
pg_restore --list comercio_backup.dump
```

Procure, na lista, as seis tabelas, as views e os dados (`TABLE DATA`). Um teste rápido de integridade, que só mostra o resultado se der certo:

No terminal

```
pg_restore --list comercio_backup.dump > /dev/null && echo "Arquivo de backup legível"
```

(No Windows PowerShell: `pg_restore --list comercio_backup.dump | Out-Null; if ($?) { "Arquivo de backup legível" }.`)

Parte 3 – Nome e organização dos arquivos. Um nome bom diz **o quê, de onde e quando**. Sugestão de padrão:

```
comercio_AAAA-MM-DD_HHMM.dump      exemplo: comercio_2026-10-05_2200.dump
```

E uma pasta organizada (valor H3, organizar arquivos):

```
backups/
  mercadinho/
    diario/      <- um por dia (guardar os últimos 7)
    semanal/    <- um por semana (guardar os últimos 4)
    mensal/     <- um por mês (guardar os últimos 6)
```

Parte 4 – Um script de backup repetível. Em vez de digitar o comando toda vez, guarde-o em um script. Este exemplo é para Linux ou macOS (`backup_comercio.sh`):

No terminal

```
#!/usr/bin/env bash
# Backup lógico do banco "comercio" (PostgreSQL local)
set -euo pipefail
```

```

DESTINO="$HOME/backups/mercadinho/diario"
ARQUIVO="$DESTINO/comercio_$(date +%F_%H%M).dump"

mkdir -p "$DESTINO"

pg_dump -h localhost -U comercio_app -d comercio \
        -n public -Fc --no-owner --no-privileges -f "$ARQUIVO"

# verifica se o arquivo é legível
pg_restore --list "$ARQUIVO" > /dev/null

echo "Backup gerado e verificado: $ARQUIVO"
ls -l "$ARQUIVO"

```

Equivalente para Windows (PowerShell, `backup_comercio.ps1`):

```

$destino = "$env:USERPROFILE\backups\mercadinho\diario"
$arquivo = "$destino\comercio_$(Get-Date -Format 'yyyy-MM-dd_HH:mm').dump"
New-Item -ItemType Directory -Force -Path $destino | Out-Null

pg_dump -h localhost -U comercio_app -d comercio -n public -Fc --no-owner --no-privileges -f $arquivo
pg_restore --list $arquivo | Out-Null

"Backup gerado e verificado: $arquivo"

```

Atenção: a senha

O script não contém senha. O `pg_dump` vai **pedir** a senha na hora. Para rodar sem intervenção (por exemplo, agendado), existem duas alternativas: a variável `PGPASSWORD` ou o arquivo de senhas `.pgpass` (`pgpass.conf` no Windows), que deve ter permissão restrita ao seu usuário. **Nunca** escreva a senha dentro do script que será compartilhado ou guardado em repositório, e nunca a envie por chat ou e-mail. Em ambientes reais, o ideal é uma conta de backup **só com os privilégios necessários** (aula 21).

Parte 5 – Agendamento. Backups que dependem de alguém lembrar **falham**. Sistemas operacionais têm agendadores de tarefas (`cron` no Linux e macOS, Agendador de Tarefas no Windows) que executam o script no horário definido. Neste curso você **descreve** o agendamento no plano, sem ser obrigado a configurá-lo. Exemplo de linha do `cron`, que roda o script todos os dias às 22h:

```
0 22 * * * /home/usuario/scripts/backup_comercio.sh
```

Parte 6 – E o backup do Supabase? Os projetos no Supabase têm recursos próprios de backup, que **dependem do plano contratado** e mudam com o tempo. Em um plano gratuito, você **não deve contar** com um backup automático que atenda às metas do seu plano. Consulte a documentação e o painel do seu projeto e **registre o que ele oferece** (frequência, retenção, como restaurar). Independentemente disso, o que é **seu e importante** deve ter um backup **que você controla**, como o `pg_dump` da parte 1.

Plano de Backup: o modelo

Um plano é um documento curto e objetivo. Use esta estrutura:

Item	O que responder
1. O que proteger	Quais bancos e quais dados; grau de sigilo.
2. Metas	RPO e RTO acordados, e por quê.
3. Tipo e ferramenta	Backup lógico completo com <code>pg_dump -Fc</code> .
4. Frequência e horário	Diário às 22h (fora do horário de vendas), por exemplo.

Item	O que responder
5. Retenção	Quantas cópias e por quanto tempo (diário, semanal, mensal).
6. Onde guardar	Locais, seguindo a regra 3-2-1.
7. Responsável	Quem executa, quem confere, quem substitui.
8. Verificação	Como e quando se confere o arquivo e se testa a restauração.
9. Segurança do backup	Quem acessa os arquivos; como são protegidos (um backup tem todos os dados, inclusive CPFs).
10. Procedimento de restauração	Passo a passo (aula 23) e quem decide restaurar.
11. Revisão	Quando o plano é revisado.

Ação

Em dupla:

1. Gere um backup do banco local com o `pg_dump -Fc` e verifique-o com `pg_restore --list`. Anote o tamanho do arquivo e quanto tempo levou.
2. Gere um backup do Supabase, se a conexão permitir, e verifique-o.
3. Escreva o script `backup_comercio.sh` (ou `.ps1`), ajustando para o seu ambiente, e execute-o duas vezes. Confira se os dois arquivos receberam nomes diferentes.
4. Elabore o Plano de Backup do Mercadinho (`06_documentacao/plano_backup.md`), preenchendo todos os itens do modelo. Para as metas, use este cenário e justifique os números: *o Mercadinho vende das 8h às 20h; cada dia de vendas perdido precisa ser refeito a partir de notas em papel; a loja não pode ficar sem sistema por mais de meio período.*
5. Registre o que o painel do Supabase oferece de backup no seu plano e como isso entra (ou não) no seu plano.

Reflexão

1. Qual **RPO** e qual **RTO** você escolheu? Que frequência de backup decorre do RPO escolhido? Se o dono pedisse RPO de 1 hora, o que mudaria?
2. Em que a regra 3-2-1 protegeria o Mercadinho de cada uma das falhas da tabela do início da aula?
3. O arquivo de backup tem **todos** os dados, inclusive CPFs. Quem deve poder abri-lo? O que você faria se um pen drive com backup fosse perdido?
4. Por que o `pg_restore --list` não prova que o backup **funciona**? O que falta provar?
5. O que há de **risco** em depender só do backup que a plataforma de nuvem oferece?

Nova ação

1. Revise o Plano de Backup com base no feedback do professor e de outra dupla.
2. Troque o plano com outra dupla: cada uma deve dizer, por escrito, onde o plano **falhari**a em um dia de desastre (um ponto cego por plano).
3. Aplique a retenção: simule uma semana de backups com nomes por data e mostre quais seriam apagados pela regra de retenção escolhida.
4. Atualize `objetos_do_banco.md` e a estrutura de pastas (`04_dados`, `05_backup`, `06_documentacao`) com o que foi acrescentado nesta aula (na pasta `05_backup` ficam os backups de estudo; em uma situação real, eles ficariam em local próprio e protegido).

Desafio

Os dados do Mercadinho crescem 10% a cada mês. Estime, para um ano, o espaço necessário para a retenção que você definiu (diário, semanal e mensal), partindo do tamanho do backup atual. O plano continua cabendo no disco que você previu? O que mudaria?

Síntese e entrega

- **Planejar** é responder: o quê, quando, onde, por quanto tempo, quem e **como testar**.
- **RPO** define a frequência; **RTO** define a velocidade de recuperação.
- `pg_dump -Fc` gera um backup lógico completo; `pg_restore --list` confere o arquivo.
- Backup contém dados sensíveis: **proteja-o** como protege o banco.

Evidência desta aula

`plano_backup.md`, o backup do banco local (e, se possível, do Supabase) com verificação, o script de backup e a simulação de retenção. **Menção parcial do indicador I5 (A/PA/NA)**, com feedback do professor.

Aula 23 – Restore: restauração e teste de recuperação

Nesta aula

- **Etapa:** 3 – Movimentar, proteger e entregar
- **Indicadores:** I5 (principal) · I4 (apoio) · I3 (apoio)
- **Conhecimentos:** C3 (falhas, disponibilidade, backup e restore); C2 (importação e exportação)
- **Habilidades:** H1, H3, H4 · **Atitudes/valores:** V2, V3
- **Você vai entregar:** restauração testada e conferida, com registro de tempo e de dados perdidos, e o Plano de Backup revisado

No caso do Mercadinho

Hoje é o dia do teste que ninguém quer fazer: o dono pediu uma **simulação de desastre**. Em um banco de ensaio, a equipe vai provocar um acidente e restaurar o banco a partir do backup. Só depois do teste o plano da aula 22 passa a valer de verdade: *um backup só é confiável depois de restaurado com sucesso*.

Conceito: restaurar

Restaurar (*restore*) é recriar o banco, ou parte dele, a partir de um backup. O programa para arquivos no formato custom é o `pg_restore`:

```
pg_restore -h servidor -U usuario -d banco_de_destino [opções] arquivo.dump
```

Opção	Significado
<code>-d banco</code>	Banco de destino (precisa existir).
<code>-n public</code>	Restaura só o esquema public .
<code>--clean --if-exists</code>	Apaga os objetos existentes antes de recriá-los (substitui o que há no destino).
<code>--no-owner --no-privileges</code>	Ignora donos e permissões do backup.
<code>--list</code>	Lista o conteúdo do backup, sem restaurar.

Três estratégias de restauração

Estratégia	Quando usar	Cuidado
A. Para um banco novo	Testar o backup; investigar um problema sem tocar no banco em uso	Não conserta o banco original; serve para conferir e para recuperar partes
B. Sobre o banco em uso (<code>--clean</code>)	Desastre: o banco original está danificado e precisa voltar ao estado do backup	Tudo o que foi feito depois do backup é perdido (esse é o seu RPO na prática)
C. Parcial (por exemplo, só uma tabela)	Só uma parte foi perdida	Chaves, restrições e views que dependem da tabela podem precisar ser recriadas; confira sempre

Como saber se a restauração deu certo?

Olhar "sem erros" não basta. É preciso **comparar com o original**. Duas conferências complementares:

1. **Contagem** de linhas por tabela.
2. Um **resumo (hash)** do conteúdo de cada tabela: se um único valor for diferente, o hash muda.

```
SELECT 'categoria' AS tabela, count(*) AS linhas,
       md5(string_agg(t::text, '|' ORDER BY id_categoria)) AS hash
FROM categoria t
UNION ALL SELECT 'fornecedor', count(*), md5(string_agg(t::text, '|' ORDER BY id_fornecedor)) FROM
fornecedor t
UNION ALL SELECT 'produto', count(*), md5(string_agg(t::text, '|' ORDER BY id_produto)) FROM
produto t
UNION ALL SELECT 'cliente', count(*), md5(string_agg(t::text, '|' ORDER BY id_cliente)) FROM
cliente t
UNION ALL SELECT 'pedido', count(*), md5(string_agg(t::text, '|' ORDER BY id_pedido)) FROM
pedido t
UNION ALL SELECT 'item_pedido', count(*), md5(string_agg(t::text, '|' ORDER BY id_pedido,
id_produto)) FROM item_pedido t;
```

Cada linha da tabela (**t**) é convertida em texto, as linhas são juntadas na ordem da chave e o conjunto vira um código **md5**. Se duas tabelas têm o **mesmo hash**, seus conteúdos são idênticos.

Guarde essa consulta em **03_scripts/15_baseline.sql**: ela será executada **antes** do acidente e **depois** da restauração.

Demonstração

Atenção

Esta aula **provoca acidentes de propósito**. Faça-a **somente no PostgreSQL local** (ou em um banco de ensaio), **nunca** no banco do Supabase que contém o seu trabalho. Se quiser ensaiar na nuvem, use outro projeto criado apenas para o teste.

Parte 1 – Backup e linha de base. Gere o backup do banco local e registre o estado atual (a "linha de base"):

No terminal

```
pg_dump -h localhost -U comercio_app -d comercio -n public -Fc --no-owner --no-privileges -f
antes_do_acidente.dump
psql -h localhost -U comercio_app -d comercio -f 03_scripts/15_baseline.sql -o baseline_antes.txt
```

A opção **-o** do **psql** grava o resultado em um arquivo. Abra **baseline_antes.txt** e observe as seis linhas.

Parte 2 – Teste de restauração em um banco novo (estratégia A). Como administrador:

Executar como administrador do servidor

```
CREATE DATABASE comercio_restore OWNER comercio_app;
```

No terminal

```
pg_restore -h localhost -U comercio_app -d comercio_restore --no-owner --no-privileges
antes_do_acidente.dump
psql -h localhost -U comercio_app -d comercio_restore -f 03_scripts/15_baseline.sql -o
baseline_restore.txt
```

O **pg_restore** termina com **errors ignored on restore: 1**. O erro é esperado: o backup tenta criar o esquema **public**, que já existe em todo banco novo. Confira se os dois arquivos de linha de base são **idênticos**:

No terminal

```
diff baseline_antes.txt baseline_restore.txt
```

(No Windows: **fc baseline_antes.txt baseline_restore.txt**.) Sem diferenças, o backup está **comprovadamente restaurável**. Esse é o primeiro resultado do seu teste.

Parte 3 – O acidente. Primeiro, simule uma venda **depois** do backup, pois ela mostrará o que se perde entre o backup e o desastre:

Acidente simulado: execute somente no ambiente de teste

```
INSERT INTO cliente (nome, cpf) VALUES ('Cliente pós-backup', '10000000099');
```

Agora, dois acidentes no banco **comercio**: o reajuste que esqueceu o **WHERE** (e foi gravado, sem transação) e a exclusão de uma tabela:

Acidente simulado: execute somente no ambiente de teste

```
UPDATE produto SET preco_venda = preco_venda * 100;
DROP TABLE item_pedido CASCADE;
```

Observe o estrago:

Acidente simulado: execute somente no ambiente de teste

```
SELECT nome, preco_venda FROM produto ORDER BY id_produto LIMIT 3;
SELECT table_name FROM information_schema.views WHERE table_schema = 'public' ORDER BY 1;
```

Os preços estão absurdos e as views que dependiam de **item_pedido** desapareceram junto com a tabela (**CASCADE**). A equipe de dados agora tem um incidente.

Parte 4 – Antes de restaurar, preserve as evidências. Gere um backup do banco **no estado danificado**. Ele pode ser útil para investigar o que aconteceu e para recuperar algum dado que só existe nele (por exemplo, o cliente pós-backup):

No terminal

```
pg_dump -h localhost -U comercio_app -d comercio -n public -Fc --no-owner --no-privileges -f
danificado.dump
```

Parte 5 – Restauração sobre o banco em uso (estratégia B). Execute e meça o tempo:

No terminal

```
time pg_restore -h localhost -U comercio_app -d comercio -n public --clean --if-exists --no-owner --
no-privileges antes_do_acidente.dump
```

(No Windows PowerShell: **Measure-Command { pg_restore ... }**.) O tempo medido é uma **evidência para o seu RTO**. Em um banco pequeno como este, são frações de segundo; em um banco real, seria muito maior.

Parte 6 – Conferência.

No terminal

```
psql -h localhost -U comercio_app -d comercio -f 03_scripts/15_baseline.sql -o baseline_depois.txt
diff baseline_antes.txt baseline_depois.txt
```

Os arquivos devem ser **idênticos**: tabelas, linhas e conteúdos voltaram ao estado do backup. Confira também o que foi **perdido**, e que as views voltaram:

```
SELECT count(*) AS clientes_pos_backup FROM cliente WHERE nome LIKE '%pós-backup%';
SELECT table_name FROM information_schema.views WHERE table_schema = 'public' ORDER BY 1;
```

O "Cliente pós-backup" **não existe mais**: é o preço de restaurar sobre o banco. Esse intervalo entre o último backup e o acidente é o **RPO real** da sua rotina. Se ele for maior do que o aceitável para o Mercadinho, o plano precisa de backups mais frequentes. No caso, a evidência de que a venda existiu está no **danificado.dump**, e poderia ser recuperada com a estratégia A ou C.

Ação

Em dupla, **execute a simulação completa** e preencha o **Relatório de Teste de Restauração** ([06_documentacao/teste_restauracao.md](#)):

Item	Registro
Data, ambiente e responsáveis	
Backup usado (nome, data, tamanho)	
Acidente provocado (o que, quando, como foi percebido)	
Estratégia de restauração escolhida e por quê	
Comandos executados (na ordem)	
Tempo de restauração medido	
Resultado da conferência (contagem e hash, antes × depois)	
Dados perdidos entre o backup e o acidente	
Problemas encontrados e como foram resolvidos	
Ajustes necessários no Plano de Backup	

Siga as partes 1 a 6 e **troque** os acidentes por outros de sua escolha, desde que tenham ao menos um acidente de **dados** (como o **UPDATE** sem **WHERE**) e um de **estrutura** (como o **DROP TABLE**).

Reflexão

1. Por que fazer a restauração de teste em um **banco novo** antes de restaurar sobre o banco em uso?
2. Por que o hash é uma conferência melhor do que a contagem de linhas? Dê um exemplo de erro que a contagem **não** perceberia.
3. Por que foi importante gerar o **danificado.dump** antes de restaurar?
4. O tempo medido é o seu **RTO**? O que mais entraria no tempo real de uma recuperação (por exemplo, decidir restaurar, localizar o arquivo, avisar a equipe)?
5. A **perda** dos dados pós-backup foi aceitável segundo o seu RPO? O que você mudaria no plano?
6. Que **atitudes** da equipe foram decisivas na simulação (cordialidade ao comunicar, flexibilidade para mudar de estratégia, proatividade para registrar)?

Nova ação

1. **Revise o Plano de Backup** (aula 22) com o que o teste mostrou: RPO e RTO reais, frequência, retenção e responsabilidades. Registre a **versão** e o motivo das mudanças.
2. **Escreva o procedimento de restauração** em formato de lista de verificação, de modo que uma pessoa que **não participou do teste** consiga executá-lo, incluindo quando **não** restaurar.
3. **Repita o teste**, desta vez com outra pessoa da equipe executando o procedimento, sem ajuda. Anote onde ela hesitou e ajuste o texto.

Desafio

Recupere **apenas os preços originais** dos produtos, **sem** restaurar o banco inteiro (estratégia C). Dica: restaure o backup em **comercio_restore**; exporte os preços de lá para um arquivo CSV (**\copy**, aula 20);

carregue o arquivo em uma tabela de preparação no banco `comercio` (aula 19) e use um `UPDATE ... FROM` conferindo antes com um `SELECT` e dentro de uma transação. Escreva o que **essa** estratégia consegue e **não** consegue recuperar, em comparação com a estratégia B.

Complementação didática: restaurar uma tabela isolada

O `pg_restore` aceita `-t tabela` para restaurar uma única tabela. Nos testes feitos para esta apostila, a tabela e seus dados voltaram, mas as **chaves primária e estrangeiras** e as **views** que dependiam dela precisaram ser **recriadas** depois. Se você experimentar, confira com `\d tabela` o que veio e o que não veio.

Síntese e entrega

- Backup só é confiável depois de **restaurado e conferido**.
- Estratégias: **A** (banco novo: testar), **B** (sobre o banco: desastre, com perda do que veio depois do backup) e **C** (parcial: exige conferência).
- A conferência compara **contagens e hashes**: antes × depois.
- O teste produz **evidências** (tempo e dados perdidos) para ajustar o plano.

Evidência desta aula

Relatório de Teste de Restauração, `15_baseline.sql`, arquivos `baseline_antes/depois.txt`, Plano de Backup revisado e procedimento de restauração. **Menção parcial do indicador I5 (A/PA/NA)**, com observação do professor (confirma ou atualiza o resultado da aula 22).

Aula 24 – Integração e entrega do pacote técnico

Nesta aula

- **Etapa:** 3 – Movimentar, proteger e entregar
- **Indicadores:** I1 · I2 · I3 · I4 · I5 (verificação final)
- **Conhecimentos:** C1, C2 e C3 (integração)
- **Habilidades:** H1, H2, H3, H4 · **Atitudes/valores:** V1, V2, V3, V4
- **Você vai entregar:** o pacote técnico completo e a apresentação técnica, com demonstração

No caso do Mercadinho

Chegou o dia de entregar ao dono do Mercadinho o que a equipe construiu: um banco de dados **modelado, implementado, populado, consultável, protegido e recuperável**. Ele não é da área técnica. Quer ver funcionando e entender, em linguagem clara, o que mudou na vida da loja. E quer ter certeza de que, se você sair da equipe amanhã, **outra pessoa consegue manter o banco**, porque tudo está documentado.

Esta aula não traz conteúdo novo. Ela **integra** o que você construiu nas aulas 1 a 23, verifica a qualidade do conjunto e leva você a **explicar e justificar** suas decisões.

O pacote técnico

O pacote reúne os sete itens do desafio da UC. Cada item tem **critérios observáveis**: o que o professor vai olhar.

#	Item	Indicador	Critérios observáveis
1	Diagramas e dicionário de dados	I1	O DER e o modelo físico correspondem ao banco construído; o dicionário descreve todas as colunas (tipo, regra, descrição); a aferição (banco × diagrama) está registrada; as versões estão anotadas.
2	Scripts SQL versionados	I3	Scripts numerados, comentados e executáveis em ordem em um banco vazio; cobrem construção, inserção, manipulação, consultas, views e índices.
3	Banco implementado (Supabase e local)	I2 · I3	O banco existe nos dois ambientes; a instalação local está documentada com passos e versão; as configurações de acesso estão registradas sem senhas .
4	Importação e exportação	I4	Procedimentos documentados, com validação, rejeitadas registradas e conferência de contagens; formatos descritos.
5	Política de acesso	I3 · (C3)	Papéis e privilégios definidos e implementados; testes que funcionam e negados; matriz de permissões e de ameaças.
6	Plano de backup e restore, com teste	I5	Plano completo (RPO, RTO, frequência, retenção, local, responsável); relatório do teste de restauração com conferência.
7	Apresentação técnica	H4 · V1 · V2	Explica o problema, as decisões e os resultados em linguagem adequada; demonstra o funcionamento; responde às perguntas com termos técnicos corretos.

Antes de entregar: verificação final do banco

Rode o script abaixo no **banco de cada ambiente** (Supabase e local) e compare o resultado com o seu **dicionário de dados** e com o seu **diagrama**. Ele é a "prova de vida" do banco.

```
SELECT 'tabelas' AS objeto, count(*) AS quantidade
FROM information_schema.tables
WHERE table_schema = 'public' AND table_type = 'BASE TABLE'
UNION ALL
SELECT 'views', count(*)
FROM information_schema.views
WHERE table_schema = 'public'
UNION ALL
SELECT 'índices', count(*)
FROM pg_indexes
WHERE schemaname = 'public'
UNION ALL
SELECT 'chaves primárias', count(*)
FROM pg_constraint
WHERE connamespace = 'public'::regnamespace AND contype = 'p'
UNION ALL
SELECT 'chaves estrangeiras', count(*)
FROM pg_constraint
WHERE connamespace = 'public'::regnamespace AND contype = 'f'
UNION ALL
SELECT 'restrições CHECK', count(*)
FROM pg_constraint
WHERE connamespace = 'public'::regnamespace AND contype = 'c'
UNION ALL
SELECT 'restrições UNIQUE', count(*)
FROM pg_constraint
WHERE connamespace = 'public'::regnamespace AND contype = 'u';
```

Verifique também a **integridade dos dados** com consultas que **não devem** retornar nenhuma linha:

```
-- pedidos sem itens
SELECT p.id_pedido
FROM pedido p
LEFT JOIN item_pedido i ON i.id_pedido = p.id_pedido
WHERE i.id_pedido IS NULL;
```

```
-- itens com quantidade ou preço inválidos
SELECT id_pedido, id_produto
FROM item_pedido
WHERE quantidade <= 0 OR preco_unitario <= 0;
```

```
-- produtos ativos com estoque negativo
SELECT id_produto, nome
FROM produto
WHERE ativo AND estoque_atual < 0;
```

Se alguma consulta retornar linhas, **corrija antes** da apresentação ou **registre** a explicação.

E a lista de **quem pode o quê**, que você fará acompanhar a política de acesso:

```
SELECT r.rolname AS papel,
       r.rolcanlogin AS faz_login
FROM pg_roles r
WHERE r.rolname LIKE 'papel\_%'
ORDER BY r.rolname;
```

(Se você removeu os papéis de teste, o resultado é vazio: confirme que isso é o esperado e que sua política de acesso está documentada.)

O roteiro da apresentação técnica

A apresentação dura **de 10 a 12 minutos por dupla**, mais perguntas. A plateia é o "dono do Mercadinho" (o professor e a turma). **Os dois integrantes falam.**

Tempo	Etapas	O que mostrar
1 min	O problema	A situação das planilhas e o que o dono precisava.
2 min	O modelo	O DER e duas decisões de modelagem justificadas (por exemplo, a normalização de pedido/itens e a escolha de tipos).
2 min	O banco funcionando	Uma consulta com junção e agregação; uma view; uma inserção rejeitada por restrição.
1 min	Ambientes	Supabase e local: o que cada um faz e como foi configurado (sem mostrar senhas).
2 min	Entrada e saída de dados	Importação com validação e rejeitadas; uma exportação.
1 min	Segurança	Um acesso permitido e um negado .
2 min	Backup e restore	O plano em uma página e o resultado do teste de restauração.
1 min	Lições e próximos passos	O que a dupla faria diferente; o que ficou pendente.

Dica

Ensaie **uma vez** com o relógio e **abra todos os arquivos antes**. Tenha um plano B para a demonstração: prints ou um vídeo curto dos passos, caso a conexão falhe. Nunca mostre senhas na tela: oculte-as ou use um arquivo de configuração fora da tela compartilhada.

Perguntas que o "dono" pode fazer

- Por que vocês separaram **pedido** e **item de pedido** em duas tabelas?
- O que impede alguém de cadastrar um produto com preço negativo?
- Se um funcionário apagar a tabela de pedidos por engano, o que acontece? Quanto tempo leva para voltar e quanto se perde?
- Por que o vendedor não vê o CPF dos clientes?
- Como vocês sabem que o backup funciona?
- O que muda se a loja abrir uma segunda unidade?

Ação

Em dupla:

1. **Monte o pacote técnico** nas pastas do projeto (**bom-preco-db/**), conforme a estrutura de pastas apresentada no início da apostila. Escreva um **LEIA-ME.md** na raiz, explicando o conteúdo de cada pasta e a **ordem de execução dos scripts** em um banco vazio.
2. **Teste a reprodutibilidade:** em um banco local **novo e vazio**, execute os scripts na ordem do **LEIA-ME.md**. O banco deve ficar igual ao entregue (verificação final + contagens). Corrija o que falhar.
3. **Execute a verificação final** (consultas desta aula) no Supabase e no local, e compare os resultados com o dicionário.
4. **Prepare a apresentação** e a demonstração, dividindo as falas entre os dois integrantes.
5. **Apresente** à turma, no horário definido pelo professor, e **responda às perguntas**.

Reflexão

Preencha a **autoavaliação** abaixo. Para cada critério, indique se alcançou (**S**, **Em parte**, **N**) e qual evidência prova isso (nome do arquivo ou comando).

Critério observável	Indicador	Alcancei?	Evidência
O DER, o modelo físico e o dicionário correspondem ao banco construído.	I1		
Os scripts executam em ordem em um banco vazio, sem erro.	I3		
Consultas, views e índices atendem às necessidades do Mercadinho.	I3		
O PostgreSQL local foi instalado e configurado, e a instalação está documentada.	I2		
O banco no Supabase está acessível e as configurações estão registradas sem senhas.	I2		
A importação valida os dados e registra o que foi rejeitado.	I4		
A exportação entrega o formato combinado, sem dados sigilosos desnecessários.	I4		
A política de acesso foi implementada e testada (permitido e negado).	I3 · C3		
O plano de backup define RPO, RTO, frequência, retenção e responsável.	I5		
A restauração foi testada e conferida com evidências.	I5		
Usei termos técnicos corretamente na apresentação.	H4		
Trabalhei em dupla com cordialidade, flexibilidade e sigilo.	V1 · V2 · V4		

Depois, responda:

1. Qual indicador você considera **mais forte** no seu trabalho? Qual evidência prova isso?
2. Qual indicador está **mais frágil**? O que falta para fortalecê-lo?
3. Que decisão do projeto você **mudaria** se começasse hoje?
4. Em que momento da UC você precisou de **flexibilidade** (V2) ou **proatividade** (V3)? O que fez?

Nova ação

Recuperação imediata. Se o professor apontou um critério **não alcançado** ou **parcialmente alcançado**, o momento de corrigir é **agora**, enquanto o conteúdo está fresco. Para cada ponto:

1. Entenda o que a evidência deveria mostrar.
2. Corrija o trabalho (script, documento ou demonstração).
3. Entregue a **nova evidência** e peça uma nova observação.

Sem pendências, aproveite para **organizar o pacote final** e arquivar uma cópia (sem senhas) do seu projeto.

Desafio

Escreva uma **carta de uma página ao dono do Mercadinho**, sem termos técnicos que ele não entenda, explicando: o que foi construído, o que ele ganhou com isso, o que **ele precisa fazer** para manter os dados

seguros (senhas, backups, quem acessa o quê) e **o que fazer se algo der errado**. Cada termo técnico indispensável (como *backup* ou *restauração*) deve vir **explicado** em uma frase.

Síntese e entrega

- Um banco de dados profissional é **modelo + implementação + dados + acesso + recuperação + documentação**.
- A prova de qualidade é a **reprodutibilidade**: alguém que não participou consegue montar e manter o banco com o seu pacote.
- Apresentar bem é **explicar as decisões** e demonstrar com evidências.

Evidência desta aula

Pacote técnico completo (itens 1 a 7), **LEIA-ME.md**, resultado da verificação final nos dois ambientes, apresentação com demonstração e autoavaliação. **Menção final por indicador: A (alcançado) ou NA (não alcançado)**, e resultado final da UC: **D** (desenvolveu) ou **ND** (não desenvolveu).

Apêndices

Apêndice A – Guia rápido de SQL e psql

Consulte este guia durante as aulas. Ele resume os comandos usados na apostila; as aulas indicadas explicam cada um.

A.1 Comandos do psql (não são SQL)

Comando	Para que serve
<code>\conninfo</code>	Mostra a conexão atual (banco, usuário, servidor).
<code>\l</code>	Lista os bancos de dados.
<code>\c banco</code>	Conecta a outro banco.
<code>\dt</code>	Lista as tabelas.
<code>\d tabela</code>	Mostra as colunas, chaves e restrições de uma tabela.
<code>\dv · \di</code>	Lista as views · os índices.
<code>\du</code>	Lista os papéis (usuários e grupos).
<code>\timing</code>	Liga/desliga a medição do tempo dos comandos.
<code>\i arquivo.sql</code>	Executa um arquivo de script.
<code>\o arquivo</code>	Grava os resultados em um arquivo (<code>\o</code> sozinho volta ao normal).
<code>\copy ...</code>	Importa/exporta CSV (aulas 19 e 20).
<code>\?</code>	Ajuda dos comandos do <code>psql</code> .
<code>\q</code>	Sai do <code>psql</code> .

A.2 Definição de dados (DDL) – aulas 9 e 10

```
CREATE TABLE t (
    id      integer GENERATED BY DEFAULT AS IDENTITY,
    nome    varchar(60) NOT NULL,
    preco   numeric(10,2) NOT NULL DEFAULT 0,
    id_cat  integer NOT NULL,
    CONSTRAINT pk_t PRIMARY KEY (id),
    CONSTRAINT uq_t_nome UNIQUE (nome),
    CONSTRAINT ck_t_preco CHECK (preco >= 0),
    CONSTRAINT fk_t_cat FOREIGN KEY (id_cat) REFERENCES categoria (id_categoria)
);

ALTER TABLE t ADD COLUMN c tipo;
ALTER TABLE t DROP COLUMN c;
ALTER TABLE t ALTER COLUMN c TYPE novo_tipo;
ALTER TABLE t ADD CONSTRAINT nome ...;
ALTER TABLE t DROP CONSTRAINT nome;
DROP TABLE t;
```

A.3 Manipulação de dados (DML) e transações – aula 11

```

INSERT INTO t (c1, c2) VALUES (v1, v2), (v3, v4);
INSERT INTO t (c1, c2) SELECT ... ;
UPDATE t SET c1 = v WHERE condição;          -- sem WHERE atinge TODAS as linhas
DELETE FROM t WHERE condição;                -- sem WHERE apaga TODAS as linhas
... RETURNING colunas;                       -- devolve as linhas afetadas

BEGIN;      -- inicia a transação
COMMIT;     -- confirma
ROLLBACK;   -- desfaz

```

A.4 Consultas (DQL) – aulas 12 a 16

```

SELECT [DISTINCT] colunas ou expressões [AS apelido]
FROM tabela [apelido]
[JOIN outra ON condição]          -- INNER, LEFT, RIGHT, FULL
[WHERE condição]                  -- filtra linhas
[GROUP BY colunas]                -- agrupa
[HAVING condição sobre agregados] -- filtra grupos
[ORDER BY coluna [DESC]]
[LIMIT n];

```

Tema	Elementos
Comparação	=, <>, <, >, <=, >=, BETWEEN a AND b, IN (...), LIKE 'A%', ILIKE, IS NULL, IS NOT NULL
Lógicos	AND, OR, NOT
Agregação	count(*), count(c), sum, avg, min, max
Texto	upper, lower, trim, length, substr, (concatenar), replace
Números	round(x, n), abs, ceil, floor
Datas	CURRENT_DATE, now(), to_char(data, 'YYYY-MM'), extract(month FROM data), data + 7
Nulos e condições	coalesce(a, b), CASE WHEN ... THEN ... ELSE ... END
Conjuntos	UNION, UNION ALL, INTERSECT, EXCEPT
Subconsultas	WHERE c IN (SELECT ...), WHERE EXISTS (SELECT ...), FROM (SELECT ...) AS x, WITH nome AS (SELECT ...)

A.5 Views e índices – aula 17

```

CREATE VIEW vw_nome AS SELECT ... ;
CREATE OR REPLACE VIEW vw_nome AS SELECT ... ;
DROP VIEW vw_nome;

CREATE INDEX idx_nome ON tabela (coluna);
DROP INDEX idx_nome;
EXPLAIN ANALYZE SELECT ... ;

```

A.6 Segurança – aula 21

```

CREATE ROLE grupo NOLOGIN;
CREATE ROLE pessoa LOGIN PASSWORD '...' IN ROLE grupo;
GRANT SELECT, INSERT ON tabela TO papel;
GRANT SELECT (c1, c2) ON tabela TO papel;
REVOKE INSERT ON tabela FROM papel;
SET ROLE papel; RESET ROLE;
ALTER TABLE t ENABLE ROW LEVEL SECURITY;
CREATE POLICY nome ON t FOR SELECT USING (condição);

```

A.7 Programas de linha de comando (fora do psql)

Programa	Uso	Aulas
psql	Cliente de linha de comando: <code>psql -h servidor -U usuario -d banco</code>	2, 3, todas
pg_dump	Gera backup/dump: <code>pg_dump -d banco -n public -Fc -f arquivo.dump</code>	20, 22
pg_restore	Restaura um backup custom: <code>pg_restore -d banco arquivo.dump</code>	23

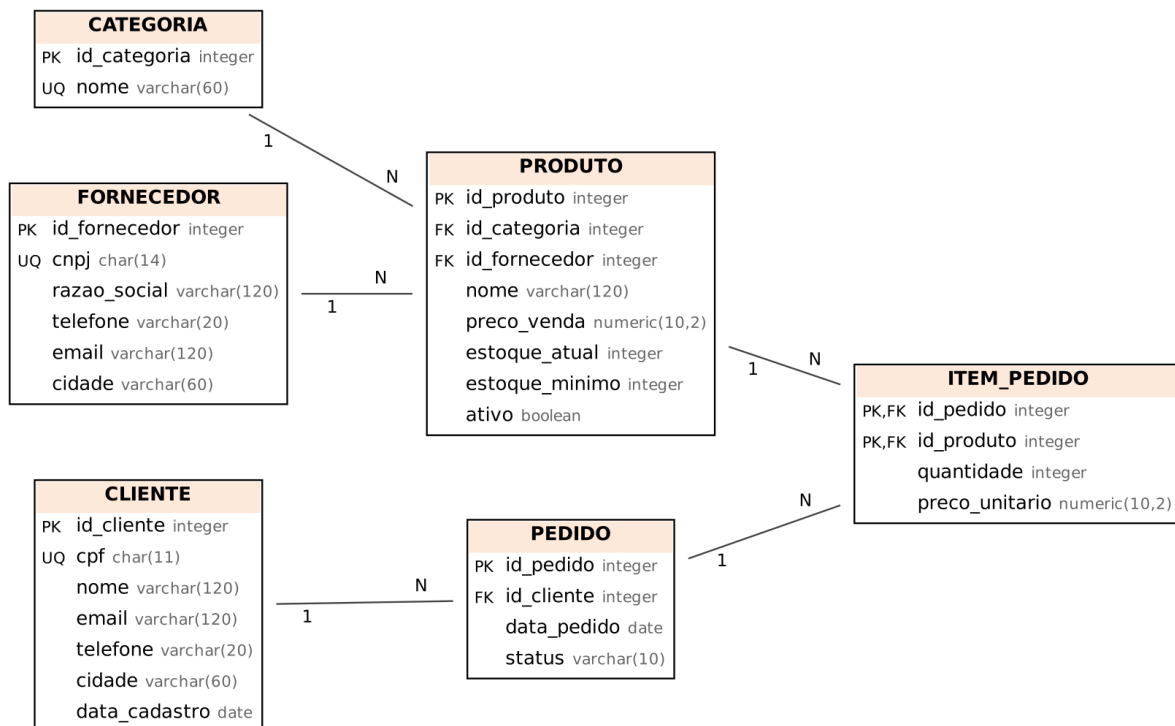
A.8 Erros comuns e o que significam

Mensagem (resumo)	Provável causa
<code>syntax error at or near ...</code>	Erro de escrita perto do ponto indicado: vírgula faltando, palavra errada, parêntese aberto.
<code>relation "x" does not exist</code>	A tabela não existe (nome errado, ou o script anterior não rodou).
<code>column "x" does not exist</code>	Nome de coluna errado ou faltando apelido da tabela.
<code>null value in column "x" violates not-null constraint</code>	Tentou gravar vazio em coluna obrigatória.
<code>duplicate key value violates unique constraint</code>	Valor repetido em chave primária ou coluna única.
<code>violates foreign key constraint</code>	Referência a um registro que não existe, ou apagar um registro que ainda é referenciado.
<code>violates check constraint</code>	O valor não respeita a regra (CHECK).
<code>column "x" must appear in the GROUP BY clause</code>	Coluna do SELECT que não está agrupada nem agregada.
<code>permission denied for table x</code>	O papel atual não tem o privilégio necessário.
<code>invalid byte sequence for encoding "UTF8"</code>	O arquivo importado não está em UTF-8.

Apêndice B – Modelo de referência do Mercadinho

Este apêndice traz o modelo **como ele deve estar ao final da etapa 2** (antes da mudança de requisito da aula 18). Use-o para **conferir** o seu trabalho, e não para copiá-lo: o valor da UC está em você ter chegado ao modelo pelo raciocínio das aulas 4 a 10. Se o seu modelo é diferente, isso **não** é necessariamente um erro; verifique se as diferenças são **justificadas** no seu dicionário de dados.

B.1 Diagrama de entidade e relacionamento (modelo físico)



Modelo de referência do Mercadinho

B.2 Script de criação das tabelas

Este script cria as seis tabelas, já com todas as restrições e a coluna **codigo_barras** (a soma dos scripts das aulas 9 e 10). Em um banco vazio, ele produz a mesma estrutura das aulas.

```

CREATE TABLE categoria (
    id_categoria integer GENERATED BY DEFAULT AS IDENTITY,
    nome          varchar(60) NOT NULL,
    CONSTRAINT pk_categoria PRIMARY KEY (id_categoria),
    CONSTRAINT uq_categoria_nome UNIQUE (nome)
);

CREATE TABLE fornecedor (
    id_fornecedor integer GENERATED BY DEFAULT AS IDENTITY,
    razao_social  varchar(120) NOT NULL,
    cnpj          char(14) NOT NULL,
    telefone      varchar(20),
    email         varchar(120),
    cidade        varchar(60),
    CONSTRAINT pk_fornecedor PRIMARY KEY (id_fornecedor),
    CONSTRAINT uq_fornecedor_cnpj UNIQUE (cnpj),
    CONSTRAINT ck_fornecedor_cnpj CHECK (cnpj ~ '^[0-9]{14}$')
);
  
```

```

CREATE TABLE produto (
    id_produto      integer GENERATED BY DEFAULT AS IDENTITY,
    nome            varchar(120) NOT NULL,
    id_categoria    integer      NOT NULL,
    id_fornecedor   integer      NOT NULL,
    preco_venda     numeric(10,2) NOT NULL,
    estoque_atual   integer      NOT NULL DEFAULT 0,
    estoque_minimo  integer      NOT NULL DEFAULT 5,
    ativo           boolean      NOT NULL DEFAULT true,
    codigo_barras   varchar(14),
    CONSTRAINT pk_produto PRIMARY KEY (id_produto),
    CONSTRAINT uq_produto_codigo_barras UNIQUE (codigo_barras),
    CONSTRAINT fk_produto_categoria FOREIGN KEY (id_categoria)
        REFERENCES categoria (id_categoria),
    CONSTRAINT fk_produto_fornecedor FOREIGN KEY (id_fornecedor)
        REFERENCES fornecedor (id_fornecedor),
    CONSTRAINT ck_produto_preco CHECK (preco_venda > 0),
    CONSTRAINT ck_produto_estoque CHECK (estoque_atual >= 0 AND estoque_minimo >= 0)
);

CREATE TABLE cliente (
    id_cliente      integer GENERATED BY DEFAULT AS IDENTITY,
    nome            varchar(120) NOT NULL,
    cpf             char(11) NOT NULL,
    email           varchar(120),
    telefone        varchar(20),
    cidade          varchar(60),
    data_cadastro   date          NOT NULL DEFAULT CURRENT_DATE,
    CONSTRAINT pk_cliente PRIMARY KEY (id_cliente),
    CONSTRAINT uq_cliente_cpf UNIQUE (cpf),
    CONSTRAINT ck_cliente_cpf CHECK (cpf ~ '^[0-9]{11}$'),
    CONSTRAINT ck_cliente_email CHECK (email IS NULL OR email LIKE '%@%')
);

CREATE TABLE pedido (
    id_pedido       integer GENERATED BY DEFAULT AS IDENTITY,
    id_cliente      integer      NOT NULL,
    data_pedido     date          NOT NULL DEFAULT CURRENT_DATE,
    status          varchar(10) NOT NULL DEFAULT 'aberto',
    CONSTRAINT pk_pedido PRIMARY KEY (id_pedido),
    CONSTRAINT fk_pedido_cliente FOREIGN KEY (id_cliente)
        REFERENCES cliente (id_cliente),
    CONSTRAINT ck_pedido_status CHECK (status IN ('aberto', 'pago', 'cancelado'))
);

CREATE TABLE item_pedido (
    id_pedido       integer      NOT NULL,
    id_produto      integer      NOT NULL,
    quantidade      integer      NOT NULL,
    preco_unitario  numeric(10,2) NOT NULL,
    CONSTRAINT pk_item_pedido PRIMARY KEY (id_pedido, id_produto),
    CONSTRAINT fk_item_pedido FOREIGN KEY (id_pedido)
        REFERENCES pedido (id_pedido) ON DELETE CASCADE,
    CONSTRAINT fk_item_produto FOREIGN KEY (id_produto)
        REFERENCES produto (id_produto),
    CONSTRAINT ck_item_quantidade CHECK (quantidade > 0),
    CONSTRAINT ck_item_preco CHECK (preco_unitario > 0)
);

```

B.3 Índices e views

Os índices da aula 17 e as views das aulas 17 e 20. A view `vw_cliente_mascarado` (aula 21) não está aqui porque faz parte do plano de acesso.

```

-- Índices
CREATE INDEX idx_pedido_cliente      ON pedido (id_cliente);
CREATE INDEX idx_pedido_data        ON pedido (data_pedido);
CREATE INDEX idx_item_pedido_produto ON item_pedido (id_produto);
CREATE INDEX idx_produto_categoria  ON produto (id_categoria);

```

```

CREATE INDEX idx_produto_fornecedor ON produto (id_fornecedor);

-- Views
CREATE VIEW vw_itens_vendidos AS
SELECT p.id_pedido, p.data_pedido, p.status,
       c.nome AS cliente,
       pr.nome AS produto,
       cat.nome AS categoria,
       i.quantidade, i.preco_unitario,
       i.quantidade * i.preco_unitario AS subtotal
FROM item_pedido i
JOIN pedido p ON p.id_pedido = i.id_pedido
JOIN cliente c ON c.id_cliente = p.id_cliente
JOIN produto pr ON pr.id_produto = i.id_produto
JOIN categoria cat ON cat.id_categoria = pr.id_categoria;

CREATE VIEW vw_total_pedido AS
SELECT p.id_pedido, p.data_pedido, p.status,
       c.nome AS cliente,
       sum(i.quantidade * i.preco_unitario) AS total
FROM pedido p
JOIN cliente c ON c.id_cliente = p.id_cliente
JOIN item_pedido i ON i.id_pedido = p.id_pedido
GROUP BY p.id_pedido, p.data_pedido, p.status, c.nome;

CREATE VIEW vw_estoque_critico AS
SELECT pr.id_produto, pr.nome, pr.estoque_atual, pr.estoque_minimo,
       pr.estoque_minimo - pr.estoque_atual AS falta,
       f.razao_social AS fornecedor,
       f.telefone AS telefone_fornecedor,
       f.email AS email_fornecedor
FROM produto pr
JOIN fornecedor f ON f.id_fornecedor = pr.id_fornecedor
WHERE pr.ativo
      AND pr.estoque_atual < pr.estoque_minimo;

CREATE VIEW vw_vendas_mensais AS
SELECT to_char(p.data_pedido, 'YYYY-MM') AS mes,
       count(DISTINCT p.id_pedido) AS pedidos,
       sum(i.quantidade * i.preco_unitario) AS faturamento
FROM pedido p
JOIN item_pedido i ON i.id_pedido = p.id_pedido
WHERE p.status = 'pago'
GROUP BY to_char(p.data_pedido, 'YYYY-MM');
```

B.4 Totais para conferência dos dados de carga

Depois de executar o script de carga da aula 11 (e **antes** da importação da aula 19), você deve obter:

Tabela	Linhas
categoria	7
fornecedor	7
produto	25
cliente	14
pedido	24
item_pedido	70

O faturamento total de pedidos **pagos** é de **R\$ 1.555,58** (agosto: R\$ 596,92; setembro: R\$ 958,66). Depois da importação completa da aula 19, são **34 produtos** e **8 categorias**.

Apêndice C – De Oracle para PostgreSQL: equivalências

O Plano de Curso cita o **SQL Plus** e o **SQL Developer**, ferramentas do banco de dados **Oracle**. Nesta UC, por decisão didática, você pratica em **PostgreSQL**. Os conceitos (tabelas, chaves, SQL, backup) são os mesmos; mudam as ferramentas e alguns detalhes. Este apêndice ajuda quem vai encontrar o Oracle no mercado.

Complementação didática

Este apêndice **não faz parte** do Plano de Curso. Ele serve para que a decisão didática pelo PostgreSQL não deixe você desconectado das ferramentas citadas no Plano. As equivalências são aproximadas: confirme detalhes na documentação de cada SGBD.

C.1 Ferramentas

Oracle (citado no Plano)	Equivalente nesta UC	Observação
SQL Plus (cliente de linha de comando)	psql	Ambos executam SQL e comandos próprios do cliente.
SQL Developer (cliente gráfico)	pgAdmin, DBeaver ou o SQL Editor do Supabase	Interface gráfica para consultar e administrar.
Exportar/importar com utilitários próprios	pg_dump e pg_restore	Backup lógico.
Carga de arquivos externos	\copy (psql)	Importação/exportação de CSV.

C.2 Comandos do cliente

SQL Plus (Oracle)	psql (PostgreSQL)
DESCRIBE tabela	\d tabela
EXIT	\q
@arquivo.sql	\i arquivo.sql
SPOOL arquivo	\o arquivo
SET TIMING ON	\timing

C.3 Tipos de dados

Oracle	PostgreSQL	Observação
NUMBER(10,2)	numeric(10,2)	Valores monetários.
NUMBER(10)	integer ou bigint	Números inteiros.
VARCHAR2(n)	varchar(n)	Texto de tamanho limitado.
CLOB	text	Texto longo.
DATE (inclui hora)	timestamp ; o date do PostgreSQL não tem hora	Atenção à diferença.
SYSDATE	now() ou CURRENT_TIMESTAMP	Data e hora atuais.

C.4 Funções e comandos SQL

Oracle	PostgreSQL
<code>NVL(a, b)</code>	<code>coalesce(a, b)</code> (o <code>COALESCE</code> também existe no Oracle)
<code>MINUS</code>	<code>EXCEPT</code>
<code>ROWNUM <= n</code>	<code>LIMIT n</code> (ambos aceitam <code>FETCH FIRST n ROWS ONLY</code>)
<code>SELECT ... FROM DUAL</code>	<code>SELECT ...</code> (não precisa de tabela)
Chave automática por sequência (e gatilho) ou <code>GENERATED ... AS IDENTITY</code> (versões recentes)	<code>GENERATED BY DEFAULT AS IDENTITY</code>
<code>TO_CHAR, TO_DATE</code>	<code>to_char, to_date</code> (existem nos dois)
Concatenação <code> </code>	<code> </code> (igual)

Atenção

Uma diferença conhecida: no **Oracle**, uma string vazia (`' '`) é tratada como `NULL`; no **PostgreSQL**, `' '` e `NULL` são valores **diferentes**. Isso muda o comportamento de filtros e validações.

Apêndice D – Modelos de documentos

Modelos para você copiar e preencher. Eles são sugestões didáticas; o professor pode ajustar os campos.

D.1 Dicionário de dados (uma tabela por documento)

Tabela: _____ Descrição: _____ Versão: _____

Coluna	Tipo	Obrigatória?	Chave	Regra / restrição	Descrição
			PK / FK / UQ		

D.2 Histórico de versões do modelo

Versão	Data	Autores	O que mudou e por quê
v1.0			Primeira versão do modelo.

D.3 Checklist de ambiente (aulas 2 e 3)

Item	Supabase	PostgreSQL local	Evidência
Ambiente criado/instalado			
Versão do PostgreSQL			Resultado de <code>SELECT version();</code>
Banco do projeto criado			
Conexão testada pelo <code>psql</code> e por cliente gráfico			
Acessos registrados sem senha			
Problemas encontrados e solução			

D.4 Registro de objetos do banco (objetos_do_banco.md)

Tipo	Nome	Finalidade	Script que o cria	Versão
Tabela				
View				
Índice				
Papel				

D.5 Cabeçalho padrão de script SQL

```
-- =====
-- Projeto : Mercadinho Bom Preço
-- Script  : 01_criar_tabelas.sql
-- Objetivo: criar as tabelas do banco
-- Autores : (nomes da dupla)
-- Data    : AAAA-MM-DD      Versão: 1.0
-- Executar em: banco vazio, após conectar em "comercio"
-- Depende de: (nenhum)
-- =====
```

D.6 Plano de importação / exportação

Item	Importação	Exportação
Origem / destino		
Formato (separador, codificação, decimais, datas)		
Regras de validação		
Tratamento de erros (rejeitadas)		
Dados sigilosos e como são protegidos		
Conferência (contagens, totais)		
Responsável		

D.7 Plano de controle de acesso

Papel	Quem pertence	Tabelas / colunas	Privilégios	Justificativa

D.8 Plano de backup e restore

Use os 11 itens do modelo da aula 22. Para o relatório do teste, use a tabela da aula 23.

D.9 Roteiro de autoavaliação por indicador

Indicador	Evidência que tenho	Evidência que falta	Ação
I1			
I2			
I3			
I4			
I5			

Apêndice E – Glossário

Termo	Significado
Aferição	Conferência do banco construído contra o diagrama e o dicionário de dados.
Agregação	Cálculo sobre um conjunto de linhas (soma, média, contagem).
Atributo	Característica de uma entidade (por exemplo, o nome do produto). Vira coluna na tabela.
Autenticação	Prova de identidade, normalmente por usuário e senha.
Backup	Cópia dos dados para recuperação em caso de perda.
Cardinalidade	Quantidade de ocorrências que se relacionam entre duas entidades (1:1, 1:N, N:N).
Chave candidata	Atributo (ou conjunto) que poderia identificar uma linha.
Chave estrangeira (FK)	Coluna que referencia a chave primária de outra tabela.
Chave primária (PK)	Coluna (ou conjunto) que identifica cada linha de forma única.
Consistência	Garantia de que o banco passa de um estado válido a outro estado válido.
CSV	Arquivo de texto com valores separados por um caractere.
DCL	Linguagem de controle de dados: GRANT , REVOKE .
DDL	Linguagem de definição de dados: CREATE , ALTER , DROP .
DER	Diagrama de entidade e relacionamento.
Dicionário de dados	Documento que descreve cada tabela e coluna do banco.
DML	Linguagem de manipulação de dados: INSERT , UPDATE , DELETE .
DQL	Linguagem de consulta: SELECT .
Dump	Arquivo gerado pelo pg_dump com o conteúdo do banco.
Entidade	Coisa do mundo real sobre a qual guardamos dados (cliente, produto).
Hash	Código de tamanho fixo calculado a partir de um dado; não se reverte.
Índice	Estrutura que acelera a busca de linhas.
Integridade referencial	Regra que impede referências a registros que não existem.
Junção (join)	Combinação de linhas de duas ou mais tabelas por uma condição.
Normalização	Organização das tabelas para reduzir repetição e inconsistência.
Papel (role)	Identidade de acesso no PostgreSQL: pessoa, aplicação ou grupo.
Privilegio mínimo	Dar a cada papel só as permissões necessárias.
Restauração (restore)	Recuperar o banco a partir de um backup.
RLS	Segurança em nível de linha: políticas que definem quais linhas cada papel vê.
RPO	Perda máxima de dados aceitável (intervalo desde o último backup).
RTO	Tempo máximo aceitável para recuperar o serviço.
SGBD	Sistema gerenciador de banco de dados (como o PostgreSQL).
Staging (preparação)	Área onde os dados importados são validados antes de entrar nas tabelas definitivas.
Subconsulta	Consulta dentro de outra consulta.
Transação	Conjunto de comandos executado por inteiro ou desfeito por inteiro.
View	Consulta com nome, guardada no banco.

Apêndice F – Referências

Bibliografia do Plano de Curso (UC 5 – Desenvolver banco de dados)

As referências abaixo constam do Plano de Curso e foram reproduzidas conforme ele as apresenta.

Bibliografia básica

- PEREIRA, Paloma Cristina. *Introdução a banco de dados*. São Paulo: Ed. Senac, 2021.
- MACHADO, F. N. R.; ABREU, M. P. D. *Projeto de banco de dados*. 17. ed. São Paulo: Saraiva, 2017. E-book.
- ALBUQUERQUE, Alexander Gobbato Paulino; REIS, Luiz Carlos. *Administração de banco de dados*. São Paulo: Ed. Senac, 2021.

Bibliografia complementar

- CARDOSO, V. M.; CARDOSO, G. C. *Linguagem SQL*. São Paulo: Saraiva, 2013. E-book.
- ELMASRI, R.; NAVATHE, S. B. *Sistemas de banco de dados*. 7. ed. São Paulo: Pearson Education do Brasil, 2019.
- GOYA, Milton. *Arquitetura de banco de dados*. São Paulo: Ed. Senac, 2021.

Documentação técnica usada como apoio (não consta do Plano de Curso)

Estas fontes complementam a apostila e **não substituem** a bibliografia do Plano. Consulte sempre a versão mais atual.

- **PostgreSQL Documentation** (documentação oficial do PostgreSQL, versões 16 em diante): <https://www.postgresql.org/docs/>.
- **Supabase Docs** (documentação da plataforma Supabase): <https://supabase.com/docs>. As funcionalidades e planos da plataforma mudam com o tempo; confira o painel do seu projeto.
- **BRASIL. Lei nº 13.709, de 14 de agosto de 2018** (Lei Geral de Proteção de Dados Pessoais – LGPD). Citada como contexto na aula 21.

Nota sobre o conteúdo

Os exemplos, os dados (fictícios), os scripts e os exercícios desta apostila foram **elaborados como material didático** para esta UC, a partir dos indicadores, conhecimentos, habilidades e atitudes/valores do Plano de Curso. Elementos que vão além do Plano estão identificados como **complementação didática**.